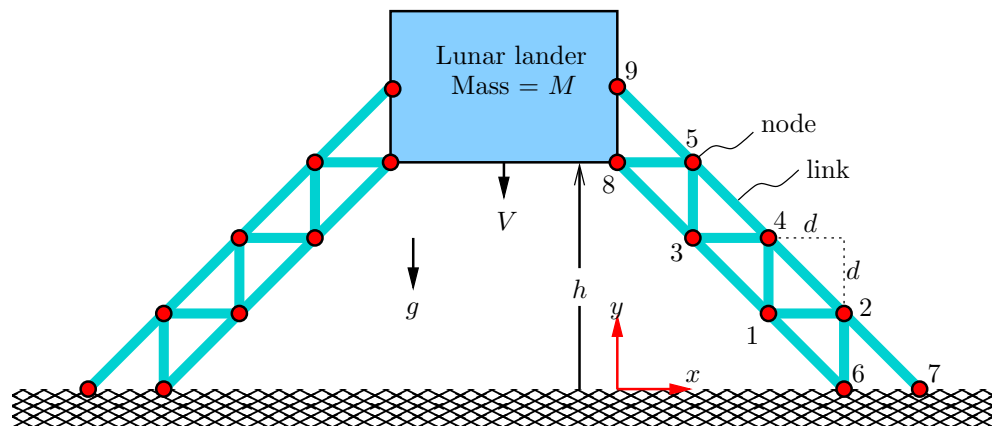


Computational Methods for Aerospace Engineering

Aero 423 Course Notes: Winter 2023



Krzysztof Fidkowski
University of Michigan, Ann Arbor, MI, 48109, USA

January 4, 2025

Contents

1	Mathematics Review	7
1.1	Linear Algebra	7
1.1.1	Vectors and Matrices	7
1.1.2	Matrix Systems	8
1.1.3	Eigenvalues and Eigenvectors	9
1.1.4	Sample Systems in Aerospace Engineering	11
1.2	Differential Equations	14
1.2.1	Classification of PDEs	15
1.2.2	Initial and Boundary Conditions	15
1.2.3	Sample Equations in Aerospace Engineering	15
1.3	Norms and Function Spaces	17
2	ODE Integration	19
2.1	Introduction	19
2.1.1	The Forward Euler Method	20
2.1.2	The Predictor-Corrector Method	21
2.2	Multi-Step Methods	24
2.3	Accuracy and Consistency	26
2.3.1	Truncation Error	26
2.3.2	Consistency	28
2.3.3	Global Error and Convergence	29
2.4	Stability	31
2.4.1	Zero Stability	31
2.4.2	Eigenvalue Stability	32
2.4.3	Implications for Convergence	37
2.5	Multi-Stage Methods	37
2.5.1	Predictor-Corrector Methods	38
2.5.2	Runge-Kutta Methods	39
2.6	Stiffness and Implicit Methods	42
2.6.1	Stiffness	42
2.6.2	Implicit Methods for Linear Systems	44
2.6.3	Implicit Methods for Nonlinear Systems	49

3	Finite Difference Methods	55
3.1	Finite Difference Approximations	55
3.2	Truncation Error Analysis	56
3.3	General Finite-Difference Formulas	59
3.3.1	Undetermined coefficients	59
3.3.2	Lagrange interpolation	61
3.4	Problem Setup and Boundary Conditions	62
3.5	Multiple Dimensions	66
3.6	Stability Analysis	69
3.6.1	Von-Neumann Analysis	70
3.6.2	Matrix Stability	73
4	Probabilistic Methods	77
4.1	Probability Review	77
4.1.1	Outcomes and Events	77
4.1.2	The Meaning of Probability	77
4.1.3	Random Variables	78
4.1.4	The Probability Density Function (PDF)	78
4.1.5	Expected Value and Mean	79
4.1.6	Variance and Standard Deviation	79
4.1.7	The Cumulative Distribution Function (CDF)	79
4.1.8	Percentiles	79
4.2	Monte Carlo Sampling	80
4.2.1	Introduction	80
4.2.2	Sampling a Distribution	82
4.2.3	Multiple Inputs	85
4.3	Error Estimates for Monte Carlo	87
4.3.1	Estimator of the Mean	87
4.3.2	Standard Error of an Estimator	89
5	Model Reduction	93
5.1	Introduction	93
5.1.1	Motivation	93
5.1.2	Overview	94
5.2	Data Compression	95
5.2.1	Introduction	95
5.2.2	Snapshots and the Singular Value Decomposition	96
5.2.3	Flowfield Example	97
5.3	Equation Projection	101
5.3.1	Introduction	101
5.3.2	Linear Projection	101
5.3.3	Parametrized Systems	102
5.3.4	Offline-Online split	102

5.3.5	Unsteady problems	103
5.4	Interpolation Surrogates	104
5.4.1	Polynomial Fitting	104
5.4.2	Kriging	105
5.4.3	Artificial Neural Networks	105
5.4.4	Example of Surrogates	108
6	Finite Element Methods	111
6.1	Function Approximation	111
6.1.1	Finite-Dimensional Function Spaces	111
6.1.2	Local Support	112
6.2	The Equations	114
6.2.1	Collocation	114
6.2.2	Weighted Residuals	115
6.3	Numerical Integration	117
6.4	The Finite Element Method in One Dimension	119
6.4.1	Mesh and Basis Functions	120
6.4.2	The Reference Element	120
6.4.3	The Weak Form	121
6.4.4	Boundary Conditions	122
6.4.5	Matrix Assembly and Solution	123
6.5	The Finite Element Method in Two Dimensions	128
6.5.1	Mesh and Basis Functions	128
6.5.2	The Reference Element	129
6.5.3	The Weak Form	130
6.5.4	Boundary Conditions	130
6.5.5	Matrix Assembly	131
6.6	High-Order Finite Elements	132
6.7	Discontinuous Finite Elements	135
6.7.1	Advection in One Dimension	136
7	Optimization	139
7.1	Introduction	139
7.2	Gradient-Based Methods	139
7.2.1	Steepest Descent	140
7.2.2	Conjugate Gradients	141
7.2.3	Line Search Methods	142
7.2.4	Computing Gradients	143
7.3	Gradient-Free Methods	146
7.3.1	Nelder-Mead/Simplex Method	146
7.3.2	Genetic Algorithms	146

Chapter 1

Mathematics Review

1.1 Linear Algebra

1.1.1 Vectors and Matrices

By $\mathbf{u} \in \mathbb{R}^N$ we mean a vector, usually a column, of N real numbers. We often write

$$\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_N \end{bmatrix} = [u_1 \ u_2 \ \dots \ u_N]^T,$$

where the superscript T denotes the *transpose* operation, which means flipping rows into columns and vice-versa. A matrix $\mathbf{A} \in \mathbb{R}^{M \times N}$ is a two-dimensional array of real numbers, with M rows and N columns,

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \dots & A_{1N} \\ A_{21} & A_{22} & \dots & A_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ A_{M1} & A_{M2} & \dots & A_{MN} \end{bmatrix} \in \mathbb{R}^{M \times N}, \quad \mathbf{A}^T = \begin{bmatrix} A_{11} & A_{21} & \dots & A_{M1} \\ A_{12} & A_{22} & \dots & A_{M2} \\ \vdots & \vdots & \ddots & \vdots \\ A_{1N} & A_{2N} & \dots & A_{MN} \end{bmatrix} \in \mathbb{R}^{N \times M}$$

A *dot* or *inner product* between two column vectors of the same size, $\mathbf{u}, \mathbf{v} \in \mathbb{R}^N$, is

$$\mathbf{u} \cdot \mathbf{v} = \mathbf{u}^T \mathbf{v} = \mathbf{v}^T \mathbf{u} = \sum_{n=1}^N u_n v_n = \text{a scalar.} \quad (1.1)$$

Two vectors are *orthogonal* if their dot product is zero. We can multiply a column vector $\mathbf{u} \in \mathbb{R}^N$ from the left by a matrix $\mathbf{A} \in \mathbb{R}^{M \times N}$. This operation produces a vector $\mathbf{v} \in \mathbb{R}^M$, the entries of which are dot products between $\mathbf{u}$ and the rows of $\mathbf{A}$:

$$\mathbf{v} = \mathbf{A}\mathbf{u} = \begin{bmatrix} A_{11} & A_{12} & \dots & A_{1N} \\ A_{21} & A_{22} & \dots & A_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ A_{M1} & A_{M2} & \dots & A_{MN} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_N \end{bmatrix} = \begin{bmatrix} \sum_{n=1}^N A_{1n} u_n \\ \sum_{n=1}^N A_{2n} u_n \\ \vdots \\ \sum_{n=1}^N A_{Mn} u_n \end{bmatrix} \in \mathbb{R}^M.$$

In a similar way we can multiply matrices, say $\mathbf{A} \in \mathbb{R}^{N \times M}$ and $\mathbf{B} \in \mathbb{R}^{M \times L}$,

$$\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{N \times L}, \quad C_{nl} = \sum_{m=1}^M A_{nm} B_{ml}.$$

Multiplying a column vector and a row vector gives an *outer product*, the result of which is a matrix,

$$\begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_M \end{bmatrix} \begin{bmatrix} u_1 & u_2 & \dots & u_N \end{bmatrix} = \begin{bmatrix} v_1 u_1 & v_1 u_2 & \dots & v_1 u_N \\ v_2 u_1 & v_2 u_2 & \dots & v_2 u_N \\ \vdots & \vdots & \ddots & \vdots \\ v_M u_1 & v_M u_2 & \dots & v_M u_N \end{bmatrix}.$$

The *determinant* of a square matrix, $\det(\mathbf{A})$, is a scalar computed from a systematic combination of products of the matrix entries. For example, for a 2×2 matrix,

$$\det \left(\begin{bmatrix} a & b \\ c & d \end{bmatrix} \right) = ad - bc.$$

The determinant has several properties,

1. $\det(\mathbf{I}) = 1$, where $\mathbf{I}$ is the identity matrix
2. $\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B})$
3. $\det(\mathbf{A}^T) = \det(\mathbf{A})$
4. $\det(\mathbf{A}^{-1}) = 1/\det(\mathbf{A})$

The inverse of a square matrix, $\mathbf{A}^{-1}$, if it exists, is a matrix that multiplies $\mathbf{A}$ to give the identity matrix,

$$\mathbf{AA}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$$

We will sometimes use “Matlab” notation, namely the colon(:), to denote parts of matrices. For example, $\mathbf{A}(i, :)$ denotes the i^{th} row of matrix $\mathbf{A}$.

1.1.2 Matrix Systems

A typical $N \times N$ linear system is

$$\mathbf{Au} = \mathbf{b}. \tag{1.2}$$

Here, $\mathbf{b}$ is a known right-hand-side, and $\mathbf{u}$ is our vector of N unknowns. The solution is

$$\mathbf{u} = \mathbf{A}^{-1}\mathbf{b},$$

and it exists when $\mathbf{A}$ is *invertible*, also called *non-singular*. A matrix is non-singular if and only if the determinant is nonzero. This means that the columns of $\mathbf{A}$ must be *linearly-independent*: i.e. we should not be able to write any column as a linear combination of

the other columns. The same holds for the rows – a nonzero determinant is equivalent to linearly-independent rows.

The *rank* of a matrix, $\text{rank}(\mathbf{A})$ is the number of linearly-independent columns, which is the same as the number of linearly-independent rows. An $N \times N$ matrix must have *full rank* of N to be invertible.

The *null space* of an $N \times N$ matrix $\mathbf{A}$ is the set of all vectors $\mathbf{u}$ for which $\mathbf{A}\mathbf{u} = \mathbf{0}$. The dimension of the null space is $(N - \text{rank}(\mathbf{A}))$, so a full-rank matrix has no null space.

One way to solve Eqn. 1.2 is through a *direct method* such as *Gaussian elimination*. This is a systematic technique for solving for one variable from one equation (in terms of the other variables), and substituting this expression into all of the other equations. The cost of Gaussian elimination of a dense $N \times N$ matrix is $\mathcal{O}(N^3)$. *Pivoting* is a technique for choosing the order in which the variables/equations are visited in order to minimize errors associated with division by small numbers.

When solving systems on a computer, an important consideration is round-off error due to finite *machine precision*. Most programs operate with “double precision” which means that 64 bits (0/1s) are used to encode a real number. This means that many (uncountably infinite) numbers are not representable on a computer. So every time we operate on numbers (e.g. add, multiply, etc.) we potentially introduce a little bit of error. For double-precision operations on $\mathcal{O}(1)$ numbers, these errors are each $\mathcal{O}(10^{-16})$. However, over many operations, these errors can accumulate and be noticeable in the final result. In addition, some (not so good) numerical algorithms can be *unstable*, meaning that errors are amplified exponentially over the operations.

Another way to solve Eqn. 1.2 is through an *iterative method*. Such a method starts with a guess for the solution and improves on that guess at each iteration. Iterations stop when the *residual*,

$$\mathbf{r} \equiv \mathbf{A}\mathbf{u} - \mathbf{b}$$

becomes small in some chosen measure. Iterative methods are often the only option for large *sparse* (mostly zero entries) matrices.

1.1.3 Eigenvalues and Eigenvectors

An *eigenvector* $\mathbf{v} \in \mathbb{R}^N$ of an $N \times N$ matrix $\mathbf{A}$ is a special nonzero vector for which

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}, \tag{1.3}$$

where the scalar λ is the associated *eigenvalue*. Rearranging this equation shows that $\mathbf{v}$ must live in the null space of $(\mathbf{A} - \lambda\mathbf{I})$, so that λ must satisfy

$$\det(\mathbf{A} - \lambda\mathbf{I}) = 0.$$

λ could be zero, in which case the corresponding eigenvector lies in the null space of $\mathbf{A}$ – this also means that the matrix is singular, or not invertible. $\mathbf{v}$ scaled by a nonzero constant is

still an eigenvector, the same one for our purposes. Multiple eigenvectors can have the same eigenvalue, in which case any linear combination of those eigenvectors is also an eigenvector.

Positive definite matrices are ones for which all eigenvalues are real and positive. If the matrix is also symmetric, it is termed *SPD* (symmetric positive definite). These matrices have a complete set of orthogonal eigenvectors.

To be precise, $\mathbf{v}$ defined in Eqn. 1.3 is a *right eigenvector*. Left eigenvectors satisfy $\mathbf{v}^T \mathbf{A} = \lambda \mathbf{v}^T$, so they are right eigenvectors of $\mathbf{A}^T$. A matrix that has a complete set of eigenvectors can be decomposed as

$$\mathbf{A} = \mathbf{R} \mathbf{\Lambda} \mathbf{L} = \mathbf{R} \mathbf{\Lambda} \mathbf{R}^{-1} = \mathbf{L}^{-1} \mathbf{\Lambda} \mathbf{L}, \quad (1.4)$$

where the columns of $\mathbf{R}$ are the right eigenvectors, the rows of $\mathbf{L}$ are the left eigenvectors, and $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_N)$ is a diagonal matrix of eigenvalues on the main diagonal.

Example 1.1 (A 3×3 matrix). Consider the following matrix,

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 2 & 1 \\ -2 & 0 & 2 \end{bmatrix}$$

We can show that $\mathbf{A}$ is not invertible by calculating its determinant,

$$\det(\mathbf{A}) = 1(2 * 2 - 0) - 2(3 * 2 - (-2) * 1) + 3(0 - (-2) * 2) = 0.$$

Since the determinant is zero, the matrix is not invertible. To identify the null space of $\mathbf{A}$, we look for vectors $\mathbf{v} = [a, b, c]^T$ for which

$$\mathbf{A} \mathbf{v} = 0 \quad \Rightarrow \quad \begin{bmatrix} 1 & 2 & 3 \\ 3 & 2 & 1 \\ -2 & 0 & 2 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = 0$$

So the system for a, b, c is

$$\begin{aligned} a + 2b + 3c &= 0 \\ 3a + 2b + c &= 0 \\ -2a + 2c &= 0 \end{aligned}$$

The last of these says that $a = c$, and substituting for a in either the first or second equation shows that $b = -2c$. So the null space consists of multiples of the vector

$$\mathbf{v} = \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}$$

Finally, we can calculate the eigenvalues of $\mathbf{A}$, λ , which must satisfy

$$\begin{aligned} \det(\mathbf{A} - \mathbf{I}\lambda) &= 0 \\ \det \begin{bmatrix} 1 - \lambda & 2 & 3 \\ 3 & 2 - \lambda & 1 \\ -2 & 0 & 2 - \lambda \end{bmatrix} &= 0 \end{aligned}$$

Expanding the determinant, we obtain a polynomial for λ ,

$$\begin{aligned}
(1 - \lambda)((2 - \lambda)^2) - 2(3(2 - \lambda) + 2) + 3(0 - (-2)(2 - \lambda)) &= 0 \\
(1 - \lambda)(4 - 4\lambda + \lambda^2) + 6\lambda - 16 + 12 - 6\lambda &= 0 \\
4 - 4\lambda + \lambda^2 - 4\lambda + 4\lambda^2 - \lambda^3 - 4 &= 0 \\
-8\lambda + 5\lambda^2 - \lambda^3 &= 0 \\
\lambda(\lambda^2 - 5\lambda + 8) &= 0
\end{aligned}$$

The roots of this equation are

$$\lambda_1 = 0, \quad \lambda_2 = \frac{5 - \sqrt{7}i}{2}, \quad \lambda_3 = \frac{5 + \sqrt{7}i}{2}$$

1.1.4 Sample Systems in Aerospace Engineering

Let's look at a few matrix systems that arise in aerospace engineering.

1.1.4.1 Prandtl's Lifting-Line Equation

In Prandtl's Lifting Line (PLL) theory, the lift distribution over the span of a wing is decomposed into sinusoidal modes, and the (non-dimensional) coefficients on these modes, A_n , are the unknowns. The PLL equation at point θ (surrogate for the coordinate $y = -b \cos(\theta)/2$) on the wing reads

$$\frac{4b}{a_0(\theta)c(\theta)} \sum_{n=1}^N A_n \sin n\theta + \sum_{n=1}^N n A_n \frac{\sin n\theta}{\sin \theta} = \alpha_\infty + \alpha_g(\theta) - \alpha_{L=0}(\theta), \quad (1.5)$$

where b is the span, $c(\theta)$ is the chord, $a_0(\theta)$ is the sectional lift curve slope, $\alpha_g(\theta)$ is the geometric twist, and $\alpha_{L=0}(\theta)$ is the sectional zero-lift angle of attack. N is the number of unknown modes, i.e. coefficients A_n . To get N equations, we enforce Eqn. 1.5 at N *collocation* points, θ_i , usually equally-spaced in θ . In matrix form, this looks like

$$\begin{aligned}
\text{collocation point 1} &\rightarrow \\
\text{collocation point 2} &\rightarrow \\
&\vdots \\
\text{collocation point N} &\rightarrow
\end{aligned}
\begin{bmatrix}
M_{11} & M_{12} & \dots & M_{1N} \\
M_{21} & M_{22} & \dots & M_{2N} \\
\vdots & \vdots & \ddots & \vdots \\
M_{N1} & M_{N2} & \dots & M_{NN}
\end{bmatrix}
\begin{bmatrix}
A_1 \\
A_2 \\
\vdots \\
A_N
\end{bmatrix}
=
\begin{bmatrix}
F_1 \\
F_2 \\
\vdots \\
F_N
\end{bmatrix} \quad (1.6)$$

where

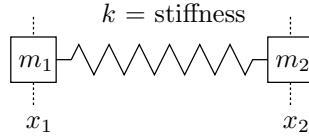
$$M_{in} = \frac{4b}{a_0(\theta_i)c(\theta_i)} \sin n\theta_i + n \frac{\sin n\theta_i}{\sin \theta_i} \quad (1.7)$$

$$F_i = \alpha_\infty + \alpha_g(\theta_i) - \alpha_{L=0}(\theta_i) \quad (1.8)$$

Given information about the wing, we can solve this matrix system, Eqn. 1.6, for the unknown A_n coefficients.

1.1.4.2 Masses on a Spring

Spring/mass systems sometimes arise not only in dynamics but also in simplified models of more complicated systems. Let's look at one example of two masses joined by one spring,



The tension force in the spring has a magnitude given by Hooke's law,

$$|F| = k(x_2 - x_1 - d),$$

where k is the spring stiffness and d is the unstretched spring length. The accelerations of each mass due to this force are then

$$\begin{aligned}\ddot{x}_1 &= \frac{|F|}{m_1} = k(x_2 - x_1 - d)/m_1 \\ \ddot{x}_2 &= -\frac{|F|}{m_2} = -k(x_2 - x_1 - d)/m_2\end{aligned}$$

Defining a state vector as

$$\mathbf{u} = \begin{bmatrix} x_1 \\ \dot{x}_1 \\ x_2 \\ \dot{x}_2 \end{bmatrix},$$

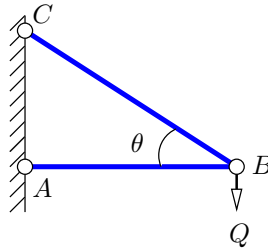
we can write down the equations of motion via the following linear system

$$\underbrace{\begin{bmatrix} \dot{x}_1 \\ \ddot{x}_1 \\ \dot{x}_2 \\ \ddot{x}_2 \end{bmatrix}}_{\dot{\mathbf{u}}} = \begin{bmatrix} \dot{x}_1 \\ k(x_2 - x_1 - d)/m_1 \\ \dot{x}_2 \\ -k(x_2 - x_1 - d)/m_2 \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & 1 & 0 & 0 \\ -k/m_1 & 0 & k/m_1 & 0 \\ 0 & 0 & 0 & 1 \\ k/m_2 & 0 & -k/m_2 & 0 \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} x_1 \\ \dot{x}_1 \\ x_2 \\ \dot{x}_2 \end{bmatrix}}_{\mathbf{u}} + \underbrace{\begin{bmatrix} 0 \\ -kd/m_1 \\ 0 \\ kd/m_2 \end{bmatrix}}_{\mathbf{q}} \quad (1.9)$$

$$\Rightarrow \dot{\mathbf{u}} = \mathbf{A}\mathbf{u} + \mathbf{q}$$

1.1.4.3 Truss Displacement

Consider a simple triangular truss, pictured below.



We are interested in the displacement of point B under a downward load of Q . Hooke's law lets us relate length changes to forces inside both of the truss members,

$$\begin{aligned} F_{AB} &= k_{AB}\delta_{AB}, \\ F_{BC} &= k_{BC}\delta_{BC}, \end{aligned}$$

where k_{AB}, k_{BC} are stiffnesses (“ AE/L ”) of the truss members, δ_{AB}, δ_{BC} are the member length changes (positive means extension), and F_{AB}, F_{BC} are forces in the members (positive means tension). If (u, v) denote the unknown x and y displacements at point B , we can write

$$\begin{aligned} \delta_{AB} &= u \\ \delta_{BC} &= u \cos \theta - v \sin \theta \end{aligned}$$

Summing the forces to zero at node B then gives

$$\begin{aligned} \begin{bmatrix} F_x \\ F_y \end{bmatrix} &= \begin{bmatrix} -F_{AB} - F_{BC} \cos \theta \\ F_{BC} \sin \theta - Q \end{bmatrix} = \begin{bmatrix} -k_{AB}\delta_{AB} - k_{BC}\delta_{BC} \cos \theta \\ k_{BC}\delta_{BC} \sin \theta - Q \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \\ &\Rightarrow \begin{bmatrix} -k_{AB}u - k_{BC}(u \cos \theta - v \sin \theta) \cos \theta \\ k_{BC}(u \cos \theta - v \sin \theta) \sin \theta \end{bmatrix} = \begin{bmatrix} 0 \\ Q \end{bmatrix} \\ &\Rightarrow \begin{bmatrix} -k_{AB} - k_{BC} \cos^2 \theta & k_{BC} \sin \theta \cos \theta \\ k_{BC} \cos \theta \sin \theta & -k_{BC} \sin^2 \theta \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} 0 \\ Q \end{bmatrix} \end{aligned}$$

Solving this 2×2 system gives the desired displacements at node B .

Example 1.2 (Setting up a system). The vertical (z -direction) force on a falling object is given by

$$F_z = -mg + D,$$

where m is the mass of the object g is the (constant) acceleration due to gravity, and the drag force is

$$D = \frac{1}{2}\rho(z)w^2 AC_D(Re), \quad \text{where } Re = \frac{\rho(z)wL}{\mu}.$$

Note that the density ρ is a function of the altitude, z . $w = \dot{z}$ is the vertical velocity, and L, μ are constants. A is the reference area of the object (e.g. cross-sectional area).

Using the state vector

$$\mathbf{u} = \begin{bmatrix} z \\ w \end{bmatrix},$$

we can write down a nonlinear equation of the form $\dot{\mathbf{u}} = \mathbf{f}(\mathbf{u})$. Since force is mass times acceleration, we have

$$\begin{aligned} m\dot{w} &= F_z = -mg + D \\ \dot{w} &= -g + D/m \end{aligned}$$

So our system is

$$\dot{\mathbf{u}} = \begin{bmatrix} \dot{z} \\ \dot{w} \end{bmatrix} = \begin{bmatrix} w \\ -g + D/m \end{bmatrix} = \mathbf{f}(\mathbf{u})$$

We can now linearize this equation about the state $\bar{\mathbf{u}} = [\bar{z}, \bar{w}]^T$ to obtain a linear system of the form

$$\dot{\mathbf{u}}' = \mathbf{A}\mathbf{u}' + \mathbf{q},$$

where $\mathbf{u} = \bar{\mathbf{u}} + \mathbf{u}'$. Substituting $\mathbf{u} = \bar{\mathbf{u}} + \mathbf{u}'$ into our equations, we have, since $\bar{\mathbf{u}}$ is constant,

$$\dot{\mathbf{u}} = \dot{\mathbf{u}}' = \mathbf{f}(\bar{\mathbf{u}} + \mathbf{u}') \approx \mathbf{f}(\bar{\mathbf{u}}) + \left. \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right|_{\bar{\mathbf{u}}} \mathbf{u}'$$

So we see that $\mathbf{q}$ is going to be $\mathbf{f}(\bar{\mathbf{u}})$, and $\mathbf{A}$ is going to be $\left. \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right|_{\bar{\mathbf{u}}}$.

$$\mathbf{q} = \mathbf{f}(\bar{\mathbf{u}}) = \begin{bmatrix} \bar{w} \\ -g + D(\bar{z}, \bar{w})/m \end{bmatrix}$$

$$\begin{aligned} \mathbf{A} = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right|_{\bar{\mathbf{u}}} &= \begin{bmatrix} \frac{\partial f_1}{\partial \bar{z}} & \frac{\partial f_1}{\partial \bar{w}} \\ \frac{\partial f_2}{\partial \bar{z}} & \frac{\partial f_2}{\partial \bar{w}} \end{bmatrix} \\ &= \begin{bmatrix} 0 & 1 \\ \frac{1}{m} \frac{\partial D}{\partial \bar{z}} & \frac{1}{m} \frac{\partial D}{\partial \bar{w}} \end{bmatrix} \end{aligned}$$

where

$$\begin{aligned} \frac{\partial D}{\partial z} &= \frac{1}{2} \frac{\partial \rho}{\partial z} w^2 A C_D(Re) + \frac{1}{2} \rho w^2 A \frac{\partial C_D}{\partial Re} \frac{\partial Re}{\partial z} \\ \frac{\partial D}{\partial w} &= \frac{1}{2} \rho 2w A C_D(Re) + \frac{1}{2} \rho w^2 A \frac{\partial C_D}{\partial Re} \frac{\partial Re}{\partial w} \\ \frac{\partial Re}{\partial z} &= \frac{wL}{\mu} \frac{\partial \rho}{\partial z} \\ \frac{\partial Re}{\partial w} &= \frac{\rho L}{\mu} \end{aligned}$$

Note all values and derivatives are calculated at $(\bar{z}, \bar{w})$.

1.2 Differential Equations

We deal with various differential equations in aerospace engineering. Ordinary differential equations (ODEs) involve derivatives with respect to one variable, such as time. An example of an ODE system is given in Eqn. 1.9. On the other hand, partial differential equations (PDEs) involve derivatives with respect to multiple variables, e.g. multiple spatial coordinates and/or time. Let's look a little more closely at what distinguishes PDEs and what kind of PDEs we find in aerospace engineering.

1.2.1 Classification of PDEs

A PDE is called *hyperbolic* if it admits wave-like solutions, i.e. solutions of the form $e^{i(\mathbf{k}\cdot\mathbf{x}-\omega t)}$, where $\mathbf{k}$ is the wave number vector and ω is the frequency. Problems involving transport, e.g. convection of a material, or wave propagation (e.g. aeroacoustics) are often governed by hyperbolic PDEs.

A PDE is called *elliptic* if it does not admit any wave-like solutions. Elliptic equations describe phenomena where source disturbances affect the entire domain. We see these equations whenever we study diffusion phenomena. If a time derivative term is added to a spatially elliptic equation, the resulting equation is called *parabolic*.

1.2.2 Initial and Boundary Conditions

PDEs require appropriate initial and boundary conditions in order to be well-posed. Initial conditions refer to the state at time $t = 0$: this is required when solving unsteady problems, which could be either hyperbolic or parabolic. Boundary conditions refer to specifying the state, or parts of the state, at the boundaries of the domain: such information is generally required when solving either steady or unsteady problems. In some cases, the boundary information may be given in terms of derivatives of states, so-called *Neumann* boundary conditions, as opposed to the state information only, so-called *Dirichlet* boundary conditions. *Robin* boundary conditions involve both the state and its derivatives. On some boundaries, we may not need to specify any boundary conditions, e.g. at the outflow of a hyperbolic propagation problem; in such cases all of the information is taken from the interior.

1.2.3 Sample Equations in Aerospace Engineering

1.2.3.1 Multi-Dimensional Heat Conduction

Conduction of heat in a material of thermal conductivity κ is governed by Fourier's law. If $q(\mathbf{x})$ is the volumetric source of heat within the material (zero if no heat sources are present), the equation governing the material temperature, $T(\mathbf{x})$, is

$$-\nabla \cdot (\kappa \nabla T) = q(\mathbf{x}).$$

This is an elliptic equation, and its solution gives the steady-state temperature distribution in the material. If we are interested in transients, we can add a time derivative term,

$$\frac{\partial T}{\partial t} - \nabla \cdot (\kappa \nabla T) = q(\mathbf{x}, t)$$

This equation for $T(\mathbf{x}, t)$ is then parabolic.

1.2.3.2 Linearized Compressible Potential Flow

Inviscid subsonic or supersonic flow with small disturbances in two dimensions can be described by a potential flow where the disturbance potential $\hat{\phi}(x, y)$ is governed by the fol-

lowing PDE:

$$(1 - M_\infty^2) \frac{\partial^2 \hat{\phi}}{\partial x^2} + \frac{\partial^2 \hat{\phi}}{\partial y^2} = 0, \quad (1.10)$$

where M_∞ is the Mach number. This equation is valid for either $M_\infty < 1$ or $M_\infty > 1$. Note that in the case of $M_\infty < 1$, the equation is elliptic, whereas for $M_\infty > 1$ the equation becomes hyperbolic. This agrees with the intuition that in subsonic flow disturbances can propagate and affect the entire domain, while in supersonic flow the disturbances propagate along certain waves, or characteristic directions, and only affect a portion of the domain.

1.2.3.3 Compressible Euler Equations of Gas Dynamics

The compressible Euler equations of gas dynamics in two spatial dimensions are given by

$$\begin{aligned} \frac{\partial}{\partial t}(\rho) &+ \frac{\partial}{\partial x}(\rho u) &+ \frac{\partial}{\partial y}(\rho v) &= 0 && \text{(mass)} \\ \frac{\partial}{\partial t}(\rho u) &+ \frac{\partial}{\partial x}(\rho u^2 + p) &+ \frac{\partial}{\partial y}(\rho uv) &= 0 && \text{(x-momentum)} \\ \frac{\partial}{\partial t}(\rho v) &+ \frac{\partial}{\partial x}(\rho vu) &+ \frac{\partial}{\partial y}(\rho v^2 + p) &= 0 && \text{(y-momentum)} \\ \frac{\partial}{\partial t}(\rho E) &+ \frac{\partial}{\partial x}(\rho u H) &+ \frac{\partial}{\partial y}(\rho v H) &= 0 && \text{(energy)} \end{aligned}$$

The variables appearing in this system of equations are defined as follows:

ρ	= density	<i>Calorically-perfect gas:</i>
u, v	= x, y components of velocity, $\vec{v}$	$e = c_v T$
E	= total energy per unit mass	$h = c_p T$
e	= internal energy per unit mass	$p = \rho R T$
H	= total enthalpy per unit mass	$= (\gamma - 1) \underbrace{\left[\rho E - \frac{1}{2} \rho (u^2 + v^2) \right]}_{\rho e}$
h	= internal enthalpy per unit mass	
p	= pressure	
R	= gas constant for air, $c_p - c_v$	$H = E + p/\rho$
γ	= ratio of specific heats, $c_p/c_v = 1.4$	$M = \vec{v} /a = \sqrt{u^2 + v^2}/c$
c_p	= specific heat at constant pressure	$c = \sqrt{\gamma R T} = \sqrt{\gamma p/\rho}$
c_v	= specific heat at constant volume	
c	= speed of sound	
M	= Mach number	

The Euler equations can be written in compact form,

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla \cdot \vec{\mathbf{F}}(\mathbf{u}) = \mathbf{0} \quad (1.11)$$

where the state and flux vectors are

$$\text{state: } \mathbf{u} = \begin{bmatrix} \rho \\ \rho u \\ \rho v \\ \rho E \end{bmatrix}, \quad \text{flux: } \vec{\mathbf{F}} = \begin{bmatrix} \rho u \\ \rho u^2 + p \\ \rho uv \\ \rho u H \end{bmatrix} \hat{\mathbf{i}} + \begin{bmatrix} \rho v \\ \rho vu \\ \rho v^2 + p \\ \rho v H \end{bmatrix} \hat{\mathbf{j}}$$

1.2.3.4 Euler-Bernoulli Beam Deflection

When a distributed load $q(x)$ is applied to a one-dimensional beam, the beam deflection $w(x)$ is governed by a bi-harmonic equation,

$$\frac{d^2}{dx^2} \left(EI \frac{d^2 w}{dx^2} \right) = q(x),$$

where EI is the product of the elastic modulus and the moment of inertia. This is an elliptic equation for $w(x)$ and to solve it we need four boundary conditions since we have four derivatives.

1.3 Norms and Function Spaces

We will use *norms*, i.e. scalar numbers, to quantify sizes of vectors and functions. Norms are sometimes induced by *inner products*, such as the one in Eqn. 1.1 for vectors – the resulting spaces are called *Hilbert* spaces. If $(\mathbf{u}, \mathbf{v})$ denotes the inner product between $\mathbf{u}$ and $\mathbf{v}$, then the induced norm is $\|\mathbf{u}\| = \sqrt{(\mathbf{u}, \mathbf{u})}$. But norms can be defined without inner products, and any (complete) linear space with a norm is called a *Banach* space.

Some useful norms for discrete vectors $\mathbf{u} \in \mathbb{R}^N$ or functions $v(\mathbf{x}) \in \mathbb{R}, \mathbf{x} \in \Omega$ are

1. L_1 norm:

$$\|\mathbf{u}\|_{L_1} = \frac{1}{N} \sum_{n=1}^N |u_n|, \quad \|v\|_{L_1} = \frac{1}{|\Omega|} \int_{\Omega} |v(\mathbf{x})| d\Omega$$

2. L_2 norm:

$$\|\mathbf{u}\|_{L_2} = \sqrt{\frac{1}{N} \sum_{n=1}^N u_n^2}, \quad \|v\|_{L_2} = \sqrt{\frac{1}{|\Omega|} \int_{\Omega} v(\mathbf{x})^2 d\Omega}$$

3. L_p norm, $p > 1$:

$$\|\mathbf{u}\|_{L_p} = \left(\frac{1}{N} \sum_{n=1}^N |u_n|^p \right)^{1/p}, \quad \|v\|_{L_p} = \left(\frac{1}{|\Omega|} \int_{\Omega} |v(\mathbf{x})|^p d\Omega \right)^{1/p}$$

4. L_{∞} norm:

$$\|\mathbf{u}\|_{L_{\infty}} = \max |u_n|, \quad \|v\|_{L_{\infty}} = \text{ess sup}_{\mathbf{x} \in \Omega} |v(\mathbf{x})|$$

The integrals appearing above are Lebesgue integrals, which basically means that they do not care about changes to v on a set of zero measure (e.g. at a point or on a line in a 2D domain). That's also why we use "ess sup" instead of "max" in the definition of the L_∞ norm for functions: "ess sup" is a maximum (supremum) that ignores regions of Ω which are of zero measure.

Associated with the function norms defined above are Lebesgue spaces, L^p . We say that a function $v(\mathbf{x})$ lies in the space L^p if it has a bounded (non-infinite) L_p norm. Another useful space and norm for functions is H^1 . The H^1 norm is defined as

$$\|v\|_{H^1}^2 = \frac{1}{|\Omega|} \left[\int_{\Omega} v(\mathbf{x})^2 d\Omega + \int_{\Omega} \nabla v \cdot \nabla v d\Omega \right].$$

So functions in the H^1 space must have square integrable values and first derivatives (gradients). This is a stricter requirement than that for L^2 , which just requires square-integrable values.

Finally, we define spaces $C^m(\Omega)$ as including only those functions that have continuous derivatives up to order m . For example, $C^0(\Omega)$ is the space of continuous functions on Ω (no jumps), $C^1(\Omega)$ is the space of functions with continuous first derivatives, etc.

Chapter 2

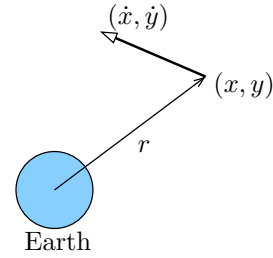
ODE Integration

2.1 Introduction

Consider the following example from orbital mechanics:

Example 2.1 (Satellite in orbit). A satellite orbiting Earth feels a radially-inward acceleration of GM_{earth}/r^2 when it is a distance r from the earth's center. Putting the Earth at the origin of a two-dimensional coordinate system, the x and y accelerations are

$$\begin{aligned}\ddot{x} &= -\frac{GM_{\text{earth}}}{r^2} \frac{x}{r} \\ \ddot{y} &= -\frac{GM_{\text{earth}}}{r^2} \frac{y}{r}\end{aligned}$$



Defining the position/velocity state by

$$\mathbf{u} = [x \quad y \quad \dot{x} \quad \dot{y}]^T,$$

the equations of motion are

$$\dot{\mathbf{u}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \ddot{x} \\ \ddot{y} \end{bmatrix} = \underbrace{\begin{bmatrix} \dot{x} \\ \dot{y} \\ -\frac{GM_{\text{earth}}}{r^2} \frac{x}{r} \\ -\frac{GM_{\text{earth}}}{r^2} \frac{y}{r} \end{bmatrix}}_{\mathbf{f}(\mathbf{u})} = \mathbf{f}(\mathbf{u}). \quad (2.1)$$

Note that $\mathbf{f}(\mathbf{u})$ is a nonlinear (because $r = \sqrt{x^2 + y^2}$), vector-valued function of the state $\mathbf{u}$.

When using numerical methods to solve an ODE, we typically divide a time horizon of interest, $t \in [0, T]$, into N intervals of equal length $\Delta t \equiv T/N$ ¹. Denote by $\mathbf{u}^n$, $0 \leq n \leq N$,

¹In general the intervals can be of different lengths, for example with small time steps at important times.

the state at *time node* n , i.e. at $t = t^n \equiv n\Delta t$. $\mathbf{u}^0$ is a known initial condition, and our objective in finite-difference methods for ODEs is to determine $\mathbf{u}^{n+1}$ using $\mathbf{u}^n$ and perhaps other prior states.

We must be aware that numerical methods are not going to give us the *exact* answer to the states at the time nodes. Instead, each $\mathbf{u}^n$ will be an approximation to the exact solution at t^n , which we will denote by $\mathbf{u}_{\text{exact}}(t^n)$. For good numerical methods, the error in our approximation will hopefully decrease as $\Delta t \rightarrow 0$.

Now, let's look at a couple methods for numerically solving a nonlinear ODE, specifically the one in Eqn. 2.1.

2.1.1 The Forward Euler Method

One of the simplest finite-difference methods for solving

$$\dot{\mathbf{u}} = \mathbf{f}(\mathbf{u}), \quad (2.2)$$

is to numerically approximate the time derivative of the state, $\dot{\mathbf{u}}$, as the difference of the states at two adjacent time nodes, divided by Δt . So the time derivative of the state at time node t^n is

$$\dot{\mathbf{u}} \Big|_{t^n} \approx \frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t}.$$

Substituting this into Eqn. 2.2 and solving for $\mathbf{u}^{n+1}$ gives the forward Euler update equation,

$$\boxed{\mathbf{u}^{n+1} = \mathbf{u}^n + \Delta t \mathbf{f}(\mathbf{u}^n)} \quad (2.3)$$

Figure 2.1 shows a graphical interpretation of this equation.

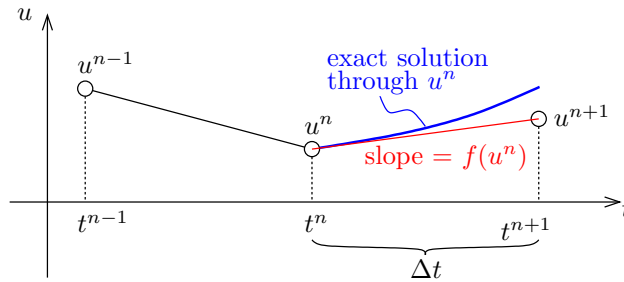


Figure 2.1: Graphical interpretation of the forward Euler method for a scalar problem. The state at t^{n+1} is given by extrapolating the state at t^n using the slope at t^n , which is just $f(u^n)$.

Example 2.2 (Forward Euler for the Satellite Problem). Let's apply the forward Euler method, Eqn. 2.3, to the system in Example 2.1. Suppose that the initial state is

$$\mathbf{u}(t^0 = 0) = [(x^0 = 4 \times 10^7 m) \quad (y^0 = 0 m) \quad (\dot{x}^0 = 0 m/s) \quad (\dot{y}^0 = 2500 m/s)]^T$$

Using this as an initial condition, we write a forward Euler code, given in Listing 2.1, for solving the system in Eqn. 2.1.

Listing 2.1: Code for computing a forward Euler solution to the satellite problem.

```

1 function [tn,un] = satFE(u0,T,N);
2 % function [tn,un] = satFE(u0,T,N);
3 % PURPOSE:
4 % This function solves the satellite orbit problem using forward Euler
5 % INPUTS:
6 % u0 : initial state [x; y; xdot; ydot] [m, m/s]
7 % T : end time (start is 0) [s]
8 % N : number of time steps
9 % OUTPUTS:
10 % tn : (N+1) vector of time node values
11 % un : 4x(N+1) matrix of state vectors
12
13 % initialize arrays
14 tn = linspace(0,T,N+1);
15 un = zeros(4,N+1);
16
17 un(:,1) = u0; % initial state
18 dt = T/N; % time step
19
20 % Forward Euler time integration
21 for n=1:N,
22     un(:,n+1) = un(:,n) + dt*fsat(un(:,n));
23 end
24
25 %-----
26 function f = fsat(u);
27 % u = [x; y; xdot; ydot]
28 GM = 6.67e-11 * 5.976e24; % M = mass of Earth [kg]
29 r = sqrt(u(1)^2 + u(2)^2); % r = sqrt(x^2 + y^2)
30 f = zeros(4,1);
31 f(1) = u(3); % xdot, directly from the state
32 f(2) = u(4); % ydot, directly from the state
33 f(3) = -GM*u(1)/r^3; % xddot = x-acceleration
34 f(4) = -GM*u(2)/r^3; % yddot = y-acceleration

```

The results in the form of orbit radius versus time for two different time steps are shown in Figure 2.2. We note that errors appear to build up during the course of the simulation, and that decreasing the time step improves the accuracy of integration relative to the exact solution.

2.1.2 The Predictor-Corrector Method

Looking at Figure 2.1, we see that the forward Euler assumes that the derivative $\dot{\mathbf{u}}$ over $t \in [t^n, t^{n+1}]$ equals the derivative at $t = t^n$. This will generally not be true, and by $t = t^{n+1}$ the time derivative might be quite different. A simple improvement is to use the average of $\mathbf{f} = \dot{\mathbf{u}}$ at $t = t^n$ and $t = t^{n+1}$. To get $\mathbf{f}$ at $t = t^{n+1}$ we need an approximate state there, and for this we can use the forward Euler state. So the improved update consists of two steps:

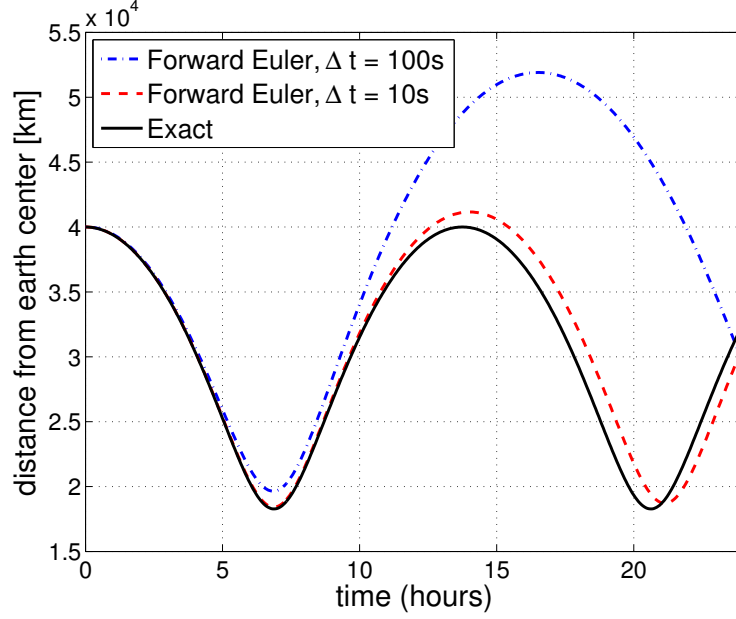


Figure 2.2: Forward Euler solution to an orbiting satellite problem.

1. Use forward Euler to *predict* the state at t^{n+1} ,

$$\tilde{\mathbf{u}}^{n+1} = \mathbf{u}^n + \Delta t \mathbf{f}(\mathbf{u}^n).$$

2. Use $\mathbf{f}(\tilde{\mathbf{u}}^{n+1})$ to *correct* the update through an improved slope,

$$\mathbf{u}^{n+1} = \mathbf{u}^n + \Delta t \frac{\mathbf{f}(\mathbf{u}^n) + \mathbf{f}(\tilde{\mathbf{u}}^{n+1})}{2}$$

Figure 2.3 shows a graphical interpretation of this equation. This two-stage update requires

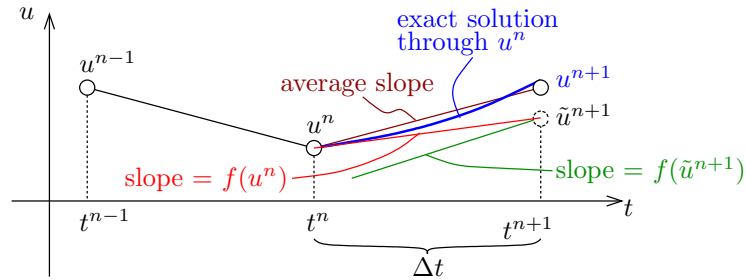


Figure 2.3: Graphical interpretation of the predictor-corrector method for a scalar problem. The state at t^{n+1} is given by extrapolating the state at t^n using the average of the slopes at t^n and at t^{n+1} .

two evaluations of $\mathbf{f}(\cdot)$, which makes this method roughly twice as expensive as forward

Euler per time step. However, we expect improved accuracy, which means possibly fewer time steps, with the predictor-corrector method.

Example 2.3 (Predictor-Corrector for the Satellite Problem). Let's apply the predictor-corrector method to our system in Example 2.1. Using the same initial condition as in Example 2.2 we run a predictor-corrector code for the system in Eqn. 2.1. The code is given in Listing 2.2.

Listing 2.2: Code for computing a predictor-corrector solution to the satellite problem.

```

1 function [tn,un] = satPC(u0,T,N);
2 % function [t,u] = satPC(u0,T,N);
3 % PURPOSE:
4 % This function solves the satellite problem using predictor-corrector
5 % INPUTS:
6 % u0 : initial state [x; y; xdot; ydot] [m, m/s]
7 % T : end time (start is 0) [s]
8 % N : number of time steps
9 % OUTPUTS:
10 % tn : (N+1) vector of time node values
11 % un : 4x(N+1) matrix of state vectors
12
13 % initialize arrays
14 tn = linspace(0,T,N+1);
15 un = zeros(4,N+1);
16
17 un(:,1) = u0; % initial state
18 dt = T/N; % time step
19
20 % Predictor-corrector time integration
21 for n=1:N,
22     uFE = un(:,n) + dt*fsat(un(:,n));
23     un(:,n+1) = un(:,n) + dt*0.5*(fsat(un(:,n))+fsat(uFE));
24 end
25
26 %-----
27 function f = fsat(u);
28 % u = [x; y; xdot; ydot]
29 GM = 6.67e-11 * 5.976e24; % M = mass of Earth [kg]
30 r = sqrt(u(1)^2 + u(2)^2); % r = sqrt(x^2 + y^2)
31 f = zeros(4,1);
32 f(1) = u(3); % xdot, directly from the state
33 f(2) = u(4); % ydot, directly from the state
34 f(3) = -GM*u(1)/r^3; % xddot = x-acceleration
35 f(4) = -GM*u(2)/r^3; % yddot = y-acceleration

```

The results in the form of orbit radius versus time for two different time steps are shown in Figure 2.4. Again, the errors appear to build up during the course of the simulation, and decreasing the time step improves the accuracy of integration. While the curves show qualitatively similar errors to those in Figure 2.2, note that we choose the time steps to be *much* larger for the predictor-corrector method. Even though the cost per time step is about twice as much when using predictor-corrector, in this case the cost is certainly worth it.

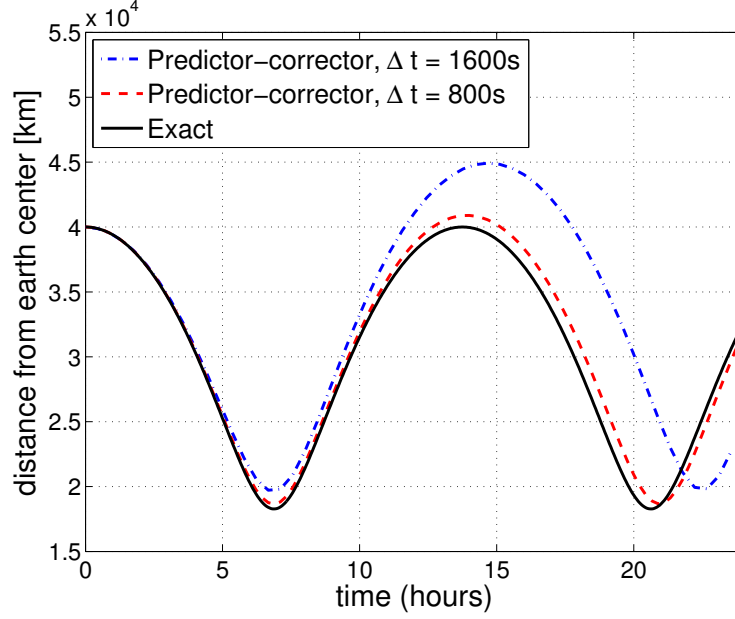


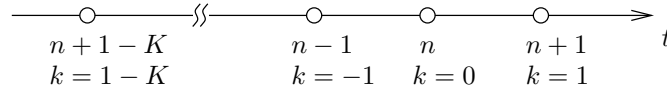
Figure 2.4: Predictor-corrector solution to an orbiting satellite problem.

2.2 Multi-Step Methods

Many time-integration methods can be classified as *multi-step*. These are methods that relate an unknown state at time node $n+1$ to a certain number of previous states and slopes. They are typically defined for time discretizations with uniform time steps, $\Delta t = \text{constant}$. The general formula for a K -step method for solving $\dot{\mathbf{u}} = \mathbf{f}(\mathbf{u})$ is

$$\sum_{k=1-K}^1 \alpha_k \mathbf{u}^{n+k} = \Delta t \sum_{k=1-K}^1 \beta_k \mathbf{f}(\mathbf{u}^{n+k}, t^{n+k}) \quad (2.4)$$

where α_k, β_k are coefficients that define the method. Note that we put t^{n+k} as an argument into $\mathbf{f}$ to include cases where $\mathbf{f}$ depends directly on time t in addition to the state $\mathbf{u}$. The relationship between k and n is illustrated below.



In Eqn. 2.4, we typically set $\alpha_1 = 1$, and our goal is to solve for $\mathbf{u}^{n+1}$. A complication arises on the right-hand side if $\beta_1 \neq 0$. In this case our equation for $\mathbf{u}^{n+1}$ involves $\mathbf{f}(\mathbf{u}^{n+1}, t^{n+1})$... in other words, $\mathbf{u}^{n+1}$ depends *implicitly* on itself through $\mathbf{f}$. Such methods are called **implicit**. On the other hand, if $\beta_1 = 0$ we can directly, or *explicitly*, solve for $\mathbf{u}^{n+1}$ in terms of previous states and slopes. Such methods are called **explicit**.

The forward Euler method in Eqn. 2.3 is an explicit multi-step method with $K = 1$, and

$$\alpha_0 = -1, \alpha_1 = 1, \beta_0 = 1, \text{ all other } \alpha_k, \beta_k \text{ zero.}$$

The predictor-corrector update is not a multi-step method because it uses $\mathbf{f}$ from a temporary state update instead of $\mathbf{f}$ from only the time nodes. Some other popular multi-step methods are as follows:

- First-order Backward Difference (BDF1), also known as Backward Euler,

$$\mathbf{u}^{n+1} - \mathbf{u}^n = \Delta t \mathbf{f}(\mathbf{u}^{n+1}, t^{n+1}).$$

- Second-order Backward Difference (BDF2),

$$\frac{3}{2}\mathbf{u}^{n+1} - 2\mathbf{u}^n + \frac{1}{2}\mathbf{u}^{n-1} = \Delta t \mathbf{f}(\mathbf{u}^{n+1}, t^{n+1}).$$

- Third-order Backward Difference (BDF3),

$$\frac{11}{6}\mathbf{u}^{n+1} - 3\mathbf{u}^n + \frac{3}{2}\mathbf{u}^{n-1} - \frac{1}{3}\mathbf{u}^{n-2} = \Delta t \mathbf{f}(\mathbf{u}^{n+1}, t^{n+1}).$$

- Second-order Adams-Bashforth (AB2),

$$\mathbf{u}^{n+1} - \mathbf{u}^n = \Delta t \left(\frac{3}{2}\mathbf{f}(\mathbf{u}^n, t^n) - \frac{1}{2}\mathbf{f}(\mathbf{u}^{n-1}, t^{n-1}) \right).$$

- Third-order Adams-Bashforth (AB3),

$$\mathbf{u}^{n+1} - \mathbf{u}^n = \Delta t \left(\frac{23}{12}\mathbf{f}(\mathbf{u}^n, t^n) - \frac{16}{12}\mathbf{f}(\mathbf{u}^{n-1}, t^{n-1}) + \frac{5}{12}\mathbf{f}(\mathbf{u}^{n-2}, t^{n-2}) \right).$$

- Second-order Adams-Moulton or Trapezoidal (AM2, Trap),

$$\mathbf{u}^{n+1} - \mathbf{u}^n = \Delta t \left(\frac{1}{2}\mathbf{f}(\mathbf{u}^{n+1}, t^{n+1}) + \frac{1}{2}\mathbf{f}(\mathbf{u}^n, t^n) \right).$$

- Third-order Adams-Moulton (AM3),

$$\mathbf{u}^{n+1} - \mathbf{u}^n = \Delta t \left(\frac{5}{12}\mathbf{f}(\mathbf{u}^{n+1}, t^{n+1}) + \frac{8}{12}\mathbf{f}(\mathbf{u}^n, t^n) - \frac{1}{12}\mathbf{f}(\mathbf{u}^{n-1}, t^{n-1}) \right).$$

A few comments are in order. First, of the methods listed, only the Adams-Bashforth methods are explicit; all others are implicit. Second, backwards differentiation methods all use just one $\mathbf{f}$, at $n + 1$, while varying the number of states on the left-hand-side. On the other hand, the Adams-Bashforth and Adams-Moulton methods all have $\mathbf{u}^{n+1} - \mathbf{u}^n$ on the left hand side and instead vary the number of $\mathbf{f}$ terms on the right-hand side. Finally, for methods that require more than one previous state (e.g. BDF2), another method (e.g. BDF1) must be used for the first time step(s).

So how do we decide which method to use? The two main considerations are

1. *Cost for a given accuracy*: we'd like to use the computationally-cheapest but most accurate method available. The choice of method will generally depend on the desired level of accuracy.
2. *Stability*: we don't want the solution to blow up, which can happen if a method is unstable. Stability depends in part on the size of the time step.

We address these considerations in the following sections.

2.3 Accuracy and Consistency

Let's look at accuracy in the context of multi-step methods. We can think of Eqn. 2.4 as an equation for $\mathbf{u}^{n+1}$ given data at previous time nodes. Let's say this data were perfect, meaning taken from the exact solution. How accurate would our answer for $\mathbf{u}^{n+1}$ be in this case? A systematic technique for answering this question is to calculate the *truncation error* associated with the update.

2.3.1 Truncation Error

The **truncation error**, τ^{n+1} , is the quantity that remains in Eqn. 2.4 after the exact solution, $\mathbf{u}_{\text{exact}}(t)$ is plugged in for both $\mathbf{u}^{n+k}$ and $\mathbf{f} = \dot{\mathbf{u}}$. Defining “what remains” by left-hand-side minus the right-hand-side, we obtain,

$$\tau^{n+1} \equiv \sum_{k=1-K}^1 \alpha_k \mathbf{u}_{\text{exact}}^{n+k} - \Delta t \sum_{k=1-K}^1 \beta_k \mathbf{f}(\mathbf{u}_{\text{exact}}^{n+k}, t^{n+k}). \quad (2.5)$$

This error directly pollutes the next state, $\mathbf{u}^{n+1}$, and it is sometimes called the *local error* or the error made in one time step. The error arises because we are using finite difference approximations to the time derivatives.

We say that a multi-step method is *of order p* if

$$\tau^{n+1} = \mathcal{O}(\Delta t^{p+1}). \quad (2.6)$$

For example, for a second-order method ($p = 2$), this equations says that the truncation error must decrease by a factor of 8 when Δt is halved. We'll see very soon that the use of p versus $p + 1$ is related to the distinction between local and global order of accuracy.

How do we calculate τ^{n+1} ? The most straightforward way is through a Taylor-series expansion of the state and derivative. Let's do this about the state $\mathbf{u}^n$. We have

$$\begin{aligned} \mathbf{u}^{n+k} &= \mathbf{u}^n + k\Delta t(\mathbf{u}_t)^n + \frac{1}{2}(k\Delta t)^2(\mathbf{u}_{tt})^n + \mathcal{O}(\Delta t^3) \\ \mathbf{f}(\mathbf{u}^{n+k}) = \mathbf{u}_t^{n+k} &= \mathbf{u}_t^n + k\Delta t(\mathbf{u}_{tt})^n + \frac{1}{2}(k\Delta t)^2(\mathbf{u}_{ttt})^n + \mathcal{O}(\Delta t^3) \end{aligned} \quad (2.7)$$

Substituting these expressions into the multi-step update equation and collecting terms with the same power of Δt gives us our truncation error. The higher the order of accuracy of a method, the more powers of Δt will cancel in this substitution.

Example 2.4 (Truncation Error of Forward Euler). The forward Euler update in Eqn. 2.3 involves $\mathbf{u}^{n+1}$ and $\mathbf{u}^n$. Expanding $\mathbf{u}^{n+1}$ using the Taylor series from Eqn. 2.7 in the forward Euler update gives the following equation,

$$\underbrace{\mathbf{u}^n + \Delta t \mathbf{u}_t^n + \frac{1}{2} \Delta t^2 \mathbf{u}_{tt}^n + \mathcal{O}(\Delta t^3)}_{\mathbf{u}^{n+1}} = \mathbf{u}^n + \Delta t \underbrace{\mathbf{u}_t^n}_{\mathbf{f}^n}$$

The truncation error is the left-hand side minus the right-hand side,

$$\boldsymbol{\tau}^{n+1} = \frac{1}{2} \Delta t^2 \mathbf{u}_{tt}^n + \mathcal{O}(\Delta t^3).$$

Since the leading remaining term is $\mathcal{O}(\Delta t^2)$, we say that forward Euler is first-order accurate, $p = 1$.

Example 2.5 (Minimizing Truncation Error). Suppose we would like to determine the most accurate $K = 2$ explicit multi-step method. Since we're looking for an explicit method, $\beta_1 = 0$ and the general multi-step formula in Eqn. 2.4 for $K = 2$ becomes

$$\alpha_{-1} \mathbf{u}^{n-1} + \alpha_0 \mathbf{u}^n + \alpha_1 \mathbf{u}^{n+1} = \Delta t (\beta_{-1} \mathbf{f}^{n-1} + \beta_0 \mathbf{f}^n).$$

Inserting the Taylor-series expansions about $t = t^n$ from Eqn. 2.7 and matching coefficients we obtain

$$\begin{aligned} \mathcal{O}(1) : \quad \alpha_{-1} + \alpha_0 + \alpha_1 &= 0 \\ \mathcal{O}(\Delta t) : \quad -\alpha_{-1} + \alpha_1 &= \beta_{-1} + \beta_0 \\ \mathcal{O}(\Delta t^2) : \quad \frac{\alpha_{-1}}{2} + \frac{\alpha_1}{2} &= -\beta_{-1} \\ \mathcal{O}(\Delta t^3) : \quad -\frac{\alpha_{-1}}{6} + \frac{\alpha_1}{6} &= \frac{\beta_{-1}}{2} \\ \mathcal{O}(\Delta t^4) : \quad \frac{\alpha_{-1}}{24} + \frac{\alpha_1}{24} &= -\frac{\beta_{-1}}{6} \end{aligned}$$

We expect our coefficients to be defined only up to an arbitrary multiplicative constant, so we set $\alpha_1 = 1$ to remove this arbitrariness. That leaves four unknowns, which means that we can match up to the $\mathcal{O}(\Delta t^3)$ terms above. Note that we should not expect to match $\mathcal{O}(\Delta t^4)$ because this equation is not even compatible with the $\mathcal{O}(\Delta t^2)$ equation. Solving the first four of the above equations yields

$$\alpha_{-1} = -5, \quad \alpha_0 = 4, \quad \alpha_1 = 1, \quad \beta_{-1} = 2, \quad \beta_0 = 4.$$

We would call this a third-order accurate method because the truncation error, given by the first term that we could not match, is $\mathcal{O}(\Delta t^4)$. Unfortunately, we'll soon see that this method is useless because it is unstable!

2.3.2 Consistency

How large of a truncation error are we willing to accept? For a time integration method to be useful, it must be at least first-order accurate, which means that we need at least $\tau^{n+1} = \mathcal{O}(\Delta t^2)$. Zeroth order accuracy won't do because it means that the truncation error is $\mathcal{O}(\Delta t)$, and we can get this by even neglecting all of the physics. For example, imagine a silly time-marching method in which the state just stayed constant, $\mathbf{u}^{n+1} = \mathbf{u}^n$, regardless of what $\mathbf{f}$ was. Assuming a continuous actual solution, at each time step we would be committing an error of $\mathcal{O}(\Delta t)$ since our state would not be changing while the actual state would be changing by $\sim \mathbf{f}\Delta t$. But of course this is a useless method, and we'd like to make sure our methods do better.

We call a method *consistent* if it is at least first-order accurate, i.e. if $\tau^{n+1} = \mathcal{O}(\Delta t^2)$. Referring to the example in the previous paragraph, we can think of consistency as a requirement that we respect the physics of the problem. Let's look at what consistency means for a multi-step method. Substituting the Taylor-series expansions from Eqn. 2.7 into Eqn. 2.5, we find

$$\tau^{n+1} = \left[\sum_{k=1-K}^1 \alpha_k \right] \mathbf{u}_{\text{exact}}^n + \left[\sum_{k=1-K}^1 (k\alpha_k - \beta_k) \right] \mathbf{u}_{\text{exact},t}^n \Delta t + \mathcal{O}(\Delta t^2)$$

For consistency, we need the $\mathcal{O}(1)$ and $\mathcal{O}(\Delta t)$ terms to cancel,

$$\sum_{k=1-K}^1 \alpha_k = 0, \quad (2.8)$$

$$\sum_{k=1-K}^1 k\alpha_k = \sum_{k=1-K}^1 \beta_k. \quad (2.9)$$

Example 2.6 (Consistent Multistep Schemes). Using Eqn. 2.8 and Eqn. 2.9, we can quickly verify that all of the schemes presented at the beginning of this section are consistent. The general form of a consistent two-step method is

$$\xi \mathbf{u}^{n-1} - (1 + 2\xi) \mathbf{u}^n + (1 + \xi) \mathbf{u}^{n+1} = \Delta t (\phi \mathbf{f}^{n-1} + (1 - \theta - \phi) \mathbf{f}^n + \theta \mathbf{f}^{n+1}). \quad (2.10)$$

We verify this by first checking that Eqn. 2.8 holds,

$$\sum_{k=1-K}^1 \alpha_k = \xi - (1 + 2\xi) + (1 + \xi) = 0.$$

Eqn. 2.9 also holds since,

$$\sum_{k=1-K}^1 k\alpha_k = (-1)\xi + 0(-(1 + 2\xi)) + 1(1 + \xi) = 1 = \phi + (1 - \theta - \phi) + \theta = \sum_{k=1-K}^1 \beta_k.$$

Also, by looking at the truncation error, one can verify that the scheme in Eqn. 2.10 will be second-order accurate if $\xi = \theta - \phi - 1/2$. It will be third-order accurate if also $\xi = -2\phi - 1/6$.

2.3.3 Global Error and Convergence

When we integrate ODEs, we're interested in the error not only in an individual time step, but also over the entire time history. We define the *global error* of a discrete solution at time nodes $\mathbf{u}^n$ relative to the exact solution $\mathbf{u}_{\text{exact}}^n = \mathbf{u}_{\text{exact}}(n\Delta t)$ as

$$\text{global error} = \epsilon^{\text{global}} \equiv \max_{n=1\dots N} \|\mathbf{u}^n - \mathbf{u}_{\text{exact}}^n\|_{L_\infty}.$$

In this expression, we use the L_∞ norm at each time node in order to pick out the maximum error over the components of the state. In short, the global error is the maximum difference between the computed state and the exact state over the course of the simulation.

We say that a finite difference method is *convergent* for a particular ODE, $\dot{\mathbf{u}} = \mathbf{f}$, if the global error goes to zero as we reduce the time step,

$$\epsilon^{\text{global}} \rightarrow 0 \quad \text{as} \quad \Delta t \rightarrow 0.$$

Note that the concept of convergence applies to the combination of method and ODE. A particular integration method may converge for one ODE but not for another.

For convergent methods, we can relate the global error, ϵ^{global} to the truncation error, τ^n . Recall that the truncation error τ^n is the error in the time step $n-1 \rightarrow n$. From an order-of-magnitude point of view, we expect these local truncation errors to add up over the course of the simulation, so that the error at the final time would be the biggest, i.e.

$$\epsilon^{\text{global}} = \mathcal{O} \left(\sum_{n=1}^N \|\tau^n\| \right). \quad (2.11)$$

Now, recall from Eqn. 2.6 that we named a method “order p ” if each truncation error was $\mathcal{O}(\Delta t^{p+1})$. If we add up $N \sim 1/\Delta t$ such truncation errors in Eqn. 2.11, we expect

$$\epsilon^{\text{global}} = \mathcal{O} \left(\sum_{n=1}^N \|\tau^n\| \right) = \mathcal{O}(N\Delta t^{p+1}) = \mathcal{O} \left(\frac{1}{\Delta t} \Delta t^{p+1} \right) = \mathcal{O}(\Delta t^p). \quad (2.12)$$

So we expect the global error to decrease at a rate of Δt to a power that is one less than that for the local or truncation error. We also see that the name “order p ” is appropriate since this is the rate at which the global error converges.

Finally, we mention that the local errors do not always sum as indicated in Eqn. 2.12. Conceivably, although not often, local errors could cancel so that the global error *super-converges* at a rate higher than p . A more typical case is that of the global error growing faster than expected, in fact exponentially, due to a lack of stability in the method. We consider this case in the next section.

Example 2.7 (Global Convergence for the Satellite Problem). In this example we measure the global convergence rates of two integration methods, forward Euler and predictor-corrector, for the satellite problem of Example 2.1. To do this, we run the two codes for

various time steps Δt and we plot the natural logarithm, “log”, of the global error versus log of Δt . Why the logarithms? Since we expect

$$\epsilon^{\text{global}} = C\Delta t^p,$$

then, taking the log of both sides, we have

$$\log \epsilon^{\text{global}} = \log C + p \log \Delta t.$$

So a plot of log of the global error versus log of Δt should reduce to a straight line with slope p . Figure 2.5 shows such plots for the two methods we’ve been considering. Note that

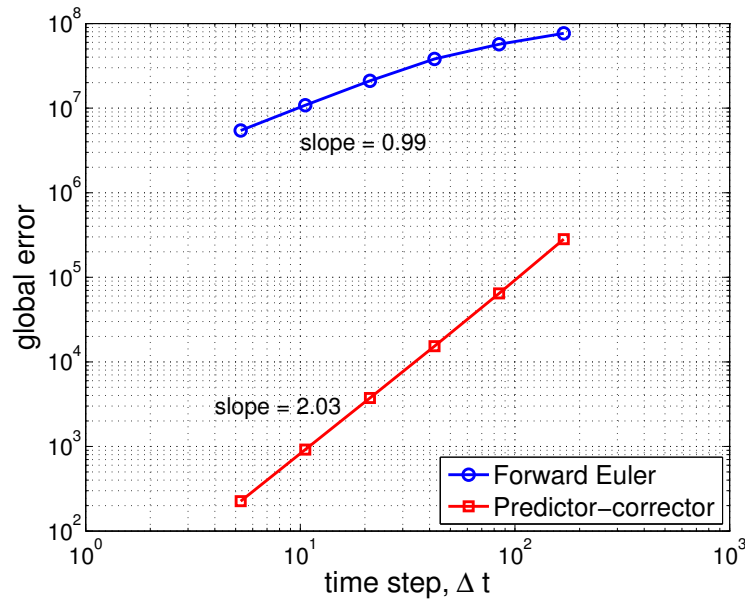


Figure 2.5: Global errors of two time-integration methods for the orbiting satellite problem. The slope of the lines on the log-log scale indicate the order of accuracy of the methods.

to make the plots easier to read, we just use a log-log scale instead of plotting the log of the quantities. These plots are not exactly straight lines, although they are pretty close. We calculate slopes using the two runs at the smallest Δt – we expect runs at small Δt to be indicative of what happens as Δt asymptotically approaches zero. From the figure, we see that the predictor-corrector method is much more accurate relative to forward Euler, and that it converges at a faster rate: approximately 2 compared to approximately 1 for forward Euler. This result is consistent with the expected convergence rates for the methods.

2.4 Stability

Numerical ODE integration methods approximate the solution $\mathbf{u}_{\text{exact}}(t)$ by a sequence of discrete states, $\mathbf{u}^n$. If these states grow unbounded (when the exact solution does not), we say that the integration method applied to that particular ODE is *unstable*. In this section we make this concept a little more precise, with emphasis on multi-step methods.

2.4.1 Zero Stability

Suppose that we are interested in a fixed time range of $t \in [0, T]$. We approximate the solution of an ODE with $N + 1$ states $\mathbf{u}^n$ and a time step of $\Delta t = T/N$. We say that a multi-step scheme is *zero-stable* if the solution remains bounded as $\Delta t \rightarrow 0$, i.e. as $N \rightarrow \infty$. Note that this concept does not address stability for a given Δt , which we discuss in the next sub-section. Instead, for now we only care about the limit $\Delta t \rightarrow 0$.

As $\Delta t \rightarrow 0$ for a multi-step method, the contribution of the “physics”, i.e. the right-hand-side of Eqn. 2.4 drops out, since all of these terms are multiplied by Δt . What’s left is a *recurrence relation*, which we write assuming the state $\mathbf{u} = u$ is a scalar for simplicity,

$$\sum_{k=1-K}^1 \alpha_k u^{n+k} = 0. \quad (2.13)$$

For zero-stability, all solutions of this recurrence relation must remain bounded as $n \rightarrow \infty$.

Recurrence relations admit solutions of the form $u^n = u^0 g^n$, where g^n here means a scalar g raised to the n^{th} power, and where g is a (complex) root of the following polynomial equation,

$$\sum_{k=1-K}^1 \alpha_k g^k = 0.$$

For zero stability, we need all roots of this equation to satisfy $|g| < 1$. $|g| = 1$ is allowed as long as the root is not repeated (otherwise $u^n = n u^0 g^n$ would be a solution).

Example 2.8 (Zero Stability of Forward Euler). For the forward Euler scheme, the recurrence relation from Eqn. 2.13 reads

$$\begin{aligned} u^{n+1} - u^n &= 0 \\ u^0 g^{n+1} - u^0 g^n &= 0 \\ u^0 g^n (g - 1) &= 0 \end{aligned}$$

We see that the solutions to this equation are $g = 0$ and a single root $g = 1$. Hence, the forward-Euler scheme is zero-stable.

Example 2.9 (Lack of Zero Stability). Recall the most-accurate $K = 2$ explicit multi-step scheme that we derived in Example 2.5,

$$-5\mathbf{u}^{n-1} + 4\mathbf{u}^n + \mathbf{u}^{n+1} = \Delta t (2\mathbf{f}^{n-1} + 4\mathbf{f}^n).$$

Ignoring the right-hand side, the recurrence relation from Eqn. 2.13 reads

$$\begin{aligned} -5u^{n-1} + 4u^n + u^{n+1} &= 0 \\ u^0 g^{n-1}(-5 + 4g + g^2) &= 0 \end{aligned}$$

The nonzero solutions to this equation are $g = 1$ and $g = -5$. The problematic root is $g = -5$ since it does not satisfy $|g| < 1$. Why is this problematic? The reason is that having $g = -5$ as a root means that $u^n = u^0(-5)^n$ is a solution to the homogeneous (zero right-hand side) part of the equation. So an error that grows exponentially with n can “slip by” undetected and un-damped by the numerical method. In practice, the source of this error can be as innocuous as machine precision (rounding) errors – exponential growth makes such an error a problem in only a few iterations.

As a demonstration, we apply this unstable scheme to the satellite problem of Example 2.1. Figure 2.6 shows the result for the orbit radius versus time. The solution blows up exponentially away from the exact solution shortly after the start of integration. If we look closely, we see that the errors are multiplied by a factor of approximately -5 from iteration to iteration. For reference, the time step here was about $\Delta t = 2.6s$. These errors are seeded by machine precision, and for smaller time steps, we would expect the solution to blow up “sooner”, albeit after a similar number of time steps.

2.4.2 Eigenvalue Stability

While zero stability tells us what happens in the limit $\Delta t \rightarrow 0$, we often care about the practical case of a finite time step, $\Delta t > 0$. For example, we might want large time steps if we’re solving a steady-state problem or if we want to solve a problem without too many time steps for reasons of efficiency. In such cases we need to know whether a time integration method is stable for a particular Δt .

We will analyze stability of linear systems of the form

$$\dot{\mathbf{u}} = \mathbf{A}\mathbf{u}, \tag{2.14}$$

Note, adding a constant source term on the right hand side would not affect the stability analysis.

Let’s assume that the above system is non-defective, meaning that $\mathbf{A}$ has a complete set of eigenvectors. We can then diagonalize $\mathbf{A}$ by writing $\mathbf{A} = \mathbf{R}\mathbf{\Lambda}\mathbf{R}^{-1}$, and the system in

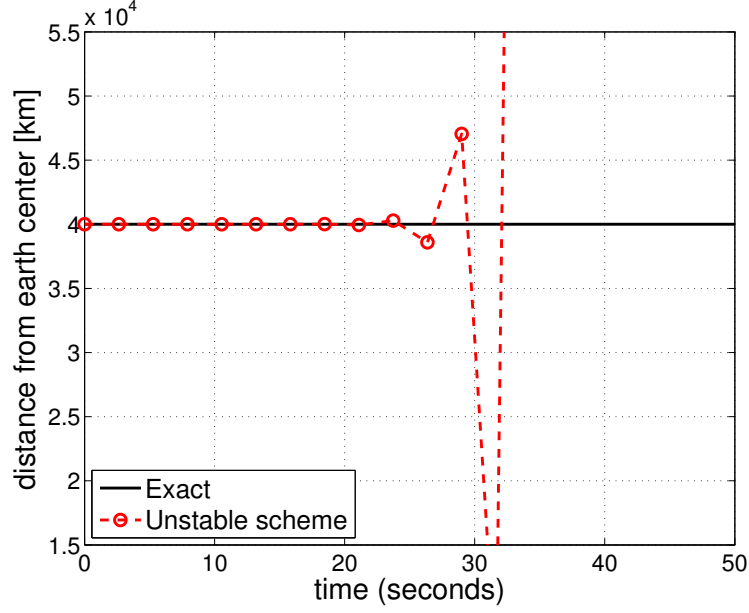


Figure 2.6: Results from applying the time-integration scheme derived in Example 2.5 to the satellite problem. This scheme is not zero stable, and hence the legend entry of “unstable” – the red dashed curve grows unbounded shortly after the start of the time integration.

Eqn. 2.14 becomes

$$\begin{aligned}
 \dot{\mathbf{u}} &= \mathbf{A}\mathbf{u} \\
 \dot{\mathbf{u}} &= \mathbf{R}\mathbf{\Lambda}\mathbf{R}^{-1}\mathbf{u} \\
 \underbrace{\mathbf{R}^{-1}\dot{\mathbf{u}}}_{\dot{\mathbf{w}}} &= \mathbf{\Lambda}\underbrace{\mathbf{R}^{-1}\mathbf{u}}_{\mathbf{w}} \\
 \dot{\mathbf{w}} &= \mathbf{\Lambda}\mathbf{w}
 \end{aligned}$$

In the last line we have a set of decoupled ODEs. That is, if the original system is $N \times N$, we now have N independent scalar equations, each of the form

$$\dot{w} = \lambda w, \quad (2.15)$$

where λ is an eigenvalue of $\mathbf{A}$. So the maximum time step allowed for integrating Eqn. 2.14 is going to be limited by the most restrictive eigenvalue of $\mathbf{A}$. More importantly, the diagonalization shows us that instead of tackling systems all at once, we can instead consider just scalar equations of the form in Eqn. 2.15. This is what we will do, and to avoid introducing new variables, we will use u instead of w . So we consider the following scalar ODE,

$$\dot{u} = \lambda u, \quad (2.16)$$

where λ is a possibly-complex scalar.

To analyze the stability of a time-integration scheme, we calculate the *amplification factor*, g , of the state after one iteration. That is, we write

$$u^{n+1} = gu^n, \quad (2.17)$$

where g is a constant that could be complex. The motivation for writing the above equation is that we're working with a homogeneous (just λu on the right-hand side) and time-independent (λ is constant) system. So there is nothing to distinguish one iteration from another, and if after one iteration the state is multiplied by g , it will also be multiplied by g after the next iteration. In particular, we can write $u^n = g^n u^0$, where g^n means g raised to the n^{th} power.

What's the use of a homogeneous system? It is the form of the *error equation* of any constant-coefficient non-homogeneous system. The amplification factor then gives us the rate at which the error grows, if $|g| > 1$, or decays, if $|g| < 1$. For stability we want the latter case, $|g| < 1$.

The amplification factor g will depend on the “physics” of the problem, i.e. λ , and on the numerics via the choice of integration scheme and time step, Δt . Let's see how this works through an example.

Example 2.10 (Eigenvalue Stability of Forward Euler). Recall the forward Euler update from Eqn. 2.3, which for $f(u) = \lambda u$ is

$$u^{n+1} = u^n + \Delta t \lambda u^n.$$

To calculate the amplification factor, we substitute $u^n = g^n u^0$ into this equation,

$$\begin{aligned} g^{n+1} u^0 &= g^n u^0 + \Delta t \lambda g^n u^0 \\ g^n u^0 (g) &= g^n u^0 (1 + \lambda \Delta t) \\ \Rightarrow g &= 1 + \lambda \Delta t \end{aligned}$$

For stability, we'd like the amplification factor to have magnitude less than 1. From the above equation, we see that for real λ , this means $-2 < \lambda \Delta t < 0$. But in general, λ can be complex, and so we need to allow for complex g . We determine the general *stability boundary* by setting $g = e^{i\theta}$, since this is the set of all complex numbers such that $|g| = 1$. Hence we have,

$$\begin{aligned} e^{i\theta} &= 1 + \lambda \Delta t \\ \Rightarrow \lambda \Delta t &= e^{i\theta} - 1 \end{aligned}$$

which is a circle of radius 1 centered at -1 in the complex number plane. This analysis gives us the stability boundary, and all that's left is figuring out which side of the boundary is stable. To do this, we pick a point, i.e. a value of $\lambda \Delta t$, and calculate $|g|$: for example, for forward Euler, choosing the point $\lambda \Delta t = -1$ gives $|g| = 0$, which is stable. This means that the region inside the unit circle centered at $\lambda \Delta t = -1$ is stable, and we obtain the stability diagram shown in Figure 2.7.

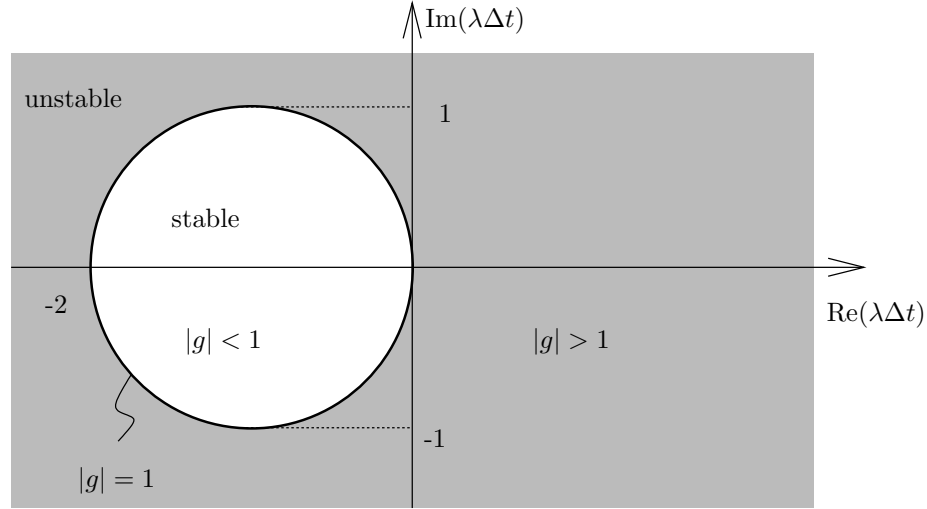


Figure 2.7: Eigenvalue stability region for the forward Euler method. The region inside the circle is stable as it yields amplification factors with $|g| < 1$.

We make a few remarks about the concept of a stability region derived in Example 2.10.

- The stability boundary is a contour in the complex number plane of $\lambda\Delta t$.
- If $\lambda\Delta t$ falls inside the stability region, for all eigenvalues λ in the case of a system, then the time integration method is stable.
- We often cannot control λ directly, as it usually is given by the physics or the spatial discretization in the case of PDEs.
- Δt will usually be the first step for controlling stability, and we will often speak of time-step restrictions, which refer to maximum values of Δt allowed for stability.
- Zero stability manifests itself in the diagram as stability in the left-half-plane as we zoom into the origin by making $\Delta t \rightarrow 0$.

Example 2.11 (Eigenvalue Stability of Backward Euler). The backward Euler method, also known as first-order backward differencing (BDF1), is a one-step method similar to forward Euler with the exception that the forcing term is evaluated at $n+1$ instead of at n . Applying this method to our prototypical eigenvalue system in Eqn. 2.16 we have

$$u^{n+1} = u^n + \Delta t \lambda u^{n+1}.$$

The stability analysis now proceeds in a way similar to the forward Euler case. Introducing the amplification factor via Eqn. 2.17,

$$\begin{aligned} g^{n+1}u^0 &= g^n u^0 + \Delta t \lambda g^{n+1} u^0 \\ g^n u^0(g) &= g^n u^0 (1 + g \lambda \Delta t) \\ g &= 1 + g \lambda \Delta t \\ \Rightarrow g &= \frac{1}{1 - \lambda \Delta t} \end{aligned}$$

To determine the stability boundary, we set $|g| = 1$, i.e. $g = e^{i\theta}$,

$$\begin{aligned} e^{i\theta} &= \frac{1}{1 - \lambda \Delta t} \\ e^{i\theta} - \lambda \Delta t e^{i\theta} &= 1 \\ \Rightarrow \lambda \Delta t &= 1 - e^{-i\theta}, \end{aligned}$$

which is a circle of radius 1 centered at 1 in the complex number plane. Choosing the circle origin, $\lambda \Delta t = 1$, shows that the inside of the circle is unstable while the outside is stable. Therefore, we obtain the stability region shown in Figure 2.8. Note the much larger extent of

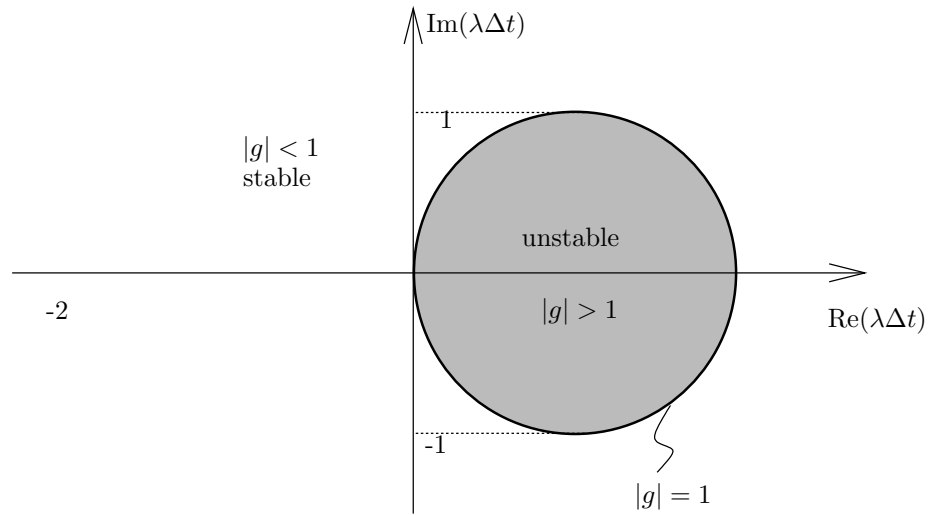


Figure 2.8: Eigenvalue stability region for the backward Euler method. The region inside the circle is unstable as it yields amplification factors with $|g| > 1$.

the backward Euler stability region compared to forward Euler. In particular, the entire left half plane in Figure 2.8 is stable. When this happens, we say that the method is A-stable.

2.4.3 Implications for Convergence

In order for a multistep time-integration method to be convergent, as defined in Section 2.3.3, it must be both consistent and stable. This result is summarized in the *Dahlquist Equivalence Theorem*, which states that a method is convergent if and only if it is zero stable and consistent. For consistency, we need at least first-order accuracy, i.e. for the truncation error to be at least $\mathcal{O}(\Delta t^2)$. For stability we are concerned with zero stability because the idea of convergence applies to the case $\Delta t \rightarrow 0$.

We gain some intuition for the Dahlquist Equivalence Theorem by looking at non-convergent situations. If a method is not zero stable, as in Example 2.9, then the error will explode after a few iterations regardless of the time step size and hence clearly we will not have convergence. If a method is not consistent, the solution might not blow up but instead it might “converge” to an incorrect value. Thus, we need both stability and consistency to see convergence.

2.5 Multi-Stage Methods

We saw that multi-step methods evaluate the state at time node $n+1$ using the state at time node n and possibly at other preceding time nodes. The higher the order of a multi-step method, the more preceding states such a method uses. Since, all of the previous states are already known, calculating the next state is relatively inexpensive. However, there are some drawbacks. A practical one is the question of what to do at the initial time steps, where the required number of preceding states is not available. The answer here is usually to use a low-order method until enough states are available. Another drawback is that accurate multi-step methods generally have poor stability for nonzero eigenvalues close to the imaginary axis (e.g. oscillatory modes of a system). Multi-stage methods address both of these drawbacks, albeit at additional cost per time step.

As illustrated in Figure 2.9, multi-stage methods use temporary intermediate states to compute the state at time node $n+1$ using only the previous state at n .

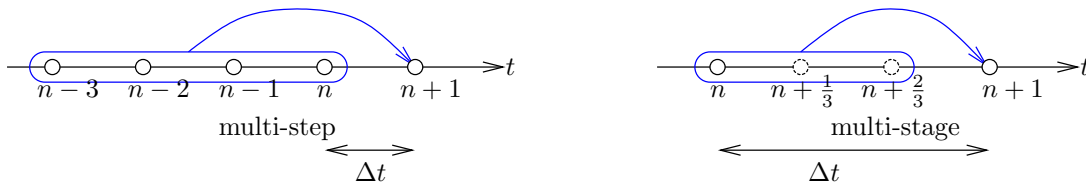


Figure 2.9: Schematic comparison of multi-step versus multi-stage time-integration methods. In multi-stage integration, the intermediate fractional states are temporary: they are only constructed for the purpose of calculating the $n+1$ state from the n state, and they are discarded in the next time step.

2.5.1 Predictor-Corrector Methods

The simplest multi-stage method is predictor-corrector integration, already introduced in Section 2.1.2. There are various forms of this method, one of them being the already-presented mix of forward Euler and trapezoidal integration:

$$\begin{aligned}\tilde{\mathbf{u}}^{n+1} &= \mathbf{u}^n + \Delta t \mathbf{f}(\mathbf{u}^n, t^n) \\ \mathbf{u}^{n+1} &= \mathbf{u}^n + \Delta t \frac{1}{2} (\mathbf{f}(\mathbf{u}^n, t^n) + \mathbf{f}(\tilde{\mathbf{u}}^{n+1}, t^{n+1})) .\end{aligned}$$

Note that here, $\tilde{\mathbf{u}}^{n+1}$ is the temporary intermediate state. This method is second-order accurate because it uses the average of the slope at the current state and at a (rough) prediction of the $n + 1$ state.

Example 2.12 (Eigenvalue Stability of the Predictor-Corrector Method). For multi-stage methods we can use the same eigenvalue stability analysis procedure that we used for multi-step methods. That is, we consider a homogeneous scalar eigenvalue problem, $f(u) = \lambda u$, and an exponentially-growing/decaying state, $u^n = g^n u^0$. Combining the two stages of predictor-corrector into one equation, we have

$$\begin{aligned}u^{n+1} &= u^n + \Delta t \frac{1}{2} [f(u^n, t^n) + f(\tilde{u}^{n+1}, t^{n+1})] \\ &= u^n + \Delta t \frac{1}{2} [\lambda u^n + \lambda \tilde{u}^{n+1}] \\ &= u^n + \Delta t \frac{1}{2} [\lambda u^n + \lambda (u^n + \Delta t \lambda u^n)] \\ &= u^n \left[1 + \lambda \Delta t + \frac{1}{2} (\lambda \Delta t)^2 \right] \\ \Rightarrow g^{n+1} u^0 &= g^n u^0 \left[1 + \lambda \Delta t + \frac{1}{2} (\lambda \Delta t)^2 \right] \\ g &= 1 + \lambda \Delta t + \frac{1}{2} (\lambda \Delta t)^2\end{aligned}$$

Having solved for the amplification factor, we can now determine the region of stability. The stability boundary is given by those $\lambda \Delta t$ values that give $|g| = 1$, i.e. $g = e^{i\theta}$. Substituting $e^{i\theta}$ for g above, we obtain the following quadratic equation,

$$\begin{aligned}\frac{1}{2} (\lambda \Delta t)^2 + \lambda \Delta t + 1 - e^{i\theta} &= 0 \\ \Rightarrow \lambda \Delta t &= -1 \pm \sqrt{2e^{i\theta} - 1}\end{aligned}$$

Figure 2.10 shows a plot of this solution for all values of $\theta \in [0, 2\pi]$. Note the vertically-oval nature of the stability boundary. We particularly care about the behavior near the origin, where eigenvalues that are close to the imaginary axis now have a chance of being inside the stability boundary (e.g. for a sufficiently-small time step). Compare, for example, to the forward Euler stability boundary in Figure 2.7, which, as a circle, turns away from the origin much sooner.

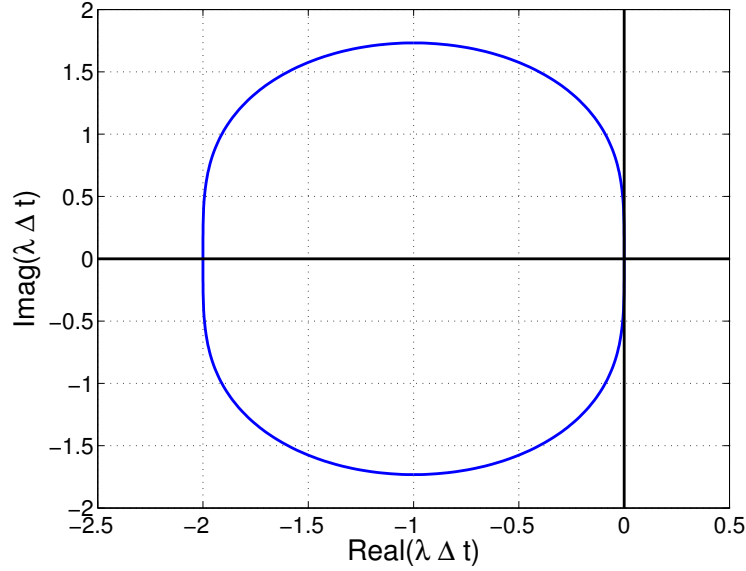


Figure 2.10: Stability boundary for the combined Euler/Trapezoidal predictor-corrector method. The region enclosed by the boundary is stable.

2.5.2 Runge-Kutta Methods

Probably the most widely-used multi-stage methods are of the Runge-Kutta family. In this section we discuss and analyze two such methods: a second-order and a fourth-order version.

2.5.2.1 Two-stage Runge-Kutta

A popular two-stage method is *modified Euler*,

$$\begin{aligned} \mathbf{u}^{n+\frac{1}{2}} &= \mathbf{u}^n + \frac{1}{2}\Delta t \mathbf{f}(\mathbf{u}^n, t^n) \\ \mathbf{u}^{n+1} &= \mathbf{u}^n + \Delta t \mathbf{f}(\mathbf{u}^{n+\frac{1}{2}}, t^{n+\frac{1}{2}}) \end{aligned} \quad (2.18)$$

In the first stage, forward Euler is used to obtain an approximation to the state at the midpoint of the time step. The slope, $\mathbf{f}$, at this midpoint state is then used to advance the state from node n to $n+1$. Figure 2.11 presents these steps graphically for a scalar problem.

Example 2.13 (Order of Accuracy of Modified Euler). We already mentioned that the modified Euler method is second-order accurate. Let's show this for a scalar problem by calculating the local error in one time step: $n \rightarrow n+1$. In the first stage, we recognize that

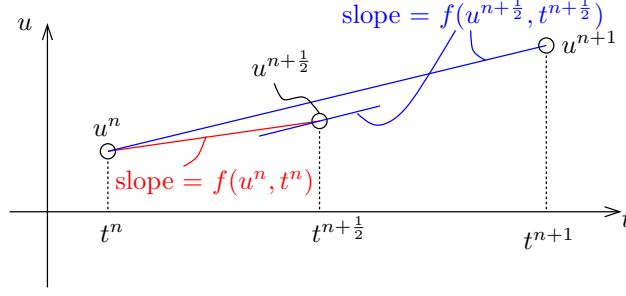


Figure 2.11: Graphical representation of the two stages in modified Euler time integration.

$f(u^n, t^n) = u_t^n$, the time derivative of the state at n ,

$$u^{n+\frac{1}{2}} = u^n + \frac{\Delta t}{2} u_t^n.$$

Now, in the second stage,

$$f(u^{n+\frac{1}{2}}, t^{n+\frac{1}{2}}) = \frac{d}{dt} \left(u^n + \frac{\Delta t}{2} u_t^n \right) = u_t^n + \frac{\Delta t}{2} u_{tt}^n.$$

So the second stage becomes

$$u^{n+1} = u^n + \Delta t \left(u_t^n + \frac{\Delta t}{2} u_{tt}^n \right) = u^n + \Delta t u_t^n + \frac{\Delta t^2}{2} u_{tt}^n.$$

But this last expression is just a truncated Taylor-series expansion of u about t^n . The leading missing term is $\mathcal{O}(\Delta t^3)$, which is then the truncation error, or local order of accuracy. Since the global order of accuracy is of one order lower, we have that the global order of accuracy is $\mathcal{O}(\Delta t^2)$, and the method is second-order accurate.

Example 2.14 (Stability of Modified Euler). Following Example 2.12, we combine the two stages of modified Euler for a scalar problem, $f(u) = \lambda u$, into one equation,

$$\begin{aligned} u^{n+1} &= u^n + \Delta t \lambda \left[u^n + \frac{1}{2} \Delta t \lambda u^n \right] \\ &= u^n + \lambda \Delta t u^n + \frac{1}{2} (\lambda \Delta t)^2 u^n \\ \Rightarrow g^{n+1} u^0 &= g^n u^0 \left[1 + \lambda \Delta t + \frac{1}{2} (\lambda \Delta t)^2 \right] \\ g &= 1 + \lambda \Delta t + \frac{1}{2} (\lambda \Delta t)^2 \end{aligned}$$

This is the same amplification factor as for the predictor-corrector method in Example 2.12, and hence the stability region is as shown in Figure 2.10. In summary, modified Euler is a second-order accurate explicit time integration method.

2.5.2.2 Four-stage Runge-Kutta

One of the most popular four-stage integration methods is the following sequence:

$$\begin{aligned}
 \mathbf{f}_0 &= \mathbf{f}(\mathbf{u}^n, t^n) \\
 \mathbf{f}_1 &= \mathbf{f}\left(\mathbf{u}^n + \frac{1}{2}\Delta t\mathbf{f}_0, t^n + \frac{\Delta t}{2}\right) \\
 \mathbf{f}_2 &= \mathbf{f}\left(\mathbf{u}^n + \frac{1}{2}\Delta t\mathbf{f}_1, t^n + \frac{\Delta t}{2}\right) \\
 \mathbf{f}_3 &= \mathbf{f}(\mathbf{u}^n + \Delta t\mathbf{f}_2, t^n + \Delta t) \\
 \mathbf{u}^{n+1} &= \mathbf{u}^n + \frac{\Delta t}{6}(\mathbf{f}_0 + 2\mathbf{f}_1 + 2\mathbf{f}_2 + \mathbf{f}_3)
 \end{aligned} \tag{2.19}$$

Note that we present this method slightly differently from the previous ones, in terms of $\mathbf{f}$ -values instead of state values. This is because that's how one would implement this method for efficiency, in order to minimize calculations of $\mathbf{f}$ which are typically the most expensive parts of the time integration. Although they are hidden, we do have three temporary states in the above sequence, namely $\mathbf{u}^n + \frac{1}{2}\Delta t\mathbf{f}_0$, $\mathbf{u}^n + \frac{1}{2}\Delta t\mathbf{f}_1$, and $\mathbf{u}^n + \Delta t\mathbf{f}_2$.

Example 2.15 (Stability of Fourth-Order Runge-Kutta). For the linear scalar problem $f(u) = \lambda u$, we can combine all four stages in Eqn. 2.19 into one equation,

$$\mathbf{u}^{n+1} = \mathbf{u}^n \left(1 + \lambda\Delta t + \frac{1}{2}(\lambda\Delta t)^2 + \frac{1}{6}(\lambda\Delta t)^3 + \frac{1}{24}(\lambda\Delta t)^4 \right)$$

Substituting $u^n = g^n u^0$ gives us the amplification factor

$$g = 1 + \lambda\Delta t + \frac{1}{2}(\lambda\Delta t)^2 + \frac{1}{6}(\lambda\Delta t)^3 + \frac{1}{24}(\lambda\Delta t)^4$$

We can attempt to set $g = e^{i\theta}$ and to solve for $\lambda\Delta t$, but this is difficult as we would have to solve a fourth-order polynomial equation. A simpler method of obtaining the stability boundary is to plot contours of $|g|$ in the real/imaginary complex number plane of $\lambda\Delta t$ and to identify the $|g| = 1$ contour. Listing 2.3 presents a code that makes such a contour plot, and Figure 2.12 shows the result.

Listing 2.3: Code for computing the stability boundary of fourth-order Runge-Kutta integration.

```
1 function stabRK4;
2 % PURPOSE:
3 % Plots the stability region for four-stage Runge-Kutta
4 % INPUTS: none
5 % OUTPUTS: none, makes a plot
6
7 % grid of points in complex lambda-delta t space
8 xv = linspace(-3,1,400);
9 yv = linspace(-3.5,3.5,400);
10 [x,y] = meshgrid(xv,yv);
11 z = x + i*y; % z = lambda * delta t
12 g = 1 + z + 0.5*z.^2 + (1/6)*z.^3 + (1/24)*z.^4; % amplification factor
13 gmag = abs(g); % magnitude
14
15 % make contour plot
16 figure(1); clf;
17 contour(x,y,gmag,[1,1.05],'b-', 'linewidth', 2); hold on;
18 xlabel('Real_(\lambda\Delta t)', 'fontsize', 16);
19 ylabel('Imag_(\lambda\Delta t)', 'fontsize', 16);
20 plot([0, 0], [-3.5, 3.5], 'k-', 'linewidth', 2); % y axis
21 plot([-3, 1], [0, 0], 'k-', 'linewidth', 2); % x axis
22 grid on;
23 axis equal;
24 axis([-3, 1, -3.5,3.5]);
25
26 % print figure to pdf
27 set(gca, 'fontsize', 16);
28 s = '../figs/stabRK4.eps';
29 print('-depsc2', s);
30 unix(sprintf('epstopdf_%s;rm_%s', s, s));
```

Testing a point such as $\lambda\Delta t = -1$ shows that the interior of the contour is stable. Note that the stability boundary encloses portions of the imaginary axis – this is good for stability of lightly-damped oscillatory modes. Also, the stability boundary extends further along the negative real axis compared to forward Euler and second-order Runge-Kutta, which means that the time steps could be larger for a given eigenvalue. Finally, the name “four-stage” is given to this method because a minimum of four function evaluations, $\mathbf{f}(\mathbf{u}, t)$, are required to take one time step. Equivalently, there are three intermediate states.

2.6 Stiffness and Implicit Methods

2.6.1 Stiffness

We call a system of equations *stiff* if it exhibits a wide range of time scales. Physically, this means that the system contains both fast and slow processes. For example, imagine slowly stretching and compressing a stiff spring: your applied forcing function is slow, but the spring’s response to any disturbances is very quick. Stiffness also occurs for systems without forcing if some responses are much faster compared to others. Mathematically, for linear or linearized systems, this means that the magnitudes of the system eigenvalues are

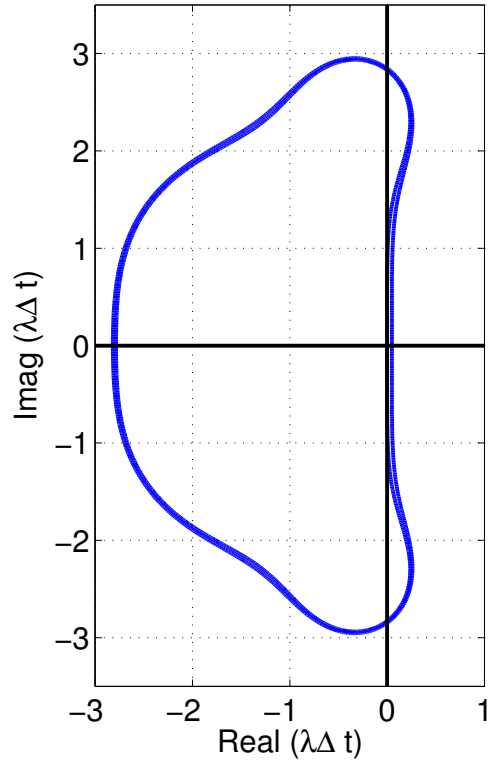


Figure 2.12: Stability boundary for the fourth-order Runge-Kutta method in Eqn. 2.19. The region inside the contour is stable.

highly-disparate. A rule of thumb is that if the ratio of the largest to smallest eigenvalue magnitudes, $|\lambda_{\max}|/|\lambda_{\min}|$ is greater than about 1000, the system is likely stiff.

As an example, consider the following system:

$$\dot{\mathbf{u}} = \mathbf{A}\mathbf{u}, \quad \text{where } \mathbf{A} = \begin{bmatrix} -1 & 1 \\ 0 & -1000 \end{bmatrix} \quad (2.20)$$

The eigenvalues are $\lambda_1 = -1$ and $\lambda_2 = -1000$. Suppose we wanted to integrate this system forward in time, starting from $t = 0, u = [1, 1]^T$ to $t = 1$. Using forward Euler, we calculate the time step restriction from Figure 2.7 as $\Delta t_{\max} = .002$, since anything larger would put $\lambda_2 \Delta t$ in the unstable region, $\lambda \Delta t < -2$. Let's see how the solution looks for various Δt .

Figure 2.13 shows the forward Euler solution versus time for three time steps, $\Delta t = .001$, $\Delta t = .002$, and $\Delta t = .0021$. We make several observations about these results. First, the eigenvalue stability analysis that we did earlier proves correct, since for time steps $\Delta t > .002$ the integration is unstable: u_1 and u_2 exhibit exponentially-growing oscillations. Second, for this problem, the decay of u_1 is the slow, “long-time”, phenomenon, while the decay of u_2 is a fast transient. Third, at the stability limit, we already see oscillations in u_2 , and while they do not grow, they also do not go away. Fourth, beyond the stability limit, oscillations in u_2 grow ... and they pollute u_1 through the coupling of the states in the system. Finally, we note that if we only care about the long-time behavior exhibited by u_1 , which has a time scale of $\mathcal{O}(1)$, then the maximum time step of $\Delta t = .002$ set by the stability restriction is likely excessively small. The remedy for this situation is an implicit time integration method.

2.6.2 Implicit Methods for Linear Systems

In Section 2.2 we made a distinction between *explicit* and *implicit* multi-step integration methods. In the following we will restrict our attention to multi-step methods, although multi-stage and other time integration methods can also be classified as explicit or implicit.

In an implicit multi-step method, the update formula for $\mathbf{u}^{n+1}$, i.e. Eqn. 2.4, depends on the unknown $\mathbf{u}^{n+1}$ through the forcing function $\mathbf{f}$. This means that we cannot evaluate $\mathbf{u}^{n+1}$ as easily as in an explicit method because $\mathbf{u}^{n+1}$ also appears on the right-hand side of the equation. However, on the face of it this is not a huge problem: we just have to re-arrange the update equation to put the unknown $\mathbf{u}^{n+1}$ on the left-hand side. Let's see how this works for a prototypical implicit time integration method: backward Euler.

Example 2.16 (The Backward Euler Method for a Stiff Linear System). Consider again the 2×2 system in Eqn. 2.20. The backward Euler update equation for this system reads

$$\mathbf{u}^{n+1} - \mathbf{u}^n = \Delta t \mathbf{A} \mathbf{u}^{n+1}. \quad (2.21)$$

The only difference between this equation and a forward Euler update is the last superscript $n + 1$, on the right-hand side, which would be n for forward Euler. However, this is a big difference for the solution approach and for stability.

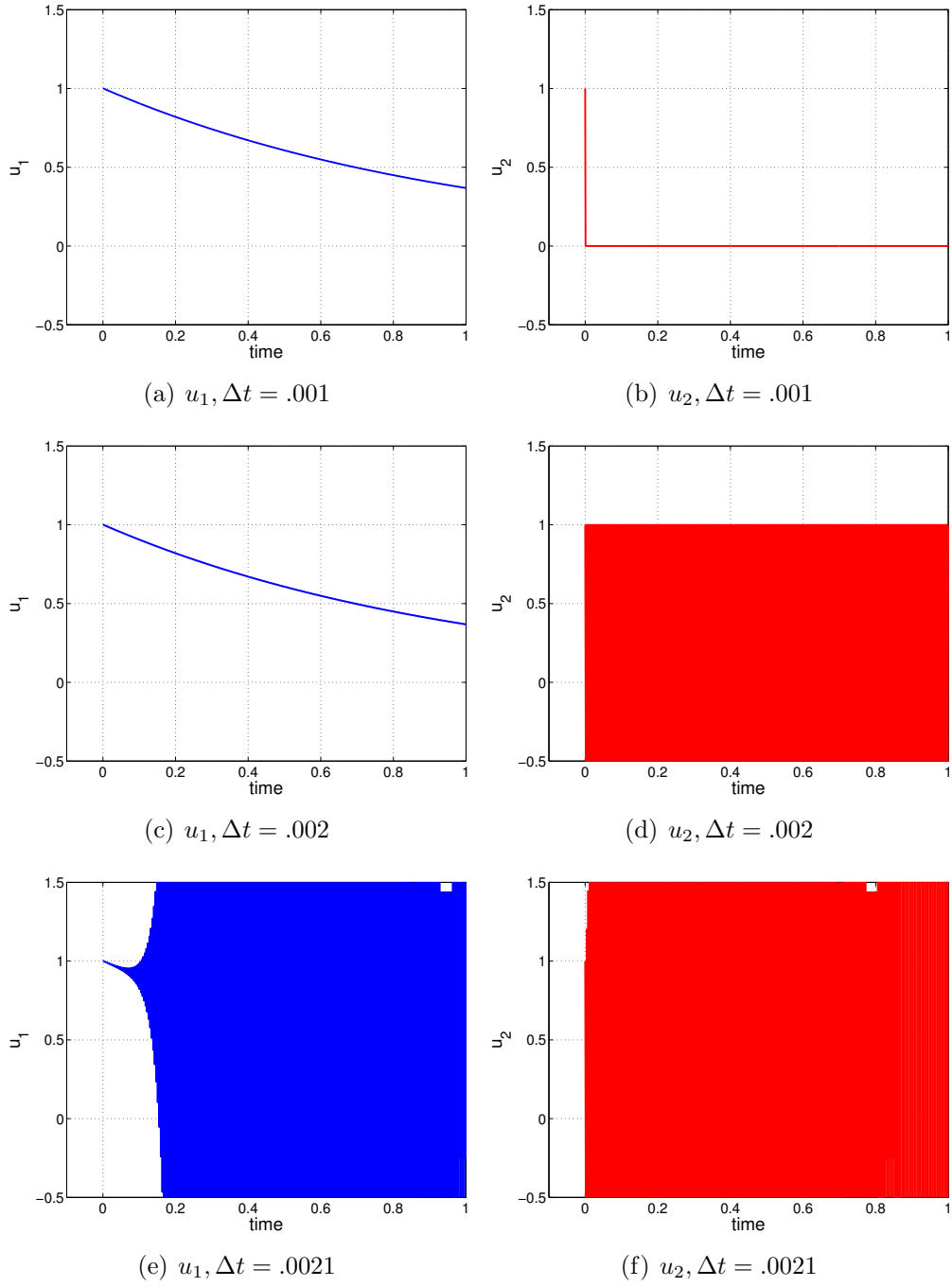


Figure 2.13: Forward Euler time integration results for the stiff system given in Eqn. 2.20.

To solve Eqn. 2.21 we put terms with the unknown $\mathbf{u}^{n+1}$ on the left-hand side and everything else on the right-hand side,

$$\begin{aligned}\mathbf{u}^{n+1} - \Delta t \mathbf{A} \mathbf{u}^{n+1} &= \mathbf{u}^n \\ (\mathbf{I} - \Delta t \mathbf{A}) \mathbf{u}^{n+1} &= \mathbf{u}^n \\ \mathbf{u}^{n+1} &= (\mathbf{I} - \Delta t \mathbf{A})^{-1} \mathbf{u}^n\end{aligned}$$

For linear systems, the matrix inversion can be precomputed before the simulation, and this saves redundant calculations at each time step.

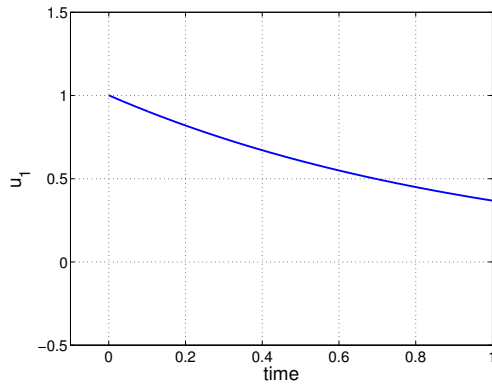
Turning back to the problem at hand, Eqn. 2.20, we integrate the 2×2 system from $t = 0, u = [1, 1]^T$ to $t = 1$ – the same setup as we had for forward Euler. For backward Euler, there is no time step restriction, as indicated in Figure 2.8, and we run with a wider range of time steps. Figure 2.14 shows the backward Euler solution versus time for three time steps, $\Delta t = .001$, $\Delta t = .01$, and $\Delta t = .1$. The lack of time step restriction allows us to obtain stable results with time steps that are much larger than we could use for an explicit method. For $\Delta t = .1$, we only have 10 time steps in the simulation. In this case, the transient in u_2 is severely under-resolved, as expected since this transient is very fast. On the other hand, the u_1 solutions are very similar for all three time steps.

The previous example showed that solving for $\mathbf{u}^{n+1}$ requires inverting an $N \times N$ system. This will be the case for all implicit methods, not just backward Euler. Depending on the properties of $\mathbf{A}$, the matrix inversion could be expensive – up to $\mathcal{O}(N^3)$ for general dense matrices. This cost has to be factored into the decision when choosing between explicit and implicit methods.

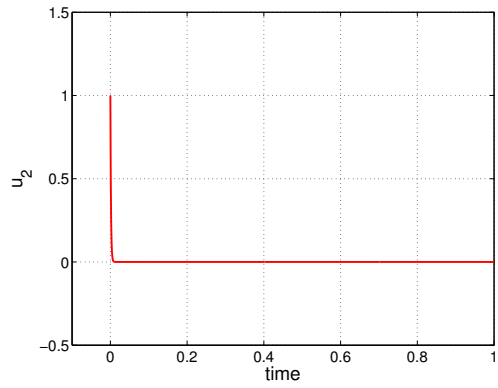
Another consideration besides stability when choosing between time integration methods is accuracy. In Figure 2.15 we compare the accuracy versus time step of several popular explicit and implicit methods for solving the system in Eqn. 2.20. We measure accuracy of the solution we care about, $u_1(t)$, and we define the error as the maximum difference over the simulated time history between the computed solution and the analytical solution, which is

$$u_1(t) = \frac{1000e^{-t} - e^{-1000t}}{999}.$$

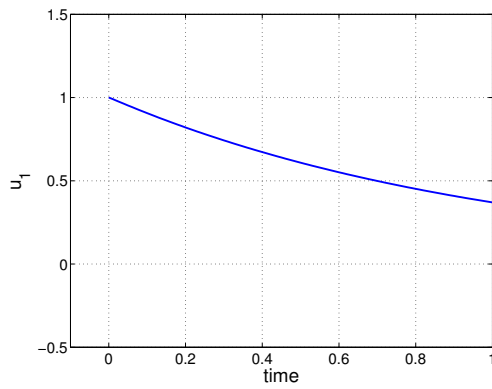
The results in Figure 2.15 show that explicit methods are desirable when high accuracy is required, in this case error levels below approximately 10^{-3} . For errors below this, the time step restriction as dictated by the accuracy requirement is below the stability limit and hence the cheaper explicit methods make sense. However, if the accuracy requirement is less stringent, say an error level of 10^{-2} , the explicit methods do not really get any cheaper because they are stuck with the small time step restriction due to stability. On the other hand, the implicit methods can deliver the desired less-stringent accuracy with many fewer time steps, almost two orders of magnitude in our particular example.



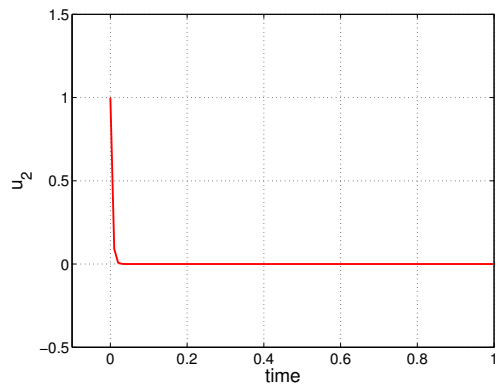
(a) $u_1, \Delta t = .001$



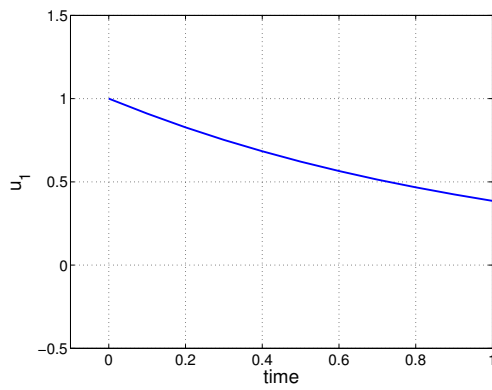
(b) $u_2, \Delta t = .001$



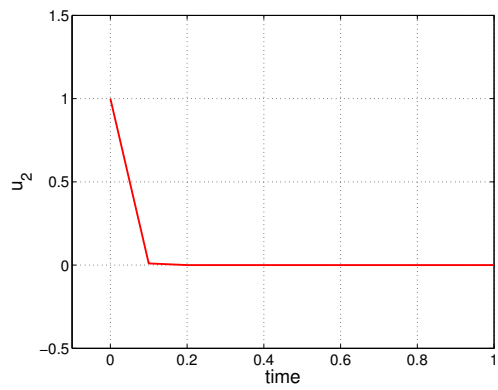
(c) $u_1, \Delta t = .01$



(d) $u_2, \Delta t = .01$



(e) $u_1, \Delta t = .1$



(f) $u_2, \Delta t = .1$

Figure 2.14: Backward Euler time integration results for the stiff system given in Eqn. 2.20.

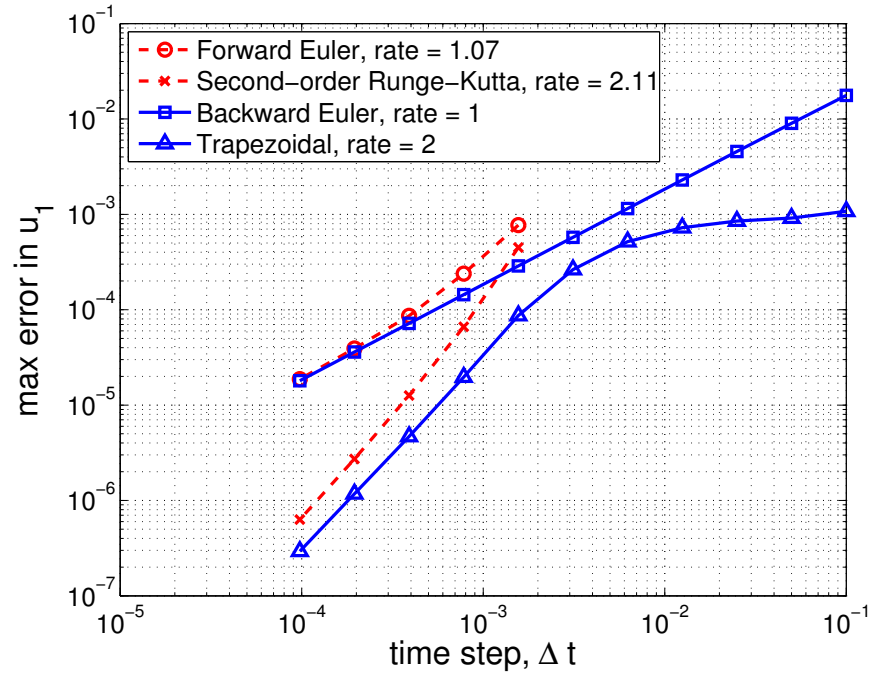


Figure 2.15: Accuracy comparison of time integration methods for the stiff system in Eqn. 2.20. The two explicit methods (dashed red) are unstable for the larger time steps, and hence their accuracy results are not shown over the entire Δt range. The convergence rates given in the legend are measured between the two smallest time-step simulations.

Finally, we remark that Figure 2.15 also shows the convergence rates for each of the methods. These are measurements of what is expected to be the global convergence rate, e.g. the p in Eqn. 2.12. We measure this rate by expressing the error as $\epsilon^{\text{global}} = C\Delta t^p$ and using the ratio of the errors at the last two points (smallest Δt) to calculate p . We see that the rates agree almost spot-on with the theoretical expectations.

2.6.3 Implicit Methods for Nonlinear Systems

Nonlinear ODEs are ones in which $\mathbf{f}(\mathbf{u}, t)$ depends non-linearly on the state $\mathbf{u}$ – this will be the case in most practical applications. Implicit methods applied to nonlinear systems require special attention because the unknown state $\mathbf{u}^{n+1}$ cannot be easily isolated out of $\mathbf{f}(\mathbf{u}^{n+1}, t^{n+1})$. Instead, we have to solve a nonlinear system of algebraic equations to compute the updated state.

Consider for example a nonlinear, time-invariant system,

$$\dot{\mathbf{u}} = \mathbf{f}(\mathbf{u}).$$

We note that the changes required for non time-invariant systems are straightforward and just require keeping track of the time at which $\mathbf{f}$ needs to be evaluated. Applying our prototypical implicit method, backward Euler, gives the update formula,

$$\begin{aligned} \mathbf{u}^{n+1} &= \mathbf{u}^n + \Delta t \mathbf{f}(\mathbf{u}^{n+1}) \\ \mathbf{u}^{n+1} - \Delta t \mathbf{f}(\mathbf{u}^{n+1}) &= \mathbf{u}^n \\ \underbrace{\mathbf{u}^{n+1} - \Delta t \mathbf{f}(\mathbf{u}^{n+1}) - \mathbf{u}^n}_{\mathbf{R}(\mathbf{u}^{n+1})} &= 0 \end{aligned}$$

This is a nonlinear system of equations which we can solve using the *Newton-Raphson method*. The first step in Newton-Raphson is to rearrange the equations to define a *residual*, $\mathbf{R}(\mathbf{u}^{n+1})$, as we did in the last line above. We now pose the following problem:

$$\text{Determine } \mathbf{w} \text{ such that } \mathbf{R}(\mathbf{w}) = 0. \quad (2.22)$$

Here $\mathbf{w}$ is just a temporary variable that we will update in an iterative manner until we drive the residual to zero, at which point $\mathbf{w}$ will be our desired state $\mathbf{u}^{n+1}$.

So how do we solve the problem in Eqn. 2.22? With the Newton-Raphson method we start with a guess for $\mathbf{w}$ – a good one is usually the most recent known state, $\mathbf{u}^n$. Then, we iteratively update $\mathbf{w}$ by linearizing the residual and driving it to zero. That is, given a value of $\mathbf{w}$ for which $\mathbf{R}(\mathbf{w}) \neq 0$, we seek an update $\Delta \mathbf{w}$ such that

$$\mathbf{R}(\mathbf{w} + \Delta \mathbf{w}) \approx 0.$$

Linearizing the left-hand side and dropping high-order terms, we have

$$\mathbf{R}(\mathbf{w}) + \left. \frac{\partial \mathbf{R}}{\partial \mathbf{w}} \right|_{\mathbf{w}} \Delta \mathbf{w} = 0, \quad (2.23)$$

where $\left. \frac{\partial \mathbf{R}}{\partial \mathbf{w}} \right|_{\mathbf{w}}$ is called the *residual Jacobian matrix*. It is the derivative of the residual vector with respect to the state. In our linear example, Example 2.16, this was just $\mathbf{I} - \Delta t \mathbf{A}$. For general nonlinear problems, the residual Jacobian will depend on the state, and so that's why we indicate the state ($\mathbf{w}$) about which the linearization is performed.

Solving Eqn. 2.23 for $\Delta \mathbf{w}$ requires inverting a linear system,

$$\Delta \mathbf{w} = - \left[\left. \frac{\partial \mathbf{R}}{\partial \mathbf{w}} \right|_{\mathbf{w}} \right]^{-1} \mathbf{R}(\mathbf{w}).$$

Once $\Delta \mathbf{w}$ is available, we update $\mathbf{w} \rightarrow \mathbf{w} + \Delta \mathbf{w}$ and repeat. We usually have to repeat because the residual at the updated state will still likely be nonzero (note that we dropped high-order terms to obtain the above linear equation). The number of times we repeat the Newton-Raphson iteration depends on the required accuracy, which is typically given as a criterion on a norm of the residual, $\|\mathbf{R}\|$. The method converges very quickly, in fact quadratically when close to the solution, so driving the residual norm to very low values is usually not difficult.

With the multiple matrix solves and residual/Jacobian evaluations, the cost of implicit methods relative to explicit ones certainly goes up for nonlinear problems. Yet, in practice, we still have very stiff nonlinear problems that make implicit methods desirable.

Example 2.17 (Newton-Raphson for Root Finding). Consider the following nonlinear equation,

$$R(w) = w^3 + 2w - 3 = 0,$$

which has a single real root at $w = 1$. In this example we will implement the Newton-Raphson method to solve this equation given a starting guess, w^0 .

First, at a given w , we linearize the above equation to determine the Newton-Raphson update, Δw . The linearized equation is

$$\begin{aligned} \frac{\partial R}{\partial w} \Delta w &= -R(w) \\ (3w^2 + 2) \Delta w &= -w^3 - 2w + 3 \end{aligned}$$

Solving for Δw gives us the Newton-Raphson update,

$$\Delta w = \frac{-w^3 - 2w + 3}{3w^2 + 2}$$

After implementing this formula numerically, we try two values for w^0 : 0, 10, and we tabulate the convergence of w to the exact solution:

$w^0 = 0$	$w^0 = 10$
Iteration 1: error = 5.0000E-01	Iteration 1: error = 5.6325E+00
Iteration 2: error = 1.1429E-01	Iteration 2: error = 3.3780E+00
Iteration 3: error = 7.3659E-03	Iteration 3: error = 1.8710E+00
Iteration 4: error = 3.2426E-05	Iteration 4: error = 8.8302E-01
Iteration 5: error = 6.3086E-10	Iteration 5: error = 2.9406E-01
	Iteration 6: error = 4.4175E-02
	Iteration 7: error = 1.1434E-03
	Iteration 8: error = 7.8394E-07
	Iteration 9: error = 3.6882E-13

An initial guess far from the root requires more Newton iterations to converge. This is because the linearization becomes less and less valid the further we move away from the root. However, note that once the approximate value is close to the root, the convergence is very rapid: the error decreases quadratically in the last few iterations in each case.

Example 2.18 (The Trapezoidal Method for a Nonlinear System). Consider the following nonlinear modification to the linear system in Eqn. 2.20,

$$\begin{aligned}\dot{u}_1 &= -u_1 + u_2^2 \\ \dot{u}_2 &= -1000u_2\end{aligned}$$

Note that u_2 now appears quadratically in the first equation. We write this as

$$\dot{\mathbf{u}} = \mathbf{f}(\mathbf{u}),$$

where $\mathbf{f} = [f_1, f_2]$, and $f_1 = -u_1 + u_2^2$, $f_2 = -1000u_2$.

Trapezoidal integration, presented in Section 2.2, consists of the following update formula,

$$\mathbf{u}^{n+1} - \mathbf{u}^n = \Delta t \left(\frac{1}{2} \mathbf{f}(\mathbf{u}^{n+1}) + \frac{1}{2} \mathbf{f}(\mathbf{u}^n) \right).$$

Putting all terms on the left-hand side gives us the residual,

$$\mathbf{R}(\mathbf{u}^{n+1}) \equiv \mathbf{u}^{n+1} - \mathbf{u}^n - \Delta t \left(\frac{1}{2} \mathbf{f}(\mathbf{u}^{n+1}) + \frac{1}{2} \mathbf{f}(\mathbf{u}^n) \right) = 0.$$

Rewriting the residual in terms of $\mathbf{w} = [w_1, w_2]^T$, we have

$$\mathbf{R}(\mathbf{w}) \equiv \mathbf{w} - \mathbf{u}^n - \Delta t \left(\frac{1}{2} \mathbf{f}(\mathbf{w}) + \frac{1}{2} \mathbf{f}(\mathbf{u}^n) \right) = 0.$$

The residual Jacobian matrix is then

$$\frac{\partial \mathbf{R}}{\partial \mathbf{w}} = I - \frac{1}{2} \Delta t \frac{\partial \mathbf{f}}{\partial \mathbf{w}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - \frac{1}{2} \Delta t \begin{bmatrix} -1 & 2w_2 \\ 0 & -1000 \end{bmatrix}$$

A Matlab code that implements the Newton-Raphson method for this residual is given in Listing 2.4.

Listing 2.4: Code for trapezoidal integration of a nonlinear problem

```
1 function [t,u] = NonLinTrap(N);
2 % PURPOSE:
3 % Performs trapezoidal integration of a nonlinear problem
4 % INPUTS:
5 % N : number of time steps
6 % OUTPUTS:
7 % t : vector of N+1 time node values
8 % u : 2 by N+1 solution vector
9
10 T = 1; % length of simulation
11 dt = T/N; % time step
12 t = linspace(0,T,N+1); % vector of time nodes
13 u = zeros(2,N+1); % state vector
14 u(:,1) = [1;1]; % initial condition
15 Rnorm = 1e-8; % Newton-Raphson residual tolerance
16
17 for n = 1:N, % loop over time steps
18
19     fn = calcf(u(:,n)); % f-value at u^n
20     w = u(:,n); % Newton-Raphson initialization
21
22     for k=1:10, % allow max 10 Newton-Raphson iterations
23         R = w - u(:,n) - dt*(0.5*calcf(w) + 0.5*fn);
24         if (norm(R) < Rnorm) break; end
25         R_w = eye(2) - 0.5*dt*calcf_u(w);
26         dw = -R_w \ R;
27         w = w + dw;
28     end
29
30     % make sure the residual tolerance was met
31     if (norm(R) > Rnorm), error('residual_norm_not_met!'); end;
32
33     u(:,n+1) = w; % store solution as next state
34
35 end
36
37 %-----
38 function f = calcf(u)
39 f = [-u(1) + u(2)^2; -1000*u(2)];
40
41 %-----
42 function f_u = calcf_u(u)
43 f_u = [-1, 2*u(2); 0, -1000];
```

Running this code with various time steps (i.e. values of N) gives the error convergence plot shown in Figure 2.16. As the error we use the maximum difference between the computed u_1 and the exact $u_1(t)$, which is

$$u_1(t) = \frac{2000e^{-t} - e^{-2000t}}{1999}.$$

Note that we achieve second-order accuracy as expected for the trapezoidal method, and that we attain stability even for large Δt .

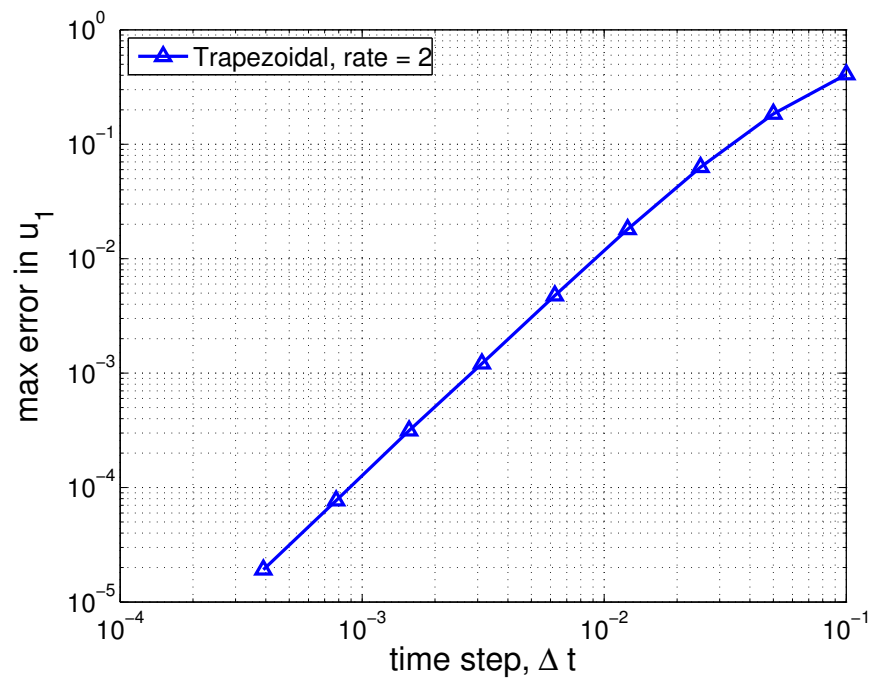


Figure 2.16: Convergence with time step of trapezoidal integration for a nonlinear 2×2 system.

Chapter 3

Finite Difference Methods

3.1 Finite Difference Approximations

Finite difference approximations are widely used in discretizing ordinary and partial differential equations. One of the simplest finite difference formulas is that of a first derivative,

$$\left. \frac{du}{dx} \right|_x \approx \frac{u(x + \Delta x) - u(x)}{\Delta x}, \quad (3.1)$$

where in the limit $\Delta x \rightarrow 0$ we recover the standard definition of a derivative. We can use this formula to approximate the slope of $u(x)$ at a point of a grid, as shown in Figure 3.1. Note that for now we assume a *uniform grid*, which is one in which the points are equally spaced. On this grid, we represent $u(x)$ through u_i , which are values of u at the $N + 1$ nodes, $x_i = i\Delta x$, $0 \leq i \leq N$.

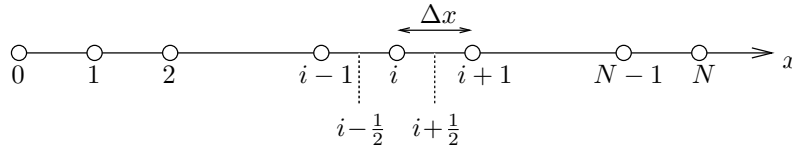


Figure 3.1: Uniform grid used for one-dimensional finite difference formulas. The spacing between all consecutive nodes is Δx .

If we're sitting at node i , we can look *forward* to $i + 1$ and calculate the first derivative using Eqn. 3.1,

$$\text{forward difference: } \left. \frac{du}{dx} \right|_i \approx \frac{u_{i+1} - u_i}{\Delta x}. \quad (3.2)$$

But we can also look *backward* to node $i - 1$,

$$\text{backward difference: } \left. \frac{du}{dx} \right|_i \approx \frac{u_i - u_{i-1}}{\Delta x}. \quad (3.3)$$

Or, for symmetry, we can average the forward and backward formulas to obtain

$$\text{central difference: } \left. \frac{du}{dx} \right|_i \approx \frac{u_{i+1} - u_{i-1}}{2\Delta x}. \quad (3.4)$$

We'll see in the next section that the central difference formula is more accurate than the forward or backward difference formulas.

We can apply the central-difference formula using consecutive nodes to get an approximation of the first derivative half-way between the nodes,

$$\left. \frac{du}{dx} \right|_{i+\frac{1}{2}} \approx \frac{u_{i+1} - u_i}{\Delta x}, \quad \text{and} \quad \left. \frac{du}{dx} \right|_{i-\frac{1}{2}} \approx \frac{u_i - u_{i-1}}{\Delta x}.$$

Note the use of $i \pm \frac{1}{2}$, as defined in Figure 3.1. Taking the difference of these two first derivatives gives us an approximation to the second derivative at node i ,

$$\left. \frac{d^2u}{dx^2} \right|_i \approx \frac{\left. \frac{du}{dx} \right|_{i+\frac{1}{2}} - \left. \frac{du}{dx} \right|_{i-\frac{1}{2}}}{\Delta x} \approx \frac{\frac{u_{i+1} - u_i}{\Delta x} - \frac{u_i - u_{i-1}}{\Delta x}}{\Delta x} = \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2}. \quad (3.5)$$

These are just a few popular finite difference formulas. Before we discuss how to derive general formulas, we first look at how to measure their accuracy.

3.2 Truncation Error Analysis

For a finite Δx , the forward difference formula in Eqn. 3.1 is just an approximation of the true first derivative at x . We identify the *truncation error* as the difference between this approximation when the exact u is used in the finite difference formula and the true derivative,

$$\text{forward difference truncation error} = \tau^{\text{forward}} \equiv \underbrace{\frac{u(x + \Delta x) - u(x)}{\Delta x}}_{\text{approximate derivative}} - \underbrace{\left. \frac{du}{dx} \right|_x}_{\text{true derivative}}. \quad (3.6)$$

This definition of the truncation error differs from that used for ODEs in Chapter 2 because here we are looking at the error in the derivative of interest, whereas before we were looking at the error in the updated state.

We can calculate the truncation error, or at least the rate of convergence with Δx , by using Taylor-series expansions. In the above equation, we expand $u(x + \Delta x)$ as

$$u(x + \Delta x) = u(x) + \Delta x \frac{du}{dx} + \frac{1}{2} \Delta x^2 \frac{d^2u}{dx^2} + \frac{1}{6} \Delta x^3 \frac{d^3u}{dx^3} + \mathcal{O}(\Delta x^4).$$

All derivatives above are taken at x . Using this expansion in Eqn. 3.6, the truncation error of our forward difference is

$$\begin{aligned} \tau^{\text{forward}} &= \frac{1}{\Delta x} \left(u(x) + \Delta x \frac{du}{dx} + \frac{1}{2} \Delta x^2 \frac{d^2u}{dx^2} + \frac{1}{6} \Delta x^3 \frac{d^3u}{dx^3} + \mathcal{O}(\Delta x^4) - u(x) \right) - \frac{du}{dx} \\ &= \frac{1}{2} \Delta x \frac{d^2u}{dx^2} + \frac{1}{6} \Delta x^2 \frac{d^3u}{dx^3} + \mathcal{O}(\Delta x^3). \end{aligned}$$

This shows that the forward difference is first-order accurate, meaning that the truncation error has a leading term of order Δx^1 . Note, here the leading term in τ gives the order of accuracy because we are looking at the error in a derivative, as opposed to the error in an updated states that we considered in ODEs. A similar analysis for the backwards difference scheme shows that it is also first-order accurate,

$$\begin{aligned}\tau^{\text{backward}} &= \frac{u(x) - u(x - \Delta x)}{\Delta x} - \left. \frac{du}{dx} \right|_x \\ &= \frac{1}{\Delta x} \left(u(x) - u(x) + \Delta x \frac{du}{dx} - \frac{1}{2} \Delta x^2 \frac{d^2u}{dx^2} + \frac{1}{6} \Delta x^3 \frac{d^3u}{dx^3} + \mathcal{O}(\Delta x^4) \right) - \frac{du}{dx} \\ &= -\frac{1}{2} \Delta x \frac{d^2u}{dx^2} + \frac{1}{6} \Delta x^2 \frac{d^3u}{dx^3} + \mathcal{O}(\Delta x^3).\end{aligned}$$

The central difference, $[u(x + \Delta x) - u(x - \Delta x)]/\Delta x$, is the average of the forward and backward differences, so its truncation error is

$$\begin{aligned}\tau^{\text{central}} &= \frac{1}{2} (\tau^{\text{forward}} + \tau^{\text{backward}}) \\ &= \frac{1}{6} \Delta x^2 \frac{d^3u}{dx^3} + \mathcal{O}(\Delta x^3).\end{aligned}$$

This shows that the central difference approximates the derivative to second order, as opposed to the first-order accuracy of forward and backward differences.

Example 3.1 (Second Derivative Difference Truncation Error). How accurate is the second-derivative difference in Eqn. 3.5?

$$\left. \frac{d^2u}{dx^2} \right|_i \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2}$$

Using a Taylor-series expansion about node i , we have

$$\begin{aligned}u_{i+1} &= u(x_i + \Delta x) = u_i + \Delta x \frac{du}{dx} + \frac{1}{2} \Delta x^2 \frac{d^2u}{dx^2} + \frac{1}{6} \Delta x^3 \frac{d^3u}{dx^3} + \mathcal{O}(\Delta x^4) \\ u_{i-1} &= u(x_i - \Delta x) = u_i - \Delta x \frac{du}{dx} + \frac{1}{2} \Delta x^2 \frac{d^2u}{dx^2} - \frac{1}{6} \Delta x^3 \frac{d^3u}{dx^3} + \mathcal{O}(\Delta x^4)\end{aligned}$$

All derivatives are taken at x_i . The truncation error is the error between the finite difference approximation via Eqn. 3.5 and the actual second derivative,

$$\begin{aligned}\tau &= \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2} - \frac{d^2u}{dx^2} \\ &= \frac{1}{\Delta x^2} \left(0 + 0 + \Delta x^2 \frac{d^2u}{dx^2} + 0 + \mathcal{O}(\Delta x^4) \right) - \frac{d^2u}{dx^2} \\ &= \mathcal{O}(\Delta x^2)\end{aligned}$$

So this finite difference gives a “second-order” accurate approximation of the second derivative.

Example 3.2 (An approximation to the fourth derivative). Let's derive a difference approximation to $\frac{d^4u}{dx^4}$ at node j by applying the second-order central difference approximation twice. The answer will involve nodes $j-2, j-1, j, j+1, j+2$.

At node j in our finite difference grid,

$$\begin{aligned}\frac{d^4u}{dx^4}(x_j) &\approx \frac{\frac{d^2u}{dx^2}(x_{j+1}) - 2\frac{d^2u}{dx^2}(x_j) + \frac{d^2u}{dx^2}(x_{j-1}))}{\Delta x^2} \\ &\approx \frac{(u_{j+2} - 2u_{j+1} + u_j) - 2(u_{j+1} - 2u_j + u_{j-1}) + (u_j - 2u_{j-1} + u_{j-2}))}{\Delta x^4} \\ &= \frac{u_{j+2} - 4u_{j+1} + 6u_j - 4u_{j-1} + u_{j-2}}{\Delta x^4}\end{aligned}$$

To determine the order of accuracy, we use Taylor-series expansions,

$$\begin{aligned}u_{j-2} &= u_j - 2\Delta x u_x + \frac{1}{2}(2\Delta x)^2 u_{xx} - \frac{1}{6}(2\Delta x)^3 u_{xxx} + \frac{1}{24}(2\Delta x)^4 u_{xxxx} \\ &\quad - \frac{1}{120}(2\Delta x)^5 u_{xxxxx} + \mathcal{O}(\Delta x^6) \\ u_{j-1} &= u_j - 1\Delta x u_x + \frac{1}{2}(1\Delta x)^2 u_{xx} - \frac{1}{6}(1\Delta x)^3 u_{xxx} + \frac{1}{24}(1\Delta x)^4 u_{xxxx} \\ &\quad - \frac{1}{120}(1\Delta x)^5 u_{xxxxx} + \mathcal{O}(\Delta x^6) \\ u_j &= u_j \\ u_{j+1} &= u_j + 1\Delta x u_x + \frac{1}{2}(1\Delta x)^2 u_{xx} + \frac{1}{6}(1\Delta x)^3 u_{xxx} + \frac{1}{24}(1\Delta x)^4 u_{xxxx} \\ &\quad + \frac{1}{120}(1\Delta x)^5 u_{xxxxx} + \mathcal{O}(\Delta x^6) \\ u_{j+2} &= u_j + 2\Delta x u_x + \frac{1}{2}(2\Delta x)^2 u_{xx} + \frac{1}{6}(2\Delta x)^3 u_{xxx} + \frac{1}{24}(2\Delta x)^4 u_{xxxx} \\ &\quad + \frac{1}{120}(2\Delta x)^5 u_{xxxxx} + \mathcal{O}(\Delta x^6)\end{aligned}$$

Summing these together with the given coefficients yields,

$$\begin{aligned}u_{j+2} - 4u_{j+1} + 6u_j - 4u_{j-1} + u_{j-2} &= 0u_j + 0u_x + 0u_{xx} + 0u_{xxx} + \\ &\quad (\Delta x)^4 u_{xxxx} + 0u_{xxxxx} + \mathcal{O}(\Delta x^6).\end{aligned}$$

Since we have to divide by Δx^4 , the leading error term that remains will be $\mathcal{O}(\Delta x^2)$ and so the finite difference approximation is second-order accurate.

3.3 General Finite-Difference Formulas

In the previous sections we presented a few common finite difference formulas that had simple interpretations related to the definition of the first derivative. Now we take a look at systematically deriving general finite difference formulas.

The problem statement reads as follows: given $l + r + 1$ consecutive points on a uniform grid, as illustrated in Figure 3.2, determine coefficients a_j , $-l \leq j \leq r$, such that the following summation (i.e. finite difference)

$$\sum_{j=-l}^r a_j u_j, \quad (3.7)$$

approximates a desired derivative quantity, usually at x_0 . Note that the finite difference *stencil* consists of the point in question, $j = 0$, as well as l points to the left and r points to the right. For example, for a forward difference, $l = 0, r = 1$; for a backward difference, $l = 1, r = 0$, and for a central difference, $l = r = 1$.

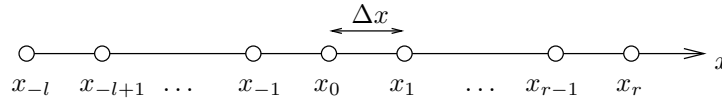


Figure 3.2: Points used in the derivation of a general finite-difference formula. The spacing between all consecutive nodes is Δx .

3.3.1 Undetermined coefficients

In this method we insert into Eqn. 3.7 Taylor-series expansions for u_j about $x = x_0$,

$$u_j = u_0 + (x_j - x_0) \frac{du}{dx} + \frac{1}{2}(x_j - x_0)^2 \frac{d^2u}{dx^2} + \dots$$

We then group powers of Δx and choose a_j so that we obtain an approximation of the desired derivative with the maximum possible accuracy.

Example 3.3 (One-sided Difference for $\frac{du}{dx}$). How accurate of an approximation can we obtain for the first derivative, $\frac{du}{dx}$, at node 0 using information from only one side? In particular, let's use two points on the left and no points on the right: $l = 2, r = 0$. So the players are u_{-2} , u_{-1} , and u_0 . We have (all derivatives are at x_0)

$$\begin{aligned} u_{-2} &= u_0 + (-2\Delta x) \frac{du}{dx} + \frac{1}{2}(-2\Delta x)^2 \frac{d^2u}{dx^2} + \frac{1}{6}(-2\Delta x)^3 \frac{d^3u}{dx^3} + \dots \\ u_{-1} &= u_0 + (-\Delta x) \frac{du}{dx} + \frac{1}{2}(-\Delta x)^2 \frac{d^2u}{dx^2} + \frac{1}{6}(-\Delta x)^3 \frac{d^3u}{dx^3} + \dots \end{aligned}$$

Inserting into Eqn. 3.7 and grouping powers of Δx leaves

$$\begin{aligned}
\sum_{j=-l}^r a_j u_j &= a_{-2} u_{-2} + a_{-1} u_{-1} + a_0 u_0 \\
&= a_{-2} \left(u_0 + (-2\Delta x) \frac{du}{dx} + \frac{1}{2} (-2\Delta x)^2 \frac{d^2 u}{dx^2} + \frac{1}{6} (-2\Delta x)^3 \frac{d^3 u}{dx^3} + \dots \right) \\
&\quad + a_{-1} \left(u_0 + (-\Delta x) \frac{du}{dx} + \frac{1}{2} (-\Delta x)^2 \frac{d^2 u}{dx^2} + \frac{1}{6} (-\Delta x)^3 \frac{d^3 u}{dx^3} + \dots \right) \\
&\quad + a_0 u_0 \\
&= (a_{-2} + a_{-1} + a_0) u_0 \\
&\quad + (-2a_{-2} - a_{-1}) \Delta x \frac{du}{dx} \\
&\quad + \left(2a_{-2} + \frac{1}{2} a_{-1} \right) \Delta x^2 \frac{d^2 u}{dx^2} \\
&\quad + \left(-\frac{4}{3} a_{-2} - \frac{1}{6} a_{-1} \right) \Delta x^3 \frac{d^3 u}{dx^3} \\
&\quad + \mathcal{O}(\Delta x^4)
\end{aligned}$$

We want this sum to approximate the first derivative, $\frac{du}{dx}$, as accurately as possible. But we only have three knobs, a_{-2}, a_{-1}, a_0 , so we can't get too ambitious about accuracy. Focusing on the first three terms above, we have the following equations:

$$\begin{aligned}
a_{-2} + a_{-1} + a_0 &= 0 \\
-2a_{-2} - a_{-1} &= \frac{1}{\Delta x} \\
2a_{-2} + \frac{1}{2} a_{-1} &= 0
\end{aligned}$$

Summing the last two of these, we obtain

$$-\frac{1}{2} a_{-1} = \frac{1}{\Delta x} \Rightarrow a_{-1} = -\frac{2}{\Delta x}$$

Substituting this result into the last equation,

$$a_{-2} = -\frac{1}{4} a_{-1} = \frac{1}{2\Delta x}.$$

Finally, substituting both results into the first equation gives

$$a_0 = -a_{-2} - a_{-1} = -\frac{1}{2\Delta x} + \frac{2}{\Delta x} = \frac{3}{2\Delta x}$$

So our one-sided approximation for the first derivative is

$$\frac{du}{dx} \approx \frac{3}{2\Delta x} u_0 - \frac{2}{\Delta x} u_{-1} + \frac{1}{2\Delta x} u_{-2} = \frac{3u_0 - 4u_{-1} + u_{-2}}{2\Delta x}$$

A quick check in the above equation with the Taylor-series expansions shows that the truncation error in this finite difference is $\mathcal{O}(\Delta x^2)$ – i.e. the difference is second-order accurate.

3.3.2 Lagrange interpolation

In this method, we derive a finite difference approximation by first fitting a polynomial of order $l + r$ to the u -values at the $l + r + 1$ points. We do this by Lagrange interpolation. We then calculate the desired derivative by differentiating the interpolated polynomial.

Specifically, the order $l + r$ polynomial that interpolates values u_j at the $l + r + 1$ points x_j is

$$\tilde{u}(x) = \sum_{j=-l}^r L_j(x) u_j, \quad \text{where} \quad L_j(x) = \prod_{i \neq j} \frac{x - x_i}{x_j - x_i}.$$

Note that the Lagrange polynomial $L_j(x)$ is 1 at node x_j and 0 at all of the other nodes. Once we have $\tilde{u}(x)$ we can differentiate it at $x = x_0$ to obtain the desired derivative.

Example 3.4 (Derivation of the Central Difference for $\frac{du}{dx}$). Using $l = r = 1$, let's see if we can use Lagrange interpolation to derive the central difference formula that we presented in Eqn. 3.4. The three Lagrange interpolating functions are

$$\begin{aligned} L_{-1}(x) &= \frac{(x - x_0)(x - x_1)}{(-\Delta x)(-2\Delta x)} & 1 \text{ at node } j = -1 \\ L_0(x) &= \frac{(x - x_{-1})(x - x_1)}{(\Delta x)(-\Delta x)} & 1 \text{ at node } j = 0 \\ L_1(x) &= \frac{(x - x_{-1})(x - x_0)}{(2\Delta x)(\Delta x)} & 1 \text{ at node } j = 1 \end{aligned}$$

The polynomial

$$\tilde{u}(x) = \sum_{j=-1}^1 L_j(x) u_j$$

will then interpolate the three states u_{-1}, u_0, u_1 . To obtain the first derivative at $x = x_0$ we differentiate this polynomial and evaluate at $x = x_0$,

$$\left. \frac{d\tilde{u}(x)}{dx} \right|_{x_0} = \sum_{j=-1}^1 \left. \frac{dL_j(x)}{dx} \right|_{x_0} u_j$$

The derivatives of the Lagrange polynomials are

$$\begin{aligned}\left.\frac{dL_{-1}}{dx}\right|_{x_0} &= \left.\frac{2x - x_1 - x_0}{2\Delta x^2}\right|_{x_0} = \frac{-1}{2\Delta x} \\ \left.\frac{dL_0}{dx}\right|_{x_0} &= \left.\frac{2x - x_1 - x_{-1}}{-\Delta x^2}\right|_{x_0} = 0 \\ \left.\frac{dL_1}{dx}\right|_{x_0} &= \left.\frac{2x - x_0 - x_{-1}}{2\Delta x^2}\right|_{x_0} = \frac{1}{2\Delta x}\end{aligned}$$

So our finite difference approximation is

$$\begin{aligned}\left.\frac{du}{dx}\right|_{x_0} &\approx \left.\frac{d\tilde{u}(x)}{dx}\right|_{x_0} = \sum_{j=-1}^1 \left.\frac{dL_j(x)}{dx}\right|_{x_0} u_j = \frac{-1}{2\Delta x}u_{-1} + 0u_0 + \frac{1}{2\Delta x}u_1 \\ &= \frac{u_1 - u_{-1}}{2\Delta x},\end{aligned}$$

and we see that we have indeed recovered our central difference result. We don't directly get the order of accuracy using Lagrange interpolation, but we can reason as follows: we fit a quadratic function of x to our three points so the slope is going to be a linear function of x , which has an error (missing term) that behaves like $\mathcal{O}(\Delta x^2)$. In general, however, getting the order of accuracy is more straightforward when using Taylor-series expansions.

3.4 Problem Setup and Boundary Conditions

Finite difference formulas can be used to discretize ordinary or partial differential equations into a set of algebraic equations relating the unknown solution values at the nodes. In this section we show how to do this for a one-dimensional scalar problem, paying particular attention to the boundary conditions.

To obtain a solvable system, we should write down as many equations as we have unknowns. The unknowns are the values of u at the nodes of our grid. The equations will generally be the discretized differential equation written down at the grid nodes, so that the number of equations is the same as the number of unknowns.

Points near or on boundaries require some attention. First of all, points close to boundaries might not have enough left or right neighbors in order to apply the desired finite difference formulas. In such cases one-sided difference formulas of matching accuracy can usually be used. Second, for points directly on the boundaries, one has to decide whether to keep these points as unknowns and, if so, what the associated equations should be. We distinguish between two types of boundary conditions (presented in 1D):

$$\begin{aligned}\textit{Dirichlet boundary condition:} & \quad u \text{ specified} \\ \textit{Neumann boundary condition:} & \quad \frac{du}{dx} \text{ specified}\end{aligned}$$

We will focus on Dirichlet boundary conditions, where u is given on the boundary nodes. We could keep the values of u on these nodes as unknowns, and throw the equations “ $u = \text{known value}$ ” into our system. This is simple, but it increases the size of the system. Instead, we will usually treat the given boundary condition values as known quantities, so that the unknowns are only the interior nodes. Then, if any finite difference needs a value from our Dirichlet boundary nodes, we use the given boundary condition value.

Example 3.5 (Steady Heat Conduction in a Bar with a Source). The following differential equation, introduced for multiple dimensions in Section 1.2.3.1, governs the steady-state temperature distribution inside a one-dimensional bar with a heat source term,

$$-\kappa \frac{d^2 T}{dx^2} = q(x), \quad (3.8)$$

where $T(x)$ is the temperature distribution, κ is the thermal conductivity, and $q(x)$ is the heat source. Our domain will be $0 \leq x \leq 1$, and we will use a uniform grid of N intervals so that the spacing is $\Delta x = 1/N$.

Let's use a second-order difference, i.e. Eqn. 3.5, to discretize the second derivative $\frac{d^2 T}{dx^2}$ in our equation. So the discretized form of Eqn. 3.8 becomes, at node x_i ,

$$-\kappa \frac{T_{i+1} - 2T_i + T_{i-1}}{\Delta x^2} = q(x_i).$$

Rearranging the terms,

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{1}{\kappa} \Delta x^2 q(x_i).$$

This equation holds at all interior nodes. Now we need boundary conditions. Suppose that the temperature is fixed at both ends of the bar, so that T_0 and T_N are both known constants. This means that we do not have any unknowns or equations at $i = 0$ and $i = N$. Also, the equations at $i = 1$ and $i = N - 1$ need to be rearranged slightly to put the known values on the right-hand side.

$$\begin{aligned} \text{at } i = 1: \quad 2T_1 - T_2 &= \frac{1}{\kappa} \Delta x^2 q(x_1) + T_0 \\ \text{at } i = N - 1: \quad -T_{N-2} + 2T_{N-1} &= \frac{1}{\kappa} \Delta x^2 q(x_{N-1}) + T_N \end{aligned}$$

So our system of equations, in matrix form, reads

$$\begin{bmatrix} 2 & -1 & 0 & 0 & 0 & \dots & 0 \\ -1 & 2 & -1 & 0 & 0 & \dots & 0 \\ 0 & -1 & 2 & -1 & 0 & \dots & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & -1 & 2 & -1 & 0 \\ 0 & \dots & 0 & 0 & -1 & 2 & -1 \\ 0 & \dots & 0 & 0 & 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ \vdots \\ T_{N-3} \\ T_{N-2} \\ T_{N-1} \end{bmatrix} = \begin{bmatrix} q(x_1)\Delta x^2/\kappa + T_0 \\ q(x_2)\Delta x^2/\kappa \\ q(x_3)\Delta x^2/\kappa \\ \vdots \\ q(x_{N-3})\Delta x^2/\kappa \\ q(x_{N-2})\Delta x^2/\kappa \\ q(x_{N-1})\Delta x^2/\kappa + T_N \end{bmatrix}$$

Listing 3.1 shows a Matlab code that sets up and solves this system and Figure 3.3 shows results for $q(x) = \sin(\pi x)$, $\kappa = 1$, $T_0 = 1$, $T_N = 2$, and $N = 16$.

Listing 3.1: Computes the temperature in a heated 1D bar with Dirichlet boundary conditions.

```

1 function [x,T] = heat1d(T0,TN, kappa, N);
2 % function [x,T] = heat1d(T0,TN, kappa, N);
3 % PURPOSE:
4 % This function determines the temperature in a heated 1D bar
5 % INPUTS:
6 % T0,TN : temperatures at bar endpoints
7 % kappa : heat conductivity inside bar
8 % N      : number of intervals
9 % OUTPUTS:
10 % x : N+1 by 1 vector of node locations, including boundary endpoints
11 % T : N+1 by 1 vector of temperature values at each x
12
13 dx = 1/N; % interval length
14 x = linspace(dx,1-dx, N-1); % node positions
15
16 % set up system: A*T = b
17 e = ones(N-1,1);
18 A = spdiags([-e, 2*e, -e], -1:1, N-1, N-1);
19 b = zeros(N-1,1);
20 for i=1:N-1,
21     b(i) = sin(pi*x(i))*dx^2/kappa; % source term
22 end
23 b( 1) = b( 1) + T0; % additions to rhs due to Dirichlet BC
24 b(N-1) = b(N-1) + TN;
25
26 T = A\b; % solve system
27
28 x = [ 0; x'; 1]; % augment with BCs
29 T = [T0; T ; TN];

```

How good is our solution? The answer depends on the number of points we use. This is a simple problem and we know the exact solution. So let's measure accuracy by an L_2 error norm,

$$\epsilon = \sqrt{\sum_{i=1}^{N-1} \Delta x (T_i - T^{\text{exact}}(x_i))^2},$$

where $T^{\text{exact}}(x)$ is the exact solution, given by

$$T^{\text{exact}}(x) = \frac{1}{\kappa\pi^2} \sin(\pi x) + T_0 + (T_N - T_0)x.$$

Figure 3.4 shows the convergence of the L_2 error norm with mesh refinement. As indicated by the slope, second-order accuracy is achieved, i.e. $\epsilon \sim \mathcal{O}(\Delta x^2)$. This supports our analysis in Example 3.1.

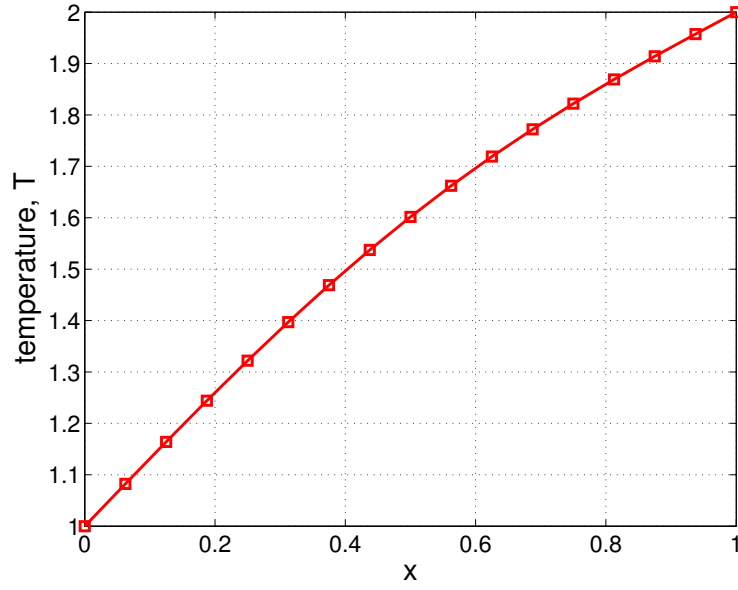


Figure 3.3: Sample finite-difference result for the temperature in a heated 1D bar, with $q(x) = \sin(\pi x)$, $\kappa = 1$, $T_0 = 1$, $T_N = 2$, and $N = 16$.

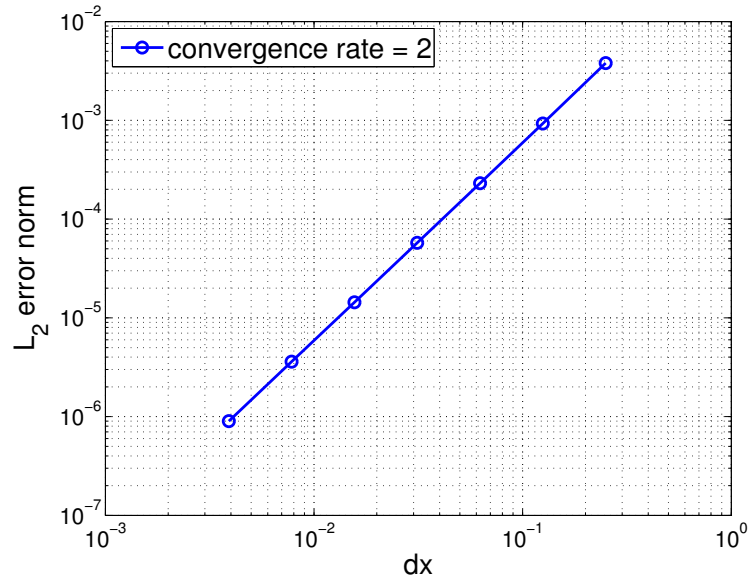


Figure 3.4: Convergence of the discrete L_2 error norm of the computed temperature distribution relative to the actual temperature distribution for Example 3.5.

3.5 Multiple Dimensions

In Section 3.3 we presented a general form for finite difference formulas in one dimension. This form extends to multiple dimensions; in two dimensions, a finite difference formula will typically involve a rectangular grid of points, as illustrated in Figure 3.5. Note that the

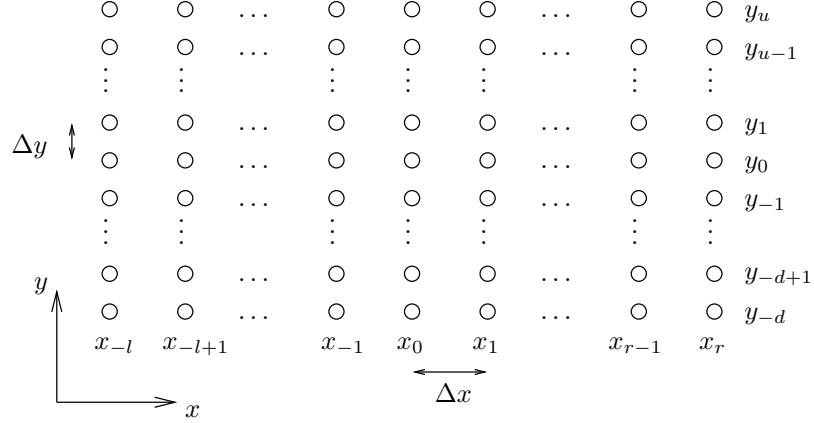


Figure 3.5: Points used in the derivation of a general finite-difference formula in two dimensions. The spacing between all consecutive nodes is Δx in the x direction and Δy in the y direction.

center point is $i = 0, j = 0$ – there are l columns of points to the left, r columns to the right, d rows below, and u rows above. The total number of points is $(l + r + 1)(u + d + 1)$. We then use a weighted sum over these points to approximate a desired derivative quantity, such as the following mixed derivative,

$$\left. \frac{\partial^{m+n} u}{\partial x^m \partial y^n} \right|_{x_0, y_0} \approx \sum_{i=-l}^r \sum_{j=-d}^u a_{ij} u_{ij}. \quad (3.9)$$

We can still use either the method of undetermined coefficients or Lagrange interpolation in order to calculate the coefficients a_{ij} . When using undetermined coefficients, the Taylor-series expansion in multiple dimensions reads

$$\begin{aligned} u_{ij} = & u_{00} + (x_i - x_0) \frac{\partial u}{\partial x} + (y_j - y_0) \frac{\partial u}{\partial y} + \frac{1}{2} (x_i - x_0)^2 \frac{\partial^2 u}{\partial x^2} \\ & + \frac{1}{2} (y_j - y_0)^2 \frac{\partial^2 u}{\partial y^2} + (x_i - x_0)(y_j - y_0) \frac{\partial^2 u}{\partial x \partial y} + \dots \end{aligned}$$

Note that all derivatives are computed at x_0, y_0 . When using Lagrange interpolants, the Lagrange polynomial at each node is given by the product of one-dimensional Lagrange

interpolants in x and y ,

$$\begin{aligned}\tilde{u}(x, y) &= \sum_{j=-l}^r \sum_{k=-d}^u L_j^x(x) L_k^y(y) \\ L_i^x(x) &= \prod_{k \neq i} \frac{x - x_k}{x_i - x_k} \\ L_j^y(y) &= \prod_{k \neq j} \frac{y - y_k}{y_j - y_k} \\ a_{jk} &= \frac{\partial^{m+n} u}{\partial x^m \partial y^n} [L_i^x(x) L_j^y(y)]\end{aligned}$$

Two-dimensional problems require more points and hence are more expensive to set up and solve compared to one-dimensional problems. However, the basic procedure for setting up the linear system remains unchanged: each node contributes one unknown and one equation.

Finite difference methods are great for multi-dimensional problems where the domain is simple – i.e. rectangular. For non-rectangular domains, more general finite difference formulas that do not rely on equal spacing of the points can be used with so-called *curvilinear grids*. Alternatively, the equations on a general domain can sometimes be mapped to a simple domain, such as a unit square. This changes the equations but allows the use of simple finite differences on uniform grids.

Example 3.6 (Two-dimensional Poisson's Equation). Poisson's equation in two dimensions is

$$\underbrace{\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}}_{\nabla^2 u} = f(x, y). \quad (3.10)$$

Consider a rectangular domain of dimensions $l_x \times l_y$ with Dirichlet boundary conditions, $u = 0$, as shown in Figure 3.6. Using a grid of $N \times M$ intervals, with boundary values known, the interval spacings are

$$\Delta x = \frac{l_x}{N}, \quad \Delta y = \frac{l_y}{M}.$$

We can use our second-order discretization, Eqn. 3.5, of each second derivative in Eqn. 3.10,

$$\begin{aligned}\frac{\partial^2 u}{\partial x^2} &\approx \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{\Delta x^2}, \\ \frac{\partial^2 u}{\partial y^2} &\approx \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{\Delta y^2},\end{aligned}$$

Substituting these formulas into Eqn. 3.10 gives the following system,

$$\begin{aligned}\frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{\Delta x^2} + \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{\Delta y^2} &= f_{i,j} \\ u_{0,j} = u_{N,j} &= 0 \\ u_{i,0} = u_{i,M} &= 0\end{aligned}$$

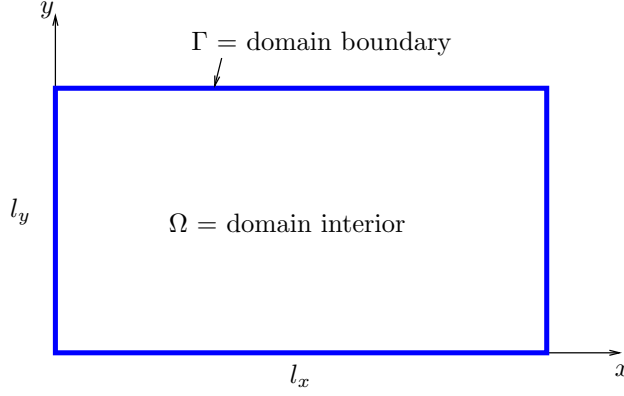


Figure 3.6: Rectangular domain for a two-dimensional Poisson problem.

Note, $f_{i,j}$ is $f(x, y)$ evaluated at node (i, j) in our lattice. We can write this system in matrix form as

$$\mathbf{A}\mathbf{U} = \mathbf{F},$$

where $\mathbf{U}$ is a column vector of all $(N-1)(M-1)$ unknowns, ordered in theory in an arbitrary fashion. Typically, some logical ordering is used, such as stacking unknowns from our x, y lattice row by row,

$$\mathbf{U} = [u_{1,1}, u_{2,1}, \dots, u_{N-1,1}, u_{1,2}, u_{2,2}, \dots, u_{N-1,2}, \dots, u_{1,M-1}, u_{2,M-1}, \dots, u_{N-1,M-1}]$$

Note, the number of unknowns is $N_u = (N-1)(M-1)$ = the number of grid nodes on the interior of the domain. Since u is prescribed on the boundary through the boundary condition, the nodes on the boundary are not treated as unknowns and do not contribute any equations. $\mathbf{F}$ is a column vector of the right-hand side $f(x, y)$ source term evaluated at the $N_u = (N-1)(M-1)$ nodes. Finally, $\mathbf{A}$ is an $N_u \times N_u$ matrix where each row contains coefficients into the $\mathbf{U}$ vector for a single equation.

For example, for $N = M = 3$ and $\Delta x = \Delta y = h$, the equations (see definitions of quantities in Figure 3.7) are

$$\begin{aligned} \text{at } i = 1, j = 1: \quad & \frac{1}{h^2} (-4u_{11} + u_{21} + u_{12}) = f_{11} \\ \text{at } i = 2, j = 1: \quad & \frac{1}{h^2} (u_{11} - 4u_{21} + u_{22}) = f_{21} \\ \text{at } i = 1, j = 2: \quad & \frac{1}{h^2} (u_{11} - 4u_{12} + u_{22}) = f_{12} \\ \text{at } i = 2, j = 2: \quad & \frac{1}{h^2} (u_{21} + u_{12} - 4u_{22}) = f_{22} \end{aligned}$$

The corresponding matrix system is

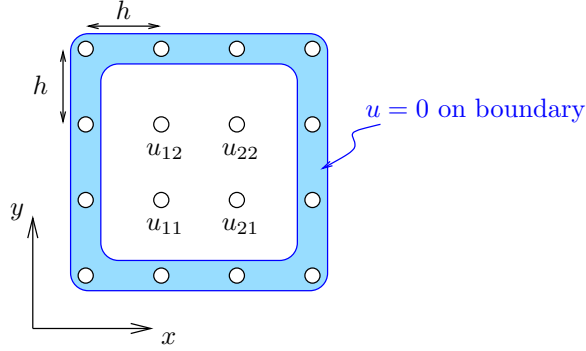


Figure 3.7: Nodes for a $N = M = 3$ grid of points for the two-dimensional Poisson problem.

$$\underbrace{\frac{1}{h^2} \begin{bmatrix} -4 & 1 & 1 & 0 \\ 1 & -4 & 0 & 1 \\ 1 & 0 & -4 & 1 \\ 0 & 1 & 1 & -4 \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} u_{11} \\ u_{21} \\ u_{12} \\ u_{22} \end{bmatrix}}_{\mathbf{U}} = \underbrace{\begin{bmatrix} f_{11} \\ f_{21} \\ f_{12} \\ f_{22} \end{bmatrix}}_{\mathbf{F}}$$

For greater N and M , the matrix will be correspondingly larger. Fortunately, however, the matrix will be *sparse* (mostly zeros) because in each equation, each unknown only “talks to” its four immediate neighbors (above, below, left, right).

3.6 Stability Analysis

We can derive various finite difference approximations for derivatives that appear in ordinary or partial differential equations. So which finite difference should we use for a particular problem? Accuracy is one consideration – at the very least, we need consistency which means that our approximations should be at least first-order accurate. Beyond consistency, high-order accuracy is usually advantageous, although it should be traded off against increased computational time. But accuracy is not the only consideration: we should also pay attention to stability. That is, a very accurate finite difference may be unstable and hence useless. The PDE analog of the ODE Dahlquist equivalence theorem presented in Section 2.4.3, is the *Lax equivalence theorem*. This theorem also states that

$$\text{stability} + \text{consistency} \rightarrow \text{convergence}$$

Some definitions are as follows:

- *stability*: the solution remains bounded as Δx and Δt approach zero on a fixed space-time domain.

- *consistency*: the numerical scheme is derived logically from the PDE; i.e. the error in a Taylor-series sense between the numerical difference equation and the PDE approaches zero as Δx and Δt approach zero.
- *convergence*: “if we do enough work, we’ll get the right answer” – i.e. the numerical solution approaches the exact solution of the PDE as Δx and Δt approach zero.

We note that the Lax equivalence theorem applies only to linear problems on a fixed space-time domain. For nonzero Δx or Δt , we might see instability even for a convergent scheme. In this section we look at two ways to analyze stability given a particular discretization.

3.6.1 Von-Neumann Analysis

Von-Neumann analysis applies to linear initial-value problems on uniform grids with periodic boundary conditions. These assumptions seem limiting, yet the applicability of von-Neumann analysis is quite broad: a scheme that fails this test has fundamental flaws that need to be addressed.

The key observation in von-Neumann analysis is that difference equations possess exact solutions of sinusoidal variation,

$$u_j^n = \text{Re} [g^n e^{ij\phi}],$$

where

- g is a modal amplification factor (a complex number),
- $i = \sqrt{-1}$,
- j is a spatial node index,
- ϕ is a phase angle which is the amount by which the phase of our solution “mode” changes per mesh interval.

On a finite mesh with N intervals, ϕ takes on discrete values, namely

$$\phi = \left\{ \frac{\pi}{N}, \frac{2\pi}{N}, \dots, \pi \right\}.$$

Low values of ϕ correspond to low-frequency modes, while high values of ϕ yield high-frequency modes, as illustrated in Figure 3.8. Therefore, if the solution at one time instant is a pure sine wave with phase change of ϕ per Δx , then the solution at the next time level will be that sine wave multiplied by g . Since g may be complex, both the phase and amplitude of the sine wave can be affected. For stability, we care about the amplitude, so our requirement is that $|g| \leq 1$ for all $\phi \in [0, \pi]$ – note that g will be a function of the frequency of the mode, i.e. ϕ . This is another way of saying that we do not want any of the modes representable on the grid to “blow up.”

The steps for performing von-Neumann stability analysis read as follows,

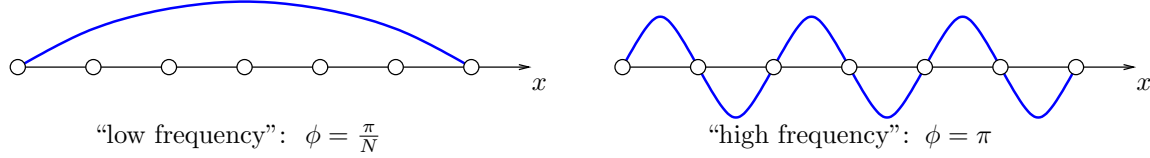


Figure 3.8: In von-Neumann analysis, $\phi \in [0, \pi]$ describes the frequency of a sinusoidal error mode. Specifically, ϕ is the phase change per mesh interval.

1. Choose a finite-difference scheme.
2. Substitute $u_j^n = g^n e^{ij\phi}$ into the scheme.
3. Simplify and solve for $g(\phi)$.
4. If possible, determine requirements on scheme parameters such that $|g(\phi)| < 1$ for all ϕ in $[0, \pi]$.

Example 3.7 (FTCS Discretization of the Advection Equation). In this example we look at the forward-in-time centered-in-space (FTCS) discretization of a linear convection (advection) PDE,

$$\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0. \quad (3.11)$$

“Forward-in-time” means that we discretize the time derivative using a forward difference. “Centered-in-space” means we discretize $\frac{\partial u}{\partial x}$ using a central difference, so that our discretized equation is

$$\begin{aligned} \frac{u_j^{n+1} - u_j^n}{\Delta t} + a \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} &= 0 \\ \Rightarrow u_j^{n+1} &= u_j^n - \frac{a\Delta t}{2\Delta x} (u_{j+1}^n - u_{j-1}^n) \end{aligned}$$

Note that the subscript j denotes the spatial index in our finite difference grid, and the superscript n denotes the temporal index, i.e. the time node. To apply von-Neumann stability analysis, we substitute $u_j^n = g^n e^{ij\phi}$ into this equation and calculate the amplification factor g ,

$$\begin{aligned} g^{n+1} e^{ij\phi} &= g^n e^{ij\phi} - \frac{a\Delta t}{2\Delta x} (g^n e^{i(j+1)\phi} - g^n e^{i(j-1)\phi}) \\ g &= 1 - \frac{a\Delta t}{2\Delta x} (e^{i\phi} - e^{-i\phi}) \\ &= 1 - \frac{a\Delta t}{\Delta x} i \sin \phi \end{aligned}$$

The magnitude of the amplification factor is

$$|g|^2 = 1 + \left(\frac{a\Delta t}{\Delta x} \right)^2 \sin^2 \phi \geq 1$$

So the FTCS scheme for advection is *unconditionally unstable* because the magnitude of g is always greater or equal to 1 (equality only in the special case when $\phi = \pi$).

Example 3.8 (FOU Discretization of the Advection Equation). In this example we look at the first-order upwind (FOU) discretization of the same PDE as in the previous example,

$$\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0.$$

The only difference is the spatial discretization, which is now “upwind”, which means

$$\begin{aligned} \frac{\partial u}{\partial x} &\approx \frac{u_j^n - u_{j-1}^n}{\Delta x} & \text{for } a > 0 \\ \frac{\partial u}{\partial x} &\approx \frac{u_{j+1}^n - u_j^n}{\Delta x} & \text{for } a < 0 \end{aligned}$$

Let’s just look at the case $a > 0$, since the other result is similar. Our discretization is

$$\begin{aligned} \frac{u_j^{n+1} - u_j^n}{\Delta t} + a \frac{u_j^n - u_{j-1}^n}{\Delta x} &= 0 \\ \Rightarrow u_j^{n+1} &= u_j^n - \frac{a\Delta t}{\Delta x} (u_j^n - u_{j-1}^n) \end{aligned}$$

To apply von-Neumann stability analysis, we substitute $u_j^n = g^n e^{ij\phi}$ into this equation and calculate the amplification factor g ,

$$\begin{aligned} g^{n+1} e^{ij\phi} &= g^n e^{ij\phi} - \frac{a\Delta t}{\Delta x} (g^n e^{ij\phi} - g^n e^{i(j-1)\phi}) \\ g &= 1 - \frac{a\Delta t}{\Delta x} (1 - e^{-i\phi}) \end{aligned}$$

Let’s define $\sigma = a\Delta t/\Delta x$; note that σ is nondimensional and positive. The amplification factor is then

$$g = 1 - \sigma + \sigma \cos \phi - i\sigma \sin \phi$$

The square magnitude of g is

$$\begin{aligned} |g|^2 &= (1 - \sigma + \sigma \cos \phi)^2 + (\sigma \sin \phi)^2 \\ &= (1 - \sigma)^2 + 2(1 - \sigma)\sigma \cos \phi + \sigma^2 \end{aligned}$$

Now we ask whether $|g|$ is less than or greater than 1. The answer depends on σ and ϕ . We don't have control over ϕ : this is the frequency of the mode we're analyzing, and in practice error modes of all frequencies appear, e.g. due to finite precision calculations. So we have to consider all possible $\phi \in [0, \pi]$. Since $-1 \leq \cos(\phi) \leq 1$, we have

$$\begin{aligned} |g|^2 &= (1 - \sigma)^2 + 2(1 - \sigma)\sigma \cos \phi + \sigma^2 \\ &\leq \max \left[(1 - \sigma)^2 + 2(1 - \sigma)\sigma(-1) + \sigma^2, (1 - \sigma)^2 + 2(1 - \sigma)\sigma(1) + \sigma^2 \right] \\ &= \max \left[(1 - \sigma - \sigma)^2, (1 - \sigma + \sigma)^2 \right] \\ &= \max \left[(1 - 2\sigma)^2, 1 \right] \end{aligned}$$

So we see that $|g| \leq 1$ as long as $(1 - 2\sigma)^2 \leq 1$. Since σ is positive, this means that we need $\sigma \leq 1$. The conclusion is that our first-order upwind discretization is stable if $\sigma = |a|\Delta t/\Delta x < 1$ – note that we have put absolute values around a in order to include the similar result for $a < 0$. σ is called the CFL (Courant-Friedrichs-Lewy) number, and this restriction is a well-known result for convection problems. Note that the requirement on σ can be used to set the time step according to a and Δx .

3.6.2 Matrix Stability

Von-Neumann stability analysis gives us a theoretical result under idealized conditions such as a uniform grid and periodic boundary conditions. But sometimes we would like to know whether a particular method will be stable on an arbitrary grid with other boundary conditions. For such practical situations, a matrix stability analysis can give us this information.

Matrix stability analysis is based on the idea of a *semi-discrete* form, which is the PDE written with only the spatial term discretized – the time derivative is left alone. The general form is given by

$$\frac{d\mathbf{U}}{dt} = \mathbf{A}\mathbf{U} + \mathbf{Q}, \quad (3.12)$$

where

- $\mathbf{U}$ is the vector of discrete unknowns, such as node values in a finite difference discretization.
- $\mathbf{A}$ is the matrix that arises from the spatial discretization; it represents how the unknowns talk to each other through the spatial derivatives.
- $\mathbf{Q}$ is a source term independent of $\mathbf{U}$.

Note that this analysis applies to linear equations. Nonlinear equations would have to be linearized prior to analyzing stability.

But we have already seen Eqn. 3.12 ... this was back in our eigenvalue stability analysis for ODE systems. For example, Eqn. 2.14 was just our current equation without the source term, which happens to be irrelevant for stability analysis. So at this point, we rely on our results from ODEs, where we found that the stability of a time integration method will generally depend on $\lambda_i \Delta t$, where λ_i are the eigenvalues of $\mathbf{A}$.

The steps for matrix stability analysis are then as follows:

- Apply the spatial discretization in order to obtain a semi-discrete system of the form in Eqn. 3.12.
- Calculate the eigenvalues of the system matrix $\mathbf{A}$; call them λ_i .
- Use the results of eigenvalue stability analysis for ODEs to determine whether a particular time integration method will be stable.

Example 3.9 (Matrix Stability of FOU for Advection). Let's look at the matrix stability of the first-order upwind method presented in Example 3.8. We'll look at one-dimensional problems with periodic and Dirichlet boundary conditions. In the case of Dirichlet boundary conditions, we'll only have to set the state on the inflow boundary. For $a > 0$ this will be the left boundary. The outflow boundary state is never used since the derivative is approximated by an upwind difference. This makes sense because physically the outflow state should not be relevant for a pure advection problem. Assuming $a > 0$, our semi-discrete PDE obtained from Eqn. 3.11 reads

$$\frac{\partial u_j}{\partial t} = -a \frac{u_j - u_{j-1}}{\Delta x}$$

The boundary condition comes into our equation for the first unknown, $j = 1$, since then $u_{j-1} = u_0$, which is the prescribed Dirichlet boundary condition. So in matrix form our equation with Dirichlet boundary conditions reads

$$\frac{d}{dt} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{N-2} \\ u_{N-1} \\ u_N \end{bmatrix} = -\frac{a}{\Delta x} \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & \dots & \boxed{0} \\ -1 & 1 & 0 & 0 & 0 & \dots & 0 \\ 0 & -1 & 1 & 0 & 0 & \dots & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & -1 & 1 & 0 & 0 \\ 0 & \dots & 0 & 0 & -1 & 1 & 0 \\ 0 & \dots & 0 & 0 & 0 & -1 & 1 \end{bmatrix}}_{\mathbf{A}} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{N-2} \\ u_{N-1} \\ u_N \end{bmatrix} + \begin{bmatrix} \boxed{au_0/\Delta x} \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

The two boxed terms above change when periodic boundary conditions are used. Specifically, in $\mathbf{A}$, the 0 becomes -1, and in the source term, $au_0/\Delta x$ becomes zero.

We use Matlab to compute the eigenvalues of the system matrix $\mathbf{A}$ using both Dirichlet and periodic boundary conditions for various N and $a = 1, \Delta x = 1/N$ (i.e. a domain of length $L = 1$). Figure 3.9 shows the results. In both cases, we note that the results are

consistent with the von-Neumann stability analysis. For example, when a forward Euler time discretization is used, the eigenvalue stability region is as shown in Figure 2.7. To get $\lambda\Delta t$ into this stability region, we would need $\Delta t < 1/N$. But this result is specific to our choice of a and L , and a dimensional analysis shows that the general result is $\Delta t < \Delta x/a$, i.e. $\sigma = a\Delta t/\Delta x < 1$, as derived in Example 3.8.

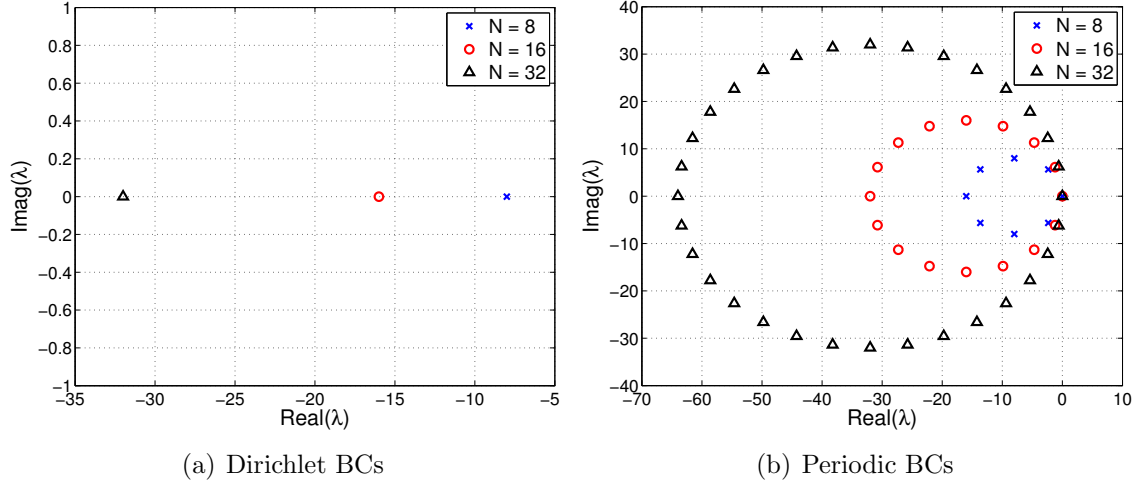


Figure 3.9: Eigenvalues of the system matrix for a first-order upwind (FOU) discretization of the one-dimensional advection equation, using both Dirichlet and periodic boundary conditions.

Chapter 4

Probabilistic Methods

4.1 Probability Review

In this section we provide a review of basic probability and statistics.

4.1.1 Outcomes and Events

When performing an experiment, an *outcome*, ζ , is a result of the experiment that is recorded. An *event*, A , is some set of outcomes to which we can assign a probability. An *elementary event* is one that consists of a single outcome.

Example 4.1 (Manufacturing of a Solar Cell Array). Consider the manufacturing process of a solar cell array. The “experiment” is the manufacturing of a single cell. A simple outcome could be the binary result of a quality control test: an outcome value of $\xi = 1$ indicates a pass of the quality-control test, while an outcome value of $\xi = 0$ indicates a fail of the test. The event that a produced cell passes the test is an example of an elementary event.

Suppose now that we have another experiment, in which we manufacture 100 solar cells and count $N_{\text{defective}}$ = the number of defective cells – i.e. the ones that fail the quality-control test. An event in this case could be the situation where $N_{\text{defective}} < 5$. This is not an elementary event, since the event consists of five possible outcomes, $N_{\text{defective}} \in \{0, 1, 2, 3, 4\}$.

4.1.2 The Meaning of Probability

We denote an event by A , and the probability of that even happening by $P(A)$. This *probability* must satisfy the following conditions:

- $P(A) \geq 0$.
- $P(A) = 1 \Leftrightarrow$ the event A is certain.
- Given two mutually exclusive events, A and B , $P(A \cup B) = P(A) + P(B)$.

4.1.3 Random Variables

A *random variable*, which we will denote by an upper-case letter, e.g. X , is a value that is a function of the outcome of the experiment. We sometimes indicate the dependence on the outcome explicitly by writing $X(\zeta)$. This could be just the value of the outcome itself, or it could be a more complicated function. A random variable could be discrete, meaning that it takes on a finite or countably infinite number of values, or continuous, meaning that it can take on a value that varies continuously in one or more intervals of real numbers.

Example 4.2 (Cost of Manufacturing of a Solar Cell Array). Consider again the manufacturing process of a solar cell array, specifically the case where we produce 100 cells and count the number of defective cells: $\zeta = N_{\text{defective}}$. Suppose now that we are interested in the cost associated with defective cells. If the cost of each defective cell is $C = 1$, then we can define a random variable that measures this cost for 100 cells as

$$X = C\zeta = \zeta.$$

A more complicated example would be the case where a too-high failure rate triggers an inspection of the manufacturing process. For example, suppose that if $N_{\text{defective}} > 10$, we incur an inspection cost of C_{inspect} . Then the random variable that measures the “cost of failure” for 100 cells is

$$X = \begin{cases} C\zeta & \text{for } \zeta \leq 10 \\ C_{\text{inspect}} + C\zeta & \text{for } \zeta > 10 \end{cases}$$

4.1.4 The Probability Density Function (PDF)

For a continuous random variable, X , we use a probability density function (PDF), $p(x)$, to describe the probability of the random variable value, x , being in some range. For example, given a random variable X , the probability that $a \leq X \leq b$ is given by the integral of the PDF over the range,

$$P(a \leq x \leq b) = \int_a^b p(x)dx,$$

One of the simplest probability density functions is a constant, $p(x) = \text{const.}$, on the interval and zero outside it. In particular,

$$p(x) = \begin{cases} \frac{1}{b-a} & \text{for } a \leq x \leq b, \\ 0, & \text{otherwise.} \end{cases} \quad (4.1)$$

Note that the integral of $p(x)$ over all possible ranges of x is 1, indicating that when considering all possible events, the probabilities must “add up” to unity.

4.1.5 Expected Value and Mean

Given the PDF, $p(x)$, of a continuous random variable X , the expected value is defined as

$$E(X) \equiv \int_{-\infty}^{\infty} xp(x) dx. \quad (4.2)$$

For a discrete random variable, the above integral is replaced with a sum over the discrete values that x could take on, and the PDF just becomes a value for each x . The expected value of X , which we will denote by μ_x , is also known as the mean value.

4.1.6 Variance and Standard Deviation

The variance of X is defined as,

$$\sigma_x^2 \equiv \int_{-\infty}^{+\infty} (x - \mu_x)^2 p(x) dx.$$

The value σ_x is known as the standard deviation of X . The variance is a measure of the variability in X about its mean value. A frequently-used relationship between the mean and variance is,

$$\sigma_x^2 = E(X^2) - \mu_x^2.$$

4.1.7 The Cumulative Distribution Function (CDF)

The cumulative distribution function (CDF) of a continuous random variable X is defined as the probability that $X \leq x$. Specifically,

$$\text{CDF}(x) \equiv P(X \leq x).$$

From the basic assumptions of probability, we can show that $\text{CDF}(-\infty) = 0$ and $\text{CDF}(+\infty) = 1$. The CDF and PDF of X are related as follows,

$$\begin{aligned} \text{CDF}(a) &= \int_{-\infty}^a p(x) dx, \\ \text{CDF}(b) - \text{CDF}(a) &= \int_a^b p(x) dx. \\ p(x) &= \frac{d}{dx} (\text{CDF}(x)). \end{aligned}$$

4.1.8 Percentiles

The u percentile of X is the smallest number x_u such that

$$u = P(X \leq x_u) = \text{CDF}(x_u).$$

Note, since u is a probability, its range is $0 \leq u \leq 1$.

4.2 Monte Carlo Sampling

The Monte Carlo method consists of using repeated simulations of an experiment in order to obtain statistics of quantities of interest. This simple method has many uses, and here we focus on one, which is the propagation of uncertainties through a system.

4.2.1 Introduction

Consider the general setup of a numerical simulation shown in Figure 4.1.

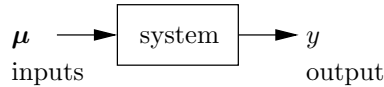


Figure 4.1: Schematic illustration of an input-to-output mapping for a system, such as a numerical simulation. We consider multiple inputs, in the vector μ and a single scalar output, y .

Here, $\mu \in \mathbb{R}^p$ is a vector of p inputs, and y is a scalar output. Multiple outputs can be easily considered, and we choose one output only for simplicity of exposition. In the Monte Carlo method, we will assume that we know the probability distributions of the inputs, i.e. the entries of μ , and our goal is to determine the probability distribution of the output y . Throughout this section we assume that the inputs and outputs are continuous random variables.

The essence of the Monte Carlo method is to obtain an approximate probability distribution of y through repeated simulations with different, independent, choices of inputs μ . These input *samples*, μ_i , are chosen randomly, according to the probability distributions governing the inputs. Note that here i indexes the different input samples: for every i , we have a separate input vector μ_i . So if we run N simulations, $i \in [1 \dots N]$.

The result of running N simulations is N outputs, y_i . We can think of this set of outputs as an approximation to the probability density function of y , $p(y)$. To see this, let's take the range of outputs, $[y_{\min}, y_{\max}]$, and divide it into M equally-spaced intervals, called *bins*. For each bin, we then count the number of output samples that lie inside the bin. Dividing these numbers (one for each bin) by N and plotting versus the bins in a bar-graph form gives a *histogram* of the output distribution. Since the samples were drawn independently at random, the histogram is an approximation to the probability density function $p(y)$. Sample histograms are shown in Figures 4.7 and 4.8. Note that the number of bins, M , affects the “smoothness” of the histogram: we want to choose an M that is sufficiently high to resolve $p(x)$ well, but low enough relative to N so that on average there are still a large number of samples per bin.

In summary, we are using the Monte Carlo method to obtain an approximation to the probability density function, $p(y)$, of an output of interest y . This approximation is in the form of samples y_i , $i \in [1 \dots N]$, which we can visualize in a histogram and from which we can compute statistics (as we will soon see). The method is approximate because we will use a finite number of samples, N . That is, to obtain exact statistics, we would need $N \rightarrow \infty$. We will see in Section 4.3 how to estimate the errors incurred by using a finite N .

Example 4.3 (Variability in Aircraft Range). Consider the Breguet range equation for a propeller aircraft,

$$R = \frac{\eta}{c} \frac{C_L}{C_D} \ln \frac{W_0}{W_1} \quad (4.3)$$

where R is the range and the parameters are

- η = propeller efficiency
- c = specific fuel consumption
- C_L/C_D = lift-to-drag ratio, assumed constant throughout cruise
- W_0 = initial weight of aircraft and fuel
- $W_1 = W_0 - W_f$ final weight of aircraft (no fuel); W_f is the fuel weight

Furthermore, we model the drag coefficient as

$$C_D = C_{D0} + \frac{C_L^2}{\pi e AR},$$

where C_{D0} is the zero-lift drag coefficient, e is the span efficiency factor, and AR is the aspect ratio. During initial stages of aircraft design, not all of these parameters may be exactly known. But if we know the probability distributions of the input parameters (we don't always, but sometimes we can make reasonable guesses), we can say something about the expected variability in the output, which in this case is the range.

Table 4.1 provides a list of all parameters with baseline values, and it notes ones that we will treat as uncertain in the following sections.

Table 4.1: Listing of parameters for the Breguet range equation example.

Parameter	Baseline value	Type
C_L	0.5	Fixed
AR	8	Fixed
W_f	170kg	Fixed
η	0.8	Uncertain
c	$7.5 \times 10^{-7} m^{-1}$	Uncertain
C_{D0}	.025	Uncertain
e	0.9	Uncertain
W_0	1300kg	Uncertain

4.2.2 Sampling a Distribution

In the Monte Carlo method we need to pick samples for input random variables that are governed by certain probability distributions. Usually, we have at our disposal a (pseudo-) random number generator, often one which returns uniform random numbers in a predefined range, such as $[0, 1]$. Let's see how we can use such a function to sample general distributions.

A very simple distribution that we can sample directly is that of a uniformly-distributed random variable. The PDF of such a variable is given in Eqn. 4.1 and Figure 4.2 illustrates it schematically.

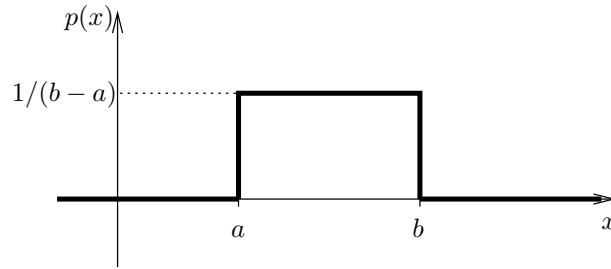


Figure 4.2: Probability density function for a uniform random variable X that takes on a constant value for x between a and b .

Given a uniform random variable U between 0 and 1 (from our random number generator), we can obtain a sample of our random variable X by the following equation,

$$X = a + U(b - a).$$

Example 4.4 (Sampling a uniform distribution). Continuing with Example 4.3, we consider the case where all uncertain parameters are at their baseline values, but the span efficiency factor e is distributed uniformly on the range $e \in [0.85, 0.95]$. We are interested in the effect of this variation on the range, R .

Listing 4.1 presents a code that implements the Monte Carlo method by sampling e and computing the range for each sample.

Listing 4.1: Propagates uniform variation in span efficiency to the range.

```
1 function [ve, vR] = MCuniform(N);
2 % function [ve, vR] = MCuniform;
3 % PURPOSE:
4 %   Propagates a uniformly-varying input parameter to the output
5 % INPUTS:
6 %   N      :Number of samples in the Monte Carlo method
7 % OUTPUTS
8 %   ve     : vector of input samples
9 %   vR     : vector of output samples
10 %   vR     : Ne+1 by 1 vector of temperatures at each node
11
12
13 % constant parameters
14 CL = 0.5;
15 AR = 8;
16 Wf = 170;
17 eta = 0.8;
18 c   = 7.5e-7; % [m^{-1}]
19 CD0 = 0.025;
20 ebase = 0.9;
21 W0 = 1300;
22
23 % input samples
24 ve = 0.85 + 0.1*rand(N,1);
25
26 % drag coefficient
27 CD = CD0 + CL*CL./(pi*ve*AR);
28
29 % range
30 vR = eta/c*CL./CD*log(W0/(W0-Wf));
```

Figure 4.3 illustrates both the input and the output histograms for $N = 10^4$ samples. Note the near-uniform distribution of the input samples, and a slight variation in the range histogram.

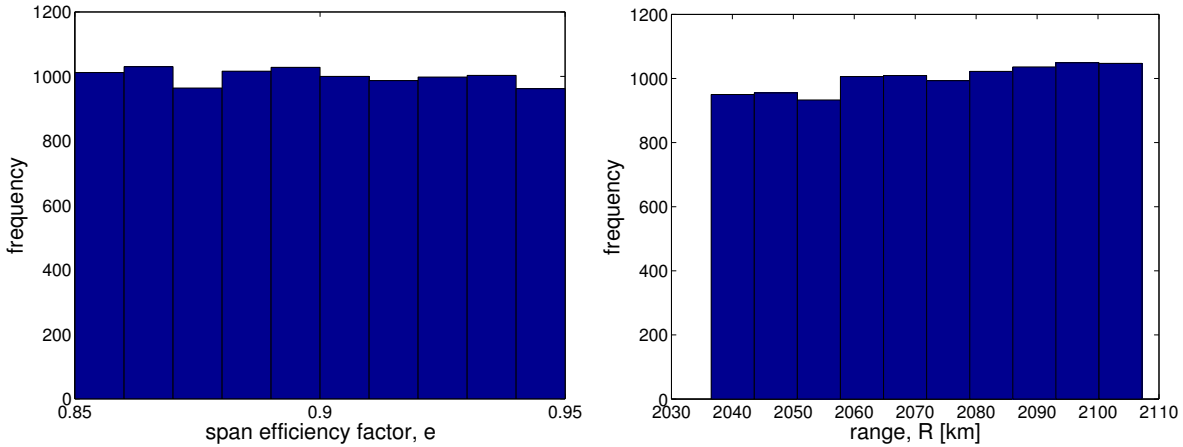


Figure 4.3: Input and output samples for $N = 10^4$ samples of the Monte Carlo method in Example 4.4.

Non-uniform random variables are somewhat harder to sample. For example, consider a random variable X governed by a linear probability density function, as illustrated in Figure 4.4. When sampling X we expect to obtain x values that are biased towards 1, since

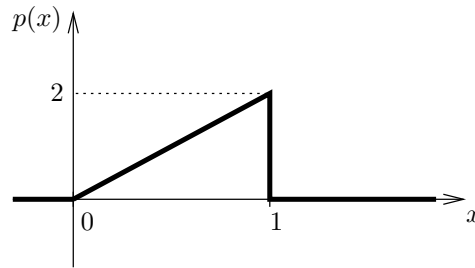


Figure 4.4: Probability density function for a linear random variable X defined over $0 \leq x \leq 1$.

those values are more probable. So we cannot just use a uniform random number generator between 0 and 1 to sample X . For more complicated probability distributions, this task becomes even more difficult. Let's now look at a general approach for sampling arbitrary probability distributions.

We consider the *inversion approach* for sampling a general probability distribution via a simple (uniform) random number generator. The goal is to map realizations of a uniform random variable U , which takes on values $0 \leq u \leq 1$, into samples of a random variable X governed by the general probability density function $p(x)$. To do this, we use as our mapping the inverse of the cumulative distribution function, $\text{CDF}(x)$. Recall that $\text{CDF}(x)$ maps values of x to the real numbers between 0 and 1. The inverse of this mapping then takes values between 0 and 1 and converts them to values of x . The resulting samples conform to the PDF of X , since the probability of drawing a sample between x and $x + dx$ is $p(x)dx$, as illustrated in Figure 4.5.

In summary, the inversion method for sampling X consists of two steps,

1. Pick a random number, $0 \leq u \leq 1$, from a random number generator.
2. Convert u to x via the inverse CDF function, $x = \text{CDF}^{-1}(u)$.

Example 4.5 (Sampling a linear distribution). Let's apply the inverse CDF method to sample a random variable X governed by the linear probability density function given in Figure 4.4. The PDF, $p(x)$, is nonzero only on the interval $0 \leq x \leq 1$, and there $p(x) = 2x$. Since the cumulative distribution function is the integral of $p(x)$, we have

$$\text{CDF}(x) = \int_0^x p(x') dx' = \begin{cases} 0 & \text{for } x < 0 \\ x^2 & \text{for } 0 \leq x \leq 1 \\ 1 & \text{for } x > 1 \end{cases}$$

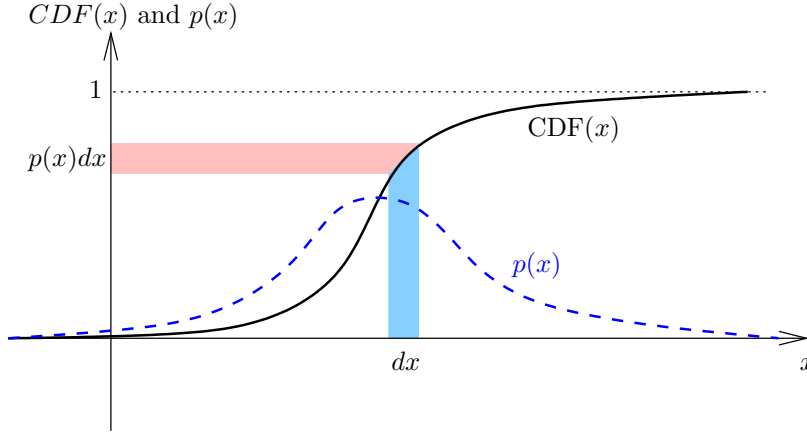


Figure 4.5: Illustration of the inverse CDF approach for sampling a random variable X governed by a general probability distribution function $p(x)$. Note that $p(x) = d(CDF)/dx$.

The inverse CDF function, $x = CDF^{-1}(u)$, maps each u , $0 \leq u \leq 1$, to x . Inverting the above equation and using $0 \leq x \leq 1$ (since that's the only interval over which x is defined with nonzero probability), we have

$$x = CDF^{-1}(u) = \sqrt{u}.$$

Using this inverse CDF, we can now sample X with a simple random number generator for u . Listing 4.2 presents a code that does this, and Figure 4.6 shows histograms of both u (should be uniform) and x (should be linear) for $N = 10^4$ samples.

Listing 4.2: Code for sampling a linear PDF via the inverse CDF method.

```

1 function [u,x] = MClinear(N);
2 % function [ve, vR] = MClinear
3 % PURPOSE:
4 % Samples a linear PDF by the inverse CDF method
5 % INPUTS:
6 % N : number of samples
7 % OUTPUTS
8 % u : samples of a uniform random variable
9 % x : samples of the linear random variable of interest
10
11 % uniform samples
12 u = rand(N,1);
13
14 % inverse CDF
15 x = sqrt(u);

```

4.2.3 Multiple Inputs

When using the Monte Carlo method to sample a system with multiple inputs, stored in the vector $\boldsymbol{\mu} \in \mathbb{R}^p$, we must create many independent sample vectors, $\boldsymbol{\mu}_i$, $i \in [1 \dots N]$. For each

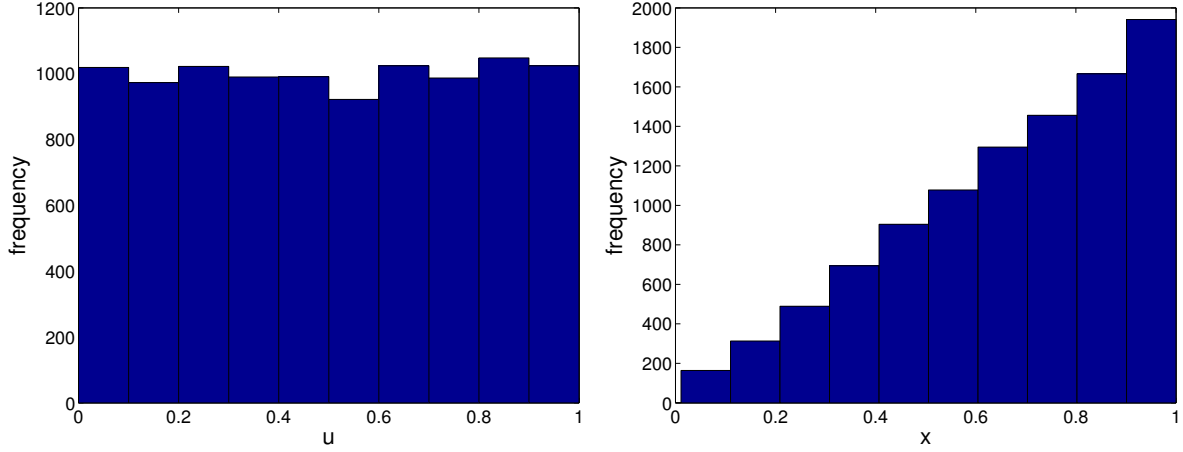


Figure 4.6: Samples of u and x for a test run of the inverse CDF method in Example 4.5.

vector, we will typically need to generate several random variables – e.g. p if all inputs are independent. If the inputs are not independent, we can use joint probability distribution functions to sample them, but we will not consider this here.

Example 4.6 (Variability in Aircraft Range with Multiple Uncertainty Sources). Consider again the Breguet range equation presented in Example 4.3. We now apply the Monte Carlo method to predict the variation in range under three sources of uncertainty: The other input

Table 4.2: Parameters and distributions for the multiple-uncertainty study in Example 4.6.

Parameter	Probability distribution
C_{D0}	Linear, with $p(.021) = 0$ up to $p(.027) = 1$
e	Uniform between 0.85 and 0.95
W_0	Normal with mean 1300kg and standard deviation of 50kg.

parameters are taken at baseline values. The new distribution here is the “normal” one, for which the PDF is given by

$$\text{normal distribution: } p(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)}.$$

In this example we take advantage of Matlab’s built-in function `randn` for sampling such a random variable.

Listing 4.3 presents a code that implements the Monte-Carlo method for these three sources of uncertainty, and Figure 4.7 shows histograms of the input samples and the final output, the range.

Listing 4.3: Code for sampling multiple uncertainty sources in the Breguet range equation.

```
1 function [CD0, e, W0, R] = BreguetMC(N)
2 % function [CD0, e, W0, R] = BreguetMC(N)
3 % PURPOSE:
4 %   Propagates multiple input uncertainties to the range output
5 % INPUTS:
6 %   N      : Number of samples in the Monte Carlo method
7 % OUTPUTS
8 %   CD0    : vector of CD0 input samples
9 %   e      : vector of e input samples
10 %   W0     : vector of W0 input samples
11 %   R      : vector of output samples
12
13 % fixed parameters
14 CL = 0.5;
15 AR = 8;
16 Wf = 170;
17 eta = 0.8;
18 c    = 7.5e-7; % [m^{-1}]
19
20 % Now we generate N independent realizations for each uncertain input
21
22 % parasitic drag coefficient
23 u = rand(N,1); % u is uniform between 0 and 1
24 x = sqrt(u);   % x is linear between 0 and 1 (peak at 1)
25 CD0 = .021 + x*(.027-.021); % linear between desired values
26
27 % span efficiency factor
28 e = 0.85 + 0.1*rand(N,1);
29
30 % Weight
31 W0 = 1300 + 50*randn(N,1);
32
33 % drag coefficient
34 CD = CD0 + CL*CL./(pi*e*AR);
35
36 % range
37 R = eta/c*CL./CD.*log(W0./(W0-Wf));
```

4.3 Error Estimates for Monte Carlo

As we already mentioned, Monte Carlo sampling is a means of approximating statistics using a finite sample size, N . As we increase N , we expect our answers to improve, but practical considerations, such as an expensive numerical calculation for each simulation, generally limit the number of samples we can take. In this section we look at how we can estimate the accuracy of statistical quantities computed using the Monte Carlo method for finite N .

4.3.1 Estimator of the Mean

Consider the output y of a system of interest. Given uncertain inputs, y is governed by a continuous random variable, Y , with probability density function $p(y)$. Recall from Section 4.1.5

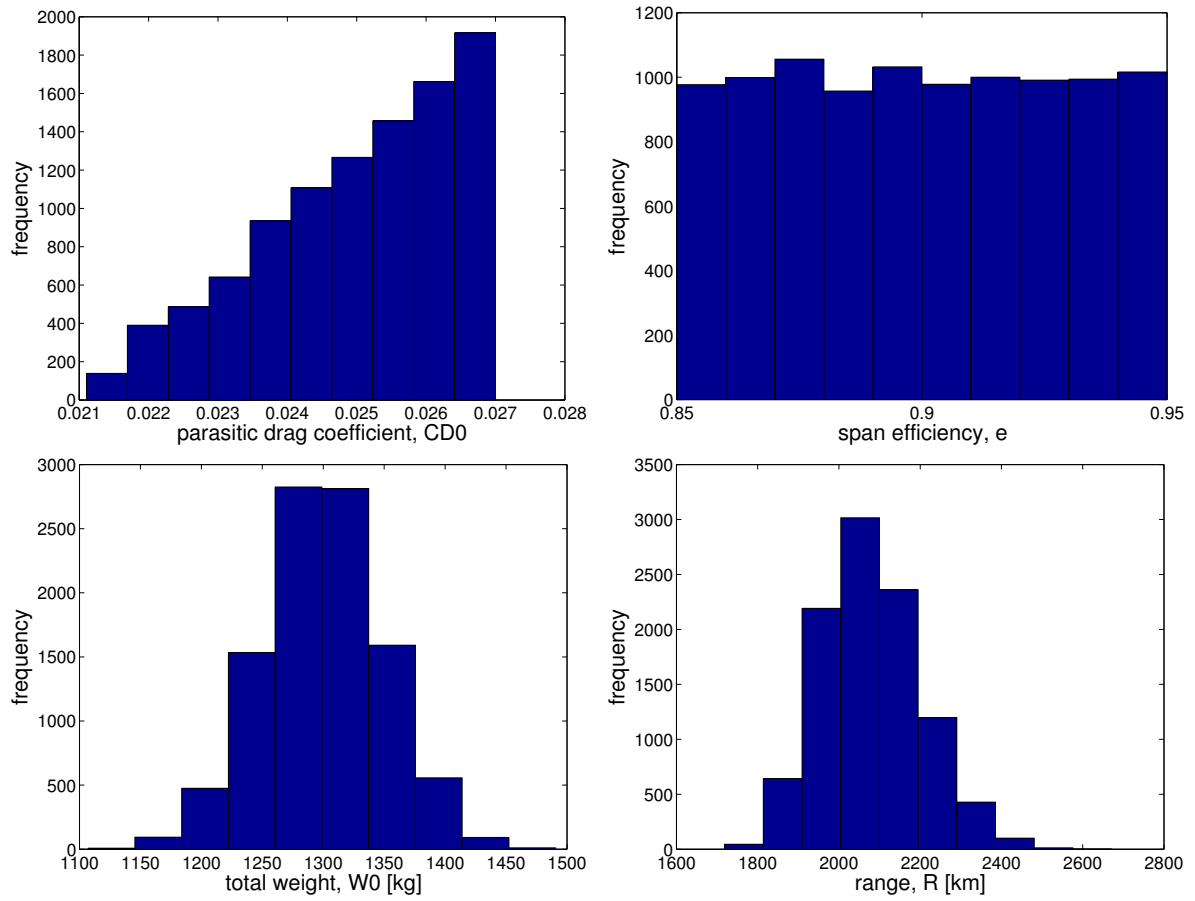


Figure 4.7: Histograms of the input samples and the range output for the multiple uncertainty source study in Example 4.6.

that the mean, or expected value, of random variable Y is given by

$$\mu_y = E(Y) = \int_{-\infty}^{\infty} yp(y)dy.$$

Now, when we run a Monte Carlo simulation, we have N output samples, y_i . Since we assume independent samples, a reasonable *estimator* of the output mean is a simple arithmetic average of the samples,

$$\bar{y} \equiv \frac{1}{N} \sum_{i=1}^N y_i. \quad (4.4)$$

This estimator is *unbiased* because it gives a zero expected error. To see this, we take the expected value of the difference between the estimator and the actual mean value μ_y ,

$$\begin{aligned} E(\bar{y} - \mu_y) &= E(\bar{y}) - E(\mu_y) \\ &= E\left(\frac{1}{N} \sum_{i=1}^N y_i\right) - \mu_y \\ &= \frac{1}{N} \sum_{i=1}^N E(y_i) - \mu_y. \end{aligned}$$

Since in the Monte Carlo method each y_i comes from an independent draw from the PDF of y , we have $E(y_i) = \mu_y$. So the above equation tells us that

$$E(\bar{y} - \mu_y) = \frac{1}{N} \sum_{i=1}^N \mu_y - \mu_y = \frac{1}{N} N \mu_y - \mu_y = 0.$$

4.3.2 Standard Error of an Estimator

In the Monte Carlo method, a computed value of $\bar{y}$ depends not only on the number of samples N , but also on the random number generator. Recall that we are randomly selecting inputs, so in general when we run Monte Carlo multiple times, we will obtain different sets of outputs $\{y_i\}$ – let’s call these sets *realizations*. For example, it’s possible that in one realization all y_i are the same because the random number generator always returned the same values ... this is possible, although in general not probable for a good random number generator. It turns out that the output dependence on N and on the random number generator are related, and we can reduce the dependence by increasing N .

Example 4.7 (Realization Variability for the Breguet Range Equation). Consider again the Breguet range equation uncertainty study presented in Example 4.6. In that example we presented a set of results showing the variation in the range (output y) given three uncertain inputs, for a single realization (which was long, $N = 10^4$ samples). Let’s now look at generating shorter realizations, but multiple times instead of just once. Table 4.3 presents

Table 4.3: Range mean estimator calculations for different Monte Carlo realizations in Example 4.6. Note, $N = 100$ for all realizations.

Realization	Range mean estimator, $\bar{y}$ [km]
1	2083
2	2088
3	2071
4	2075
5	2089
...	...

results for the range mean estimator, $\bar{y}$, for several realizations with $N = 100$ samples each.

If we think about creating many realizations, in our case 1000 of them, we can ask how the estimator values will be distributed. Figure 4.8 presents histograms of these distributions for $N = 100$ and $N = 500$. As shown in the figure, increasing N decreases the variation

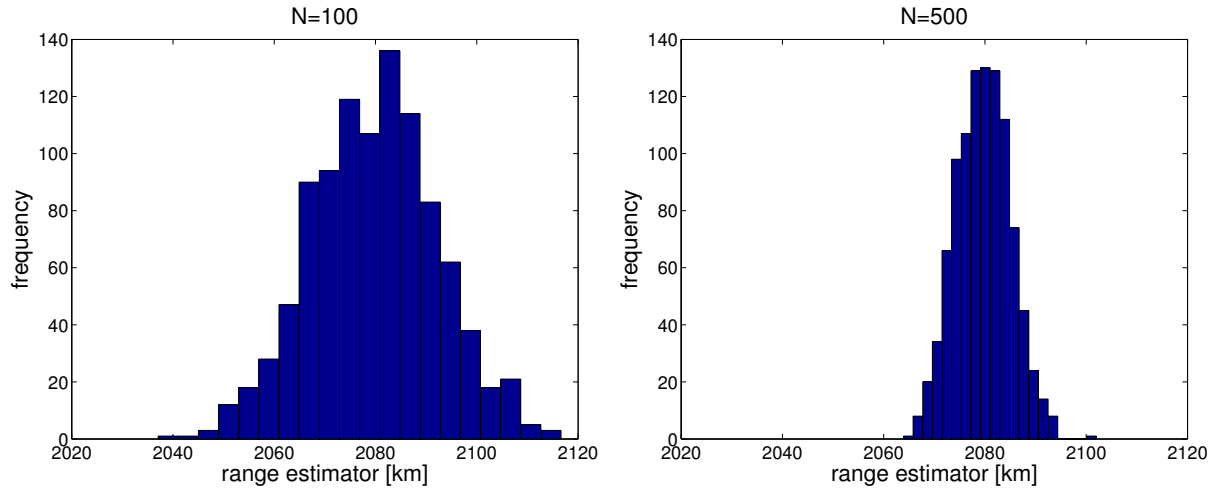


Figure 4.8: Histogram of the mean range estimator across 1000 realizations of Monte Carlo with $N = 100$ (left) and $N = 500$ (right), as described in Example 4.7.

of the mean estimator, since the spread in the histogram for $N = 500$ is smaller than that for $N = 100$. Also, the shape of the distribution resembles a “bell curve”, i.e. a normal distribution, especially for $N = 500$.

Note that the histograms in Figure 4.8 in the previous example do not show the PDF of y , $p(y)$. Instead, the present histograms show different information: they illustrate the

spread in the output mean value estimator across different realizations of a Monte-Carlo experiment. In practice, we would/could not compute multiple Monte-Carlo experiments, but we often are still interested in how much (statistical) error we make for one realization.

We have shown that the mean estimator in Eqn. 4.4 is unbiased, which means that over many Monte Carlo realizations the error will average out to zero. To get a sense of how much error we expect to see in a single Monte Carlo realization, we look at the variance of $\bar{y} - \mu_y$ across realizations,

$$\begin{aligned}
E[(\bar{y} - \mu_y)^2] &= E\left[\left(\frac{1}{N} \sum_{i=1}^N y_i - \mu_y\right)^2\right] \\
&= E\left[\left(\frac{1}{N} \sum_{i=1}^N (y_i - \mu_y)\right)^2\right] \\
&= E\left[\frac{1}{N^2} \left(\sum_{i=1}^N (y_i - \mu_y)\right) \left(\sum_{j=1}^N (y_j - \mu_y)\right)\right] \\
&= E\left[\frac{1}{N^2} \sum_{i=1}^N (y_i - \mu_y)^2 + \frac{1}{N^2} \sum_{j=1, j \neq i}^N (y_i - \mu_y)(y_j - \mu_i)\right] \\
&= \frac{1}{N^2} \sum_{i=1}^N E[(y_i - \mu_y)^2] + \frac{1}{N^2} \sum_{j=1, j \neq i}^N E[(y_i - \mu_y)(y_j - \mu_i)]
\end{aligned}$$

Since the samples y_i are independent draws, their covariance is zero, which means that the second term in the last equation above drops out. Now, in the first term, the summand is just the variance of y , i.e.

$$E[(y_i - \mu_y)^2] \equiv \sigma_y^2,$$

where σ_y is the standard deviation of the output distribution for y – this quantity is independent of N . So, returning to the realization-to-realization variance in our mean value estimator, we have

$$\sigma_{\bar{y}}^2 \equiv E[(\bar{y} - \mu_y)^2] = \frac{1}{N^2} N \sigma_y^2 = \frac{\sigma_y^2}{N}. \quad (4.5)$$

We call $\sigma_{\bar{y}} = \sigma_y/\sqrt{N}$ the *standard error* of the mean estimator, and this quantity depends inversely on $\sqrt{N}$. So the larger the sample size, the smaller the standard error, but the reduction is not very quick. For example, an order of magnitude reduction in the standard error requires 100 times as many samples, N .

For a large sample size N , the central limit theorem tells us that the distribution of the mean estimator, $\bar{y}$, over Monte Carlo realizations will be approximately normal with mean μ_y and standard deviation $\sigma_y/\sqrt{N}$. This explains the bell-shaped histograms in Example 4.7, and the reduction in the spread with increasing N .

Now, a caveat is that in practical simulations, we do not know σ_y , which is the standard deviation of the actual output y . But, after running Monte Carlo, we do have a realization, the set $\{y_i\}$, and we can estimate σ_y as the standard deviation of the samples,

$$\sigma_y^2 \approx s_y^2 \equiv \frac{1}{N-1} \sum_{i=1}^N (y_i - \bar{y})^2.$$

As defined above, s_y^2 is an unbiased estimate of the actual variance in y , σ_y^2 . Directly using s_y^2 in place of σ_y^2 in the standard error estimate (Eqn. 4.5) ignores the fact that s_y^2 is itself only approximate, and for small N the resulting additional errors may pollute the standard error estimate.

Example 4.8 (Standard Error Estimates for the Breguet Range Equation). In Example 4.7 we presented statistics for the range estimator, $\bar{y} = \bar{R}$, for $N = 100$ and for $N = 500$. Let's see if the above standard error estimates are accurate for these data. Running a long realization, $N = 1000$, we find that the standard deviation of the range is

$$\sigma_y \approx s_y = 124\text{km}.$$

So for $N = 100$, we would expect a standard error of $\sigma_{\bar{y}} = \sigma_y/\sqrt{100} = 12.4\text{km}$, and for $N = 500$ we would expect $\sigma_{\bar{y}} = \sigma_y/\sqrt{500} = 5.54\text{km}$. Using the actual data in Figure 4.8, we have that for $N = 100$, the standard deviation of $\bar{y}$ over the computed 1000 realizations is 12.06km and for $N = 500$ it is 5.49km. So the standard error estimates are quite close to the actual standard deviations over realizations.

In practice, a typical use of the standard error estimates is to determine an adequate sample size for a Monte Carlo simulation, given some desired tolerance on the statistics. This determination is done on the fly during the Monte Carlo sampling, since it is based on the sample data y_i . Finally, we note that while we have only discussed measuring error for the mean estimator, estimators exist for other statistical quantities of interest, such as variances, probabilities of failure, etc.

Chapter 5

Model Reduction

5.1 Introduction

5.1.1 Motivation

Numerical solutions to partial differential equations can become very expensive for the complex problems that are often of interest in aerospace engineering applications. Such problems include computational fluid dynamics simulations around entire aircraft, detailed structural dynamics simulations of wings, chemical reaction simulations in combustion chambers, molecular dynamics simulations of re-entry vehicles, and more. Problems involving up to billions of degrees of freedom are now common, and solutions may require hours or days to compute, even on large high-performance computing clusters. Moreover, the computational expense grows significantly for unsteady problems that require integration over many time steps.

These large, expensive calculations are generally termed *high-fidelity*, in that they yield answers derived from fundamental governing equations, such as Navier-Stokes for computational fluid dynamics. Given sufficient resolution of the discretization, e.g. a large number of points in a finite-difference discretization, the solutions obtained from high-fidelity simulations can be thought of as the *exact* or *truth* solutions. This will not actually be the case if the models used are inadequate; for example, when assuming inviscid flow, or a linear stress-strain relationship in a structure. However, such modeling errors are a separate quantity, and the notion of a truth solution can be separately defined for each model. Here, we will assume that high-fidelity solutions are computed on sufficiently refined meshes, and with models of sufficient accuracy, such that these solution can be treated as exact.

When analyzing a single aerospace vehicle, for example to assess its performance under specific conditions, the computational expense of high-fidelity simulations can certainly be worthwhile, especially compared to the cost of experiments on the vehicle or a model of it. However, many engineering tasks require more than just single-point analyses. For example, when designing a vehicle (shape, structure, materials, controls), a large number of configurations may need to be analyzed during an optimization procedure. Similarly, when deploying a real-time controller, answers may be needed in fractions of seconds rather than

hours or days. In such cases, the computational expense of high-fidelity methods is too great for their practical application. This is where model reduction comes in.

5.1.2 Overview

Model reduction is a term used to describe a systematic technique for reducing the complexity of an expensive, high-fidelity simulation. The resulting model should be much cheaper to evaluate, ideally with a cost that does not scale with the number of degrees of freedom of the high-fidelity problem. The reduced model will be inherently less accurate than the high-fidelity model, and its domain of applicability, i.e. the set of input parameters, will be smaller.

Figure 5.1 shows two possible approaches to model reduction. In both approaches, the idea is to map *input parameters*, such as flow conditions or shape parameters, to *outputs*, such as forces or moments (e.g. for a fluid dynamics problem). In the high-fidelity problem, the input parameters dictate the large number of high-fidelity equations, through boundary conditions, geometrical shape, and physical parameters. These equations are solved to give the high-fidelity state, which consists of all of the information required to represent the solution – this again is a large number of values. The state is then post-processed to provide a few outputs, or quantities of interest. Reduced models bypass the expensive part of solving

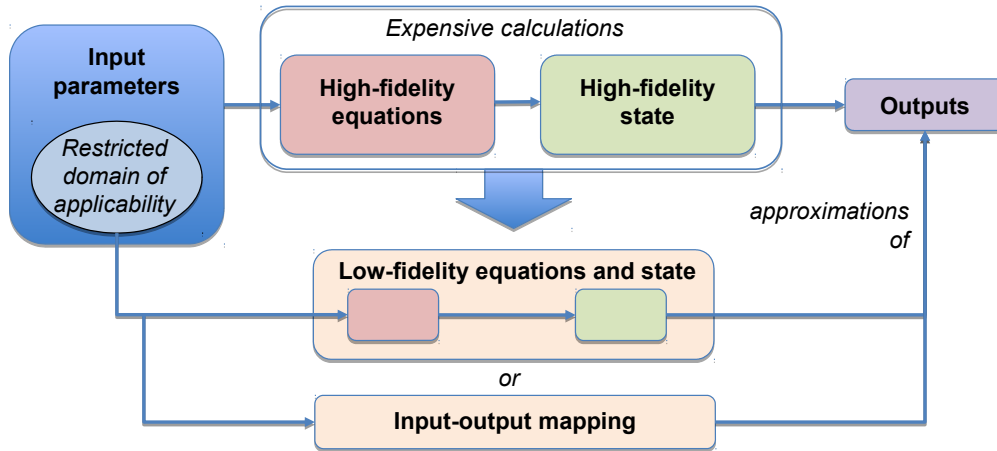


Figure 5.1: Schematic of the idea of model reduction applied to lower the computational expense of high-fidelity simulations.

the high-fidelity equations. Two possible ways of doing this are (1) by directly mapping inputs to outputs; and (2) by creating a set of lower-fidelity equations that are much cheaper to solve. We call these *interpolation* and *projection* methods, respectively, and we will look at both.

5.2 Data Compression

5.2.1 Introduction

A task related to model reduction is data compression. The question is: how do we represent or “encode” a lot of data with significantly fewer numbers? Let’s start with an example. Suppose that we are interested in describing the values of arbitrary parabolas on $N = 100$ equally-spaced points $x_j \in [0, 1]$. Some parabolas are shown in Figure 5.2. The *state*

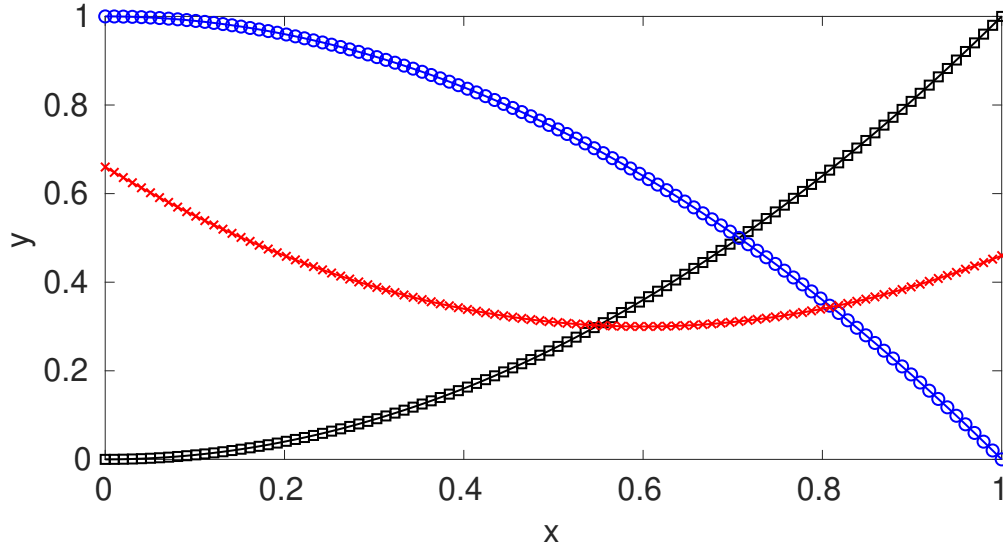


Figure 5.2: 100 point evaluations on arbitrary parabolas.

vector in this case consists of $N = 100$ values of a parabola at the prescribed points: $\mathbf{u} = [u_1, u_2, \dots, u_{100}]^T$. One way to store such a state is using these N values. That amounts to N numbers per state, which can add up to a lot of numbers if we would like to describe many parabolas. Is there a more efficient way to do this? In this case, yes, we can take advantage of the fact that the N numbers are not themselves arbitrary: the point values come from a parabola, and we do not need 100 values to describe a parabola. For example, we could use the standard monomial expansion for the i^{th} parabola

$$\text{parabola } i = a_i x^2 + b_i x + c_i, \quad (5.1)$$

where a_i, b_i, c_i are coefficients specific to parabola i . This is only 3 numbers per parabola. How do we get the 100 point values from these 3 numbers? One way is to store positions of the points, a single set of 100 numbers, in order to evaluate the parabola using Eqn. 5.1. Another way is to precompute the three monomials, $x^2, x, 1$ at the N points to obtain three vectors ϕ^a, ϕ^b, ϕ^c , all in $\mathbb{R}^N$. For a given parabola i , the values at the N points are then given by the vector

$$\mathbf{u}_i = a_i \phi^a + b_i \phi^b + c_i \phi^c. \quad (5.2)$$

The ϕ vectors are basis vectors, which we can think of as building blocks for representing the state vector $\mathbf{u}$.

5.2.2 Snapshots and the Singular Value Decomposition

In the previous example it was easy to compress the 100 point values to 3 numbers because we knew ahead of time that we were storing evaluations of parabolas. For general data compression problems, we do not have such prior information. However, for data obtained from physical systems, we can usually still rely on some “smoothness” in the data. For example, the density of a fluid cannot vary arbitrarily from point to point. The problem now is: how do we identify the smoothness automatically, with no prior information? We present one solution to this using just the data that need to be compressed.

Let’s say that we have K state vectors, $\mathbf{u}_i$, $1 \leq i \leq K$. These could be solutions of high-fidelity problems at K different parameter values. We call these vectors *snapshots*, and each snapshot consists of N values, where $N \gg 1$. Out of these vectors we construct a *snapshot matrix*, $\mathbf{S} \in \mathbb{R}^{N \times K}$, whose columns are the snapshot vectors:

$$\mathbf{S} = \begin{bmatrix} | & | & \cdots & | \\ \mathbf{u}_1 & \mathbf{u}_2 & \cdots & \mathbf{u}_K \\ | & | & \cdots & | \end{bmatrix}. \quad (5.3)$$

We then compute the singular value decomposition (SVD) of $\mathbf{S}$,

$$\mathbf{S} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T,$$

where $\mathbf{\Sigma}$ is a diagonal matrix of singular values and the matrices $\mathbf{U}$ and $\mathbf{V}$ contain the left and right singular vectors. The SVD is obtained from an eigenvalue decomposition of $\mathbf{S}^T\mathbf{S}$. It is similar to the eigenvalue decomposition, but has notable differences: for example, the singular values are all non-negative. We will not discuss the properties of the SVD in detail here. For our purposes, it is sufficient to think of the SVD as identifying the optimal basis vectors for representing the data in the snapshots. These basis vectors, ϕ_i are the columns of the matrix $\mathbf{U}$, and their importance for representing the snapshots depends on the singular values, σ_i , in $\mathbf{\Sigma}$ (note, $\sigma_1 \geq \sigma_2 \geq \sigma_3 \cdots \geq 0$):

$$\mathbf{U} = \begin{bmatrix} | & | & \cdots \\ \phi_1 & \phi_2 & \cdots \\ | & | & \cdots \end{bmatrix}, \quad \mathbf{\Sigma} = \begin{bmatrix} \sigma_1 & & \\ & \sigma_2 & \\ & & \ddots \end{bmatrix}. \quad (5.4)$$

The number of nonzero singular values and corresponding basis vectors depends on the snapshots. For example, if all snapshots were identical, then there would be only one nonzero singular value, and one basis vector needed to represent all of the snapshots. The largest number of nonzero singular values is $\min(K, N)$. However, for typical situations where the snapshots represent high-fidelity solutions to some parametrized physical problem, the singular values will decay quickly. This means that all of the snapshots can be well-represented

by a linear combination of only a small number, $n \ll N$, of basis vectors:

$$\mathbf{u} \approx u_{r,1}\phi_1 + u_{r,2}\phi_2 + \cdots + u_{r,n}\phi_n \quad \Rightarrow \quad \mathbf{u} \approx \Phi \mathbf{u}_r. \quad (5.5)$$

In this equation, the values $u_{r,j}$ are coefficients on the basis functions in the approximation of $\mathbf{u}$. The expression on the right is the matrix-form equivalent of this equation, in which $\Phi \in \mathbb{R}^{N \times n}$ is the *basis matrix*, whose columns are the n basis vectors, ϕ_j . The vector $\mathbf{u}_r \in \mathbb{R}^n$ is then the *reduced state*.

An important property of the SVD is that the basis vectors are orthonormal: $\phi_i^T \phi_j = \delta_{ij}$. This simplifies projection of a high-fidelity state to the reduced state. Multiplying the matrix form in Eqn. 5.5 by Φ^T on both sides gives the desired reduced state:

$$\Phi^T \mathbf{u} = \underbrace{\Phi^T \Phi}_{\text{identity}} \mathbf{u}_r \quad \Rightarrow \quad \mathbf{u}_r = \Phi^T \mathbf{u}. \quad (5.6)$$

5.2.3 Flowfield Example

In this section we present a data-compression example using CFD flowfield snapshots. The flowfield representations consist of entire state vectors associated with each solution. The CFD snapshots are computed on unstructured triangular meshes, where the unknowns are stored on the mesh nodes. At each mesh node, j , the unknowns consist of 5 quantities,

$$\mathbf{u}_j = [\rho, \rho u, \rho v, \rho E, \rho \tilde{v}]^T. \quad (5.7)$$

These are the states that appear in the two-dimensional Navier-Stokes equations: density, x -momentum, y -momentum, and total energy, together with one additional term, $\rho \tilde{v}$, which is used in an auxiliary equation to model Reynolds-averaged turbulence. Specifically, the turbulence model is that of Spalart-Allmaras. The mesh has 7045 unknowns, so the total number of values in each state vector is $N = 5 \times 7045 = 35225$.

The case of interest is turbulent flow over an airfoil, and we consider two parameters: the Mach number (M) and the angle of attack (α). Suppose that we have two sets of snapshots: one set of 20 *training* state vectors, and one set of 100 *test* state vectors. The (M, α) combinations for these sets are illustrated in Figure 5.3, along with Mach number contours at the corner test points in the parameter space. Our goal is to use the training solutions to derive basis vectors for compressing the test data.

We perform a proper orthogonal decomposition (POD) of the 20 training snapshots. This means we calculate $\mathbf{S} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$. Since $\mathbf{S}$ is far from a square matrix (it is 35225×20), we should compute an “economy-size” (`econ` in Matlab) SVD that will only return the nonzero singular values and corresponding singular vectors. Otherwise, $\mathbf{U}$ would be unnecessarily calculated as a very large 35225×35225 matrix.

Once we have the economy-size SVD, the 20 columns of $\mathbf{U}$ are all of the basis vectors, and the entries in the diagonal matrix $\mathbf{\Sigma}$ are the singular values. In Figure 5.4, we plot the ranked singular values (largest to smallest), using a log scale for the singular values, versus their index. We see that by $n_b = 16$, the singular values have dropped by about four orders

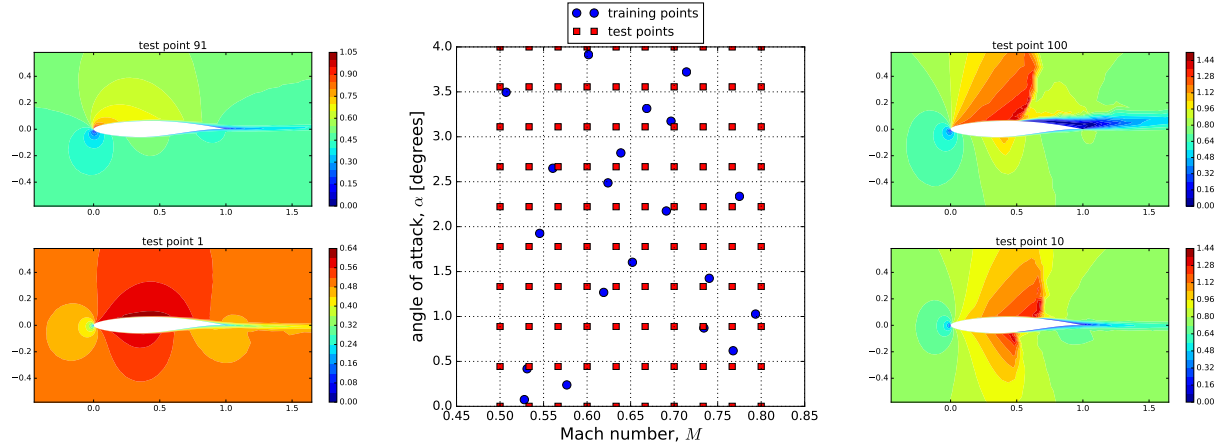


Figure 5.3: Parameter space and sample flowfield snapshots for the airfoil data compression case.

of magnitude compared to the largest (first) singular value. Since the magnitude of the singular values indicates the importance of the corresponding basis vector for approximating the snapshots, this means that we may be able to obtain a decent basis using fewer than the 20 available basis vectors.

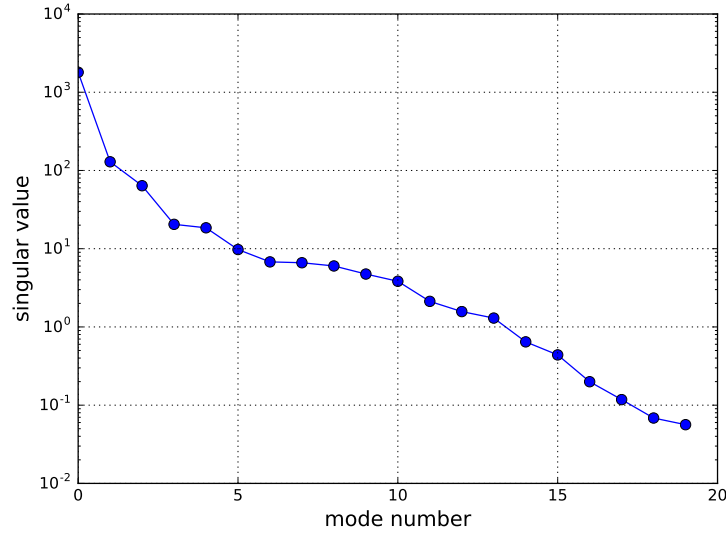


Figure 5.4: Singular values of the training snapshot matrix for the airfoil example.

The basis vectors are building blocks of field solutions, and so we can plot them just like we would plot the states. Figure 5.5 shows the density and x -momentum components of the first four basis vectors. The first two modes focus on the main flow features common to all/most of the snapshots: stagnation points, boundary layers, overall flowfields. The

subsequent modes, e.g. mode 3, enable resolution of features that do not appear in all of the snapshots, such as shocks.

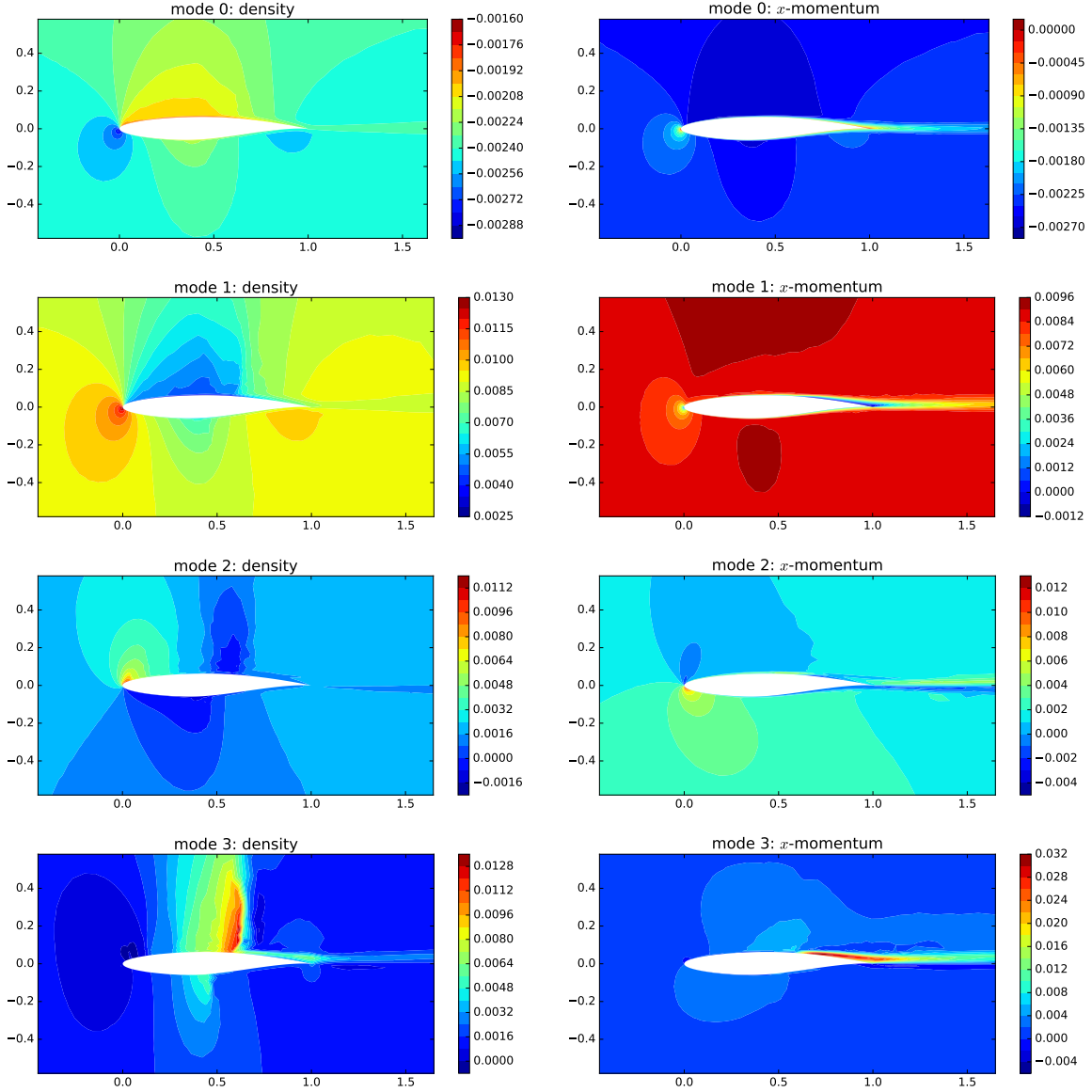


Figure 5.5: Density and x -momentum components of the first four basis vectors for the airfoil example.

Using the basis vectors, we now compress the 100 test snapshots by projecting them onto the space spanned by the basis vectors. We look at compression using different numbers of basis vectors: $n = 2, 4, 8, 16$ (the most we could use is all 20). For each n , we calculate the discrete L_2 error between the test snapshots and projections. For a given test snapshot $\mathbf{u}_i$,

the L_2 error is

$$e_i \equiv \|\mathbf{u}_i - \Phi \Phi^T \mathbf{u}_i\|_2, \quad (5.8)$$

where $\Phi \in \mathbb{R}^{N \times n}$ is the basis matrix. From Eqn. 5.6, we see that $\Phi^T \mathbf{u}_i$ is the size n reduced state corresponding to $\mathbf{u}_i$. Then, by Eqn. 5.5, $\Phi \Phi^T \mathbf{u}_i$ is the corresponding high-fidelity approximation. If the reduced basis were perfect at representing all of the snapshots, this result would be identical to the starting snapshot $\mathbf{u}_i$. However, in practical cases we will be using a truncated basis or approximating a state out of the training snapshot space, and hence the error e_i will generally be nonzero. The magnitude of e_i then indicates the accuracy of the approximation using a given number of basis vectors (columns in Φ). Note, we compute the L_2 error over all physical state components together (mixing density, x -momentum, etc.).

Figure 5.6 shows a contour plot of the L_2 error in the parameter space, (M, α) . Making this contour plot is straightforward because the test data are obtained on a regular 10×10 grid in parameter space. We see that as the number of basis vectors (n) increases, the error decreases, and by $n = 16$, most of the test data are well approximated. Note that the error is highest at the large Mach numbers and angles of attack, where the flow is transonic (shocks are present), and where we do not have many training points.

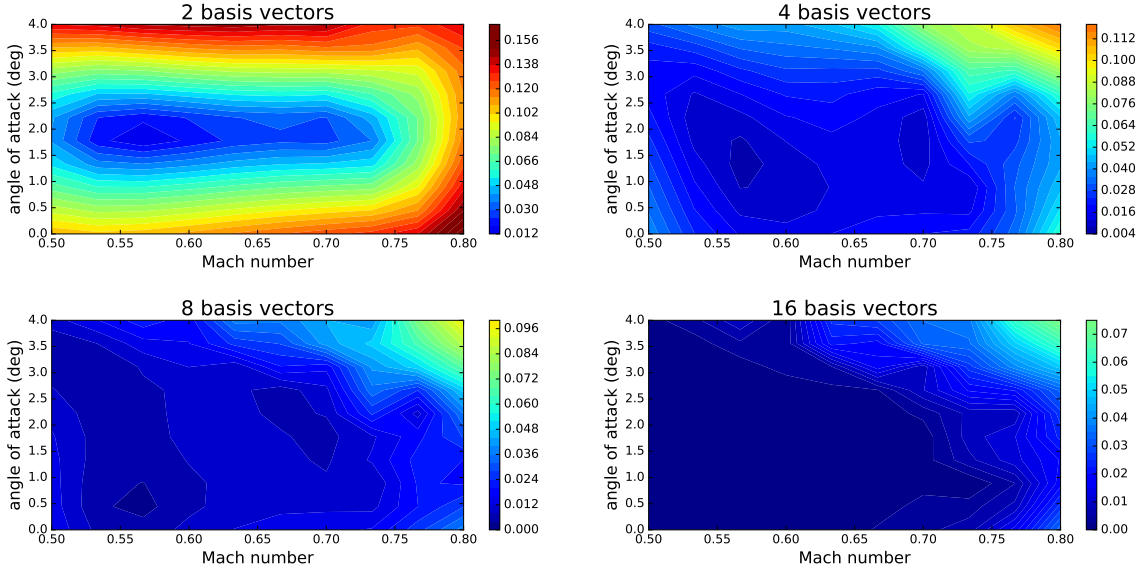


Figure 5.6: L_2 error plots for the airfoil example, computed using different numbers of basis vectors.

In summary, we showed how a singular value decomposition of snapshots can be used to compress data, which can be either the same snapshots or other data in nearby regions

in parameter space. For the airfoil case, we used 20 training flowfield snapshots distributed randomly in the two-dimensional parameter space of Mach number and angle of attack. An SVD of the training snapshot matrix gave 20 singular vectors, subsets of which were used as basis vectors for approximating the state. Compression of the data is attained because for each state vector, we only need to store $n \leq 20$ numbers, the basis vector coefficients, instead of the $N = 35225$ state values. To reconstruct the flowfields, we do need to store the n basis vectors, but these are applicable to all states that we approximate. Finally, we note that the application of the SVD to snapshot data is termed the *proper orthogonal decomposition* (POD).

5.3 Equation Projection

5.3.1 Introduction

The previous section showed how we can use the SVD to compress high-fidelity state data. This compressed, or reduced, representation is a key ingredient in what's called *projection-based* model reduction. In this section we show how to derive the reduced equations by combining the compressed state approximation with projection of the discretized equations.

5.3.2 Linear Projection

Consider the following high-fidelity linear system of equations:

$$\mathbf{A}\mathbf{u} = \mathbf{f}, \quad (5.9)$$

where $\mathbf{A} \in \mathbb{R}^{N \times N}$, $\mathbf{u} \in \mathbb{R}^N$, $\mathbf{f} \in \mathbb{R}^N$, and $N \gg 1$. Our goal is to obtain a reduced version of this system, using $\mathbf{u}_r$ as the state. We do this in two steps. First, we reduce the number of unknowns by restricting $\mathbf{u}$ to lie in the space spanned by n basis vectors, the columns of $\Phi \in \mathbb{R}^{N \times n}$. Using the $\mathbf{u}$ representation from Eqn. 5.5, Eqn. 5.9 becomes

$$\mathbf{A}\Phi\mathbf{u}_r = \mathbf{f}. \quad (5.10)$$

This is now an overdetermined system of N equations for n unknowns. To make it solvable, we apply the second step: we project the equations by multiplying both sides by Φ^T on the left. We are effectively creating n equations as weighted linear combinations of the N equations, with the entries in Φ as the weights. The result is:

$$\underbrace{\Phi^T \mathbf{A} \Phi}_{\mathbf{A}_r} \mathbf{u}_r = \underbrace{\Phi^T \mathbf{f}}_{\mathbf{f}_r} \Rightarrow \mathbf{A}_r \mathbf{u}_r = \mathbf{f}_r. \quad (5.11)$$

This is now a small, $n \times n$, system of equations for the reduced state. Its solution can be reconstructed in the high-fidelity space by $\mathbf{u} \approx \Phi\mathbf{u}_r$.

We note that using the same basis matrix for both the approximation and projection is called *Galerkin* projection. When the spaces differ, which is often desirable, the projection is called *Petrov-Galerkin*.

5.3.3 Parametrized Systems

The reduced system derived in Eqn. 5.11 requires some modification to become useful for problems typically encountered in engineering. An important problem of interest is a *parametrized* system of equations, where the matrix $\mathbf{A}$ and/or right-hand side vector $\mathbf{f}$ depend on p parameters, $\boldsymbol{\mu} \in \mathbb{R}^p$. For example, we may want to create a reduced model for solving a heat transfer problem with different Dirichlet boundary conditions, or different thermal conductivity values. Both of these would affect the matrix and/or right-hand side of the linear system. We do not wish to go through the process described in the previous section every time we need another reduced solution, as those are expensive steps. For example, creating the reduced matrix $\mathbf{A}_r = \Phi^T \mathbf{A} \Phi$ requires matrix-matrix multiplications, and even though $\mathbf{A}$ is usually sparse, the complexity is at least $\mathcal{O}(N)$. We would instead like to obtain the reduced solutions in a complexity that does not depend on N .

Let's assume that $\mathbf{A}$ and $\mathbf{f}$ depend linearly on the parameter vector $\boldsymbol{\mu}$:

$$\mathbf{A} = \mathbf{A}_0 + \sum_k^p \mathbf{A}_k \mu_k, \quad \mathbf{f} = \mathbf{f}_0 + \sum_k^p \mathbf{f}_k \mu_k, \quad (5.12)$$

where $\mathbf{A}_0 \in \mathbb{R}^{N \times N}$ is the matrix corresponding to $\boldsymbol{\mu} = \mathbf{0}$, and $\mathbf{A}_k \in \mathbb{R}^{N \times N}$ is the part of $\mathbf{A}$ that scales linearly with μ_k . Similar definitions hold for the vectors $\mathbf{f}_0$ and $\mathbf{f}_k$. Substituting these expressions into Eqn. 5.9 and applying the two steps from the previous section (approximation + projection) gives

$$\left[\sum_{k=0}^p \underbrace{\Phi^T \mathbf{A}_k \Phi}_{\mathbf{A}_{rk}} \mu_k \right] \mathbf{u}_r = \sum_{k=0}^p \underbrace{\Phi^T \mathbf{f}_k}_{\mathbf{f}_{rk}} \mu_k \quad \Rightarrow \quad \mathbf{A}_r \mathbf{u}_r = \mathbf{f}_r, \quad (5.13)$$

where for convenience we have defined $\mu_0 \equiv 1$. Note that $\mathbf{A}_r \in \mathbb{R}^{n \times n}$ and $\mathbf{f}_r \in \mathbb{R}^n$ are obtained by summing the $\mathbf{A}_{rk}$ and $\mathbf{f}_k$ matrices, respectively, with μ_k as the weights. By precomputing $\mathbf{A}_{rk}$ and $\mathbf{f}_k$, we can then efficiently form and solve the reduced system for an arbitrary set of new parameters, with no $\mathcal{O}(N)$ operations.

5.3.4 Offline-Online split

The application of projection-based model reduction involves two stages: offline and online. The *offline* stage consists of taking snapshots, creating basis vectors, and forming all of the reduced matrices: all $\mathcal{O}(N)$ operations. On the other hand, the *online* stage pertains to actually using the reduced-model, i.e. forming and solving the reduced system. All operations in the online stage scale only with n , and not N . Figure 5.7 shows the breakdown of tasks into the two stages.

Although the calculations in the offline stage may be expensive, they are done once and the resulting reduced matrices are stored. Then, given new parameters, the reduced system is constructed inexpensively by a linear combination of the reduced matrices and right-hand sides. In practical problems, N can be very large, e.g. millions or billions, whereas n may

Offline	Online
1. Take K snapshots of the full state by solving the high-fidelity system with K different parameters. 2. Calculate the SVD of the snapshot matrix: $\mathbf{S} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ 3. Set the basis matrix $\mathbf{\Phi}$ to the first n columns of $\mathbf{U}$. 4. Calculate the reduced matrices $\mathbf{A}_{rk}$ and right-hand sides $\mathbf{f}_{rk}$.	1. Given a new parameter vector $\boldsymbol{\mu}$. 2. Form the reduced matrix $\mathbf{A}_r$ and right-hand side $\mathbf{f}_r$: $\mathbf{A}_r = \sum_{k=0}^p \mathbf{A}_{rk}\mu_k, \quad \mathbf{f}_r = \sum_{k=0}^p \mathbf{f}_{rk}\mu_k$ 3. Solve the reduced system $\mathbf{A}_r \mathbf{u}_r = \mathbf{f}_r$. 4. Post-process $\mathbf{u}_r$ or reconstruct $\mathbf{u} = \mathbf{\Phi}\mathbf{u}_r$ for visualization.

Figure 5.7: Description of offline and online stages in model reduction.

only be $\mathcal{O}(100)$, which yields tremendous savings in the computational time when solving the reduced system compared to the high-fidelity system.

5.3.5 Unsteady problems

Projection-based model reduction extends naturally to unsteady problems. We still assume linearity in the state and parameters, so that the high-fidelity system can be written as

$$\dot{\mathbf{u}} = \mathbf{A}\mathbf{u} + \mathbf{f}, \quad (5.14)$$

where $\dot{\mathbf{u}}$ is the time derivative of $\mathbf{u}$, and $\mathbf{A}$ and $\mathbf{f}$ depend on the parameter vector $\boldsymbol{\mu}$ according to Eqn. 5.12. Substituting the state approximation and multiplying the entire equation by $\mathbf{\Phi}^T$ on the left gives

$$\mathbf{\Phi}^T \mathbf{\Phi} \dot{\mathbf{u}}_r = \mathbf{\Phi}^T \mathbf{A} \mathbf{\Phi} \mathbf{u} + \mathbf{\Phi}^T \mathbf{f}. \quad (5.15)$$

We recall that the basis returned by the singular-value decomposition is orthonormal, so that $\mathbf{\Phi}^T \mathbf{\Phi} = \mathbf{I} \in \mathbb{R}^{n \times n}$, i.e. the $n \times n$ identity matrix. Eqn. 5.15 then becomes

$$\dot{\mathbf{u}}_r = \mathbf{A}_r \mathbf{u}_r + \mathbf{f}_r, \quad (5.16)$$

where $\mathbf{A}_r$ and $\mathbf{f}_r$ are the same as those given in Eqn. 5.13. For a given set of parameters, these matrices are constant and so Eqn. 5.16 can be integrated in time to determine the evolution of the reduced state, $\mathbf{u}_r$. The initial condition for the time integration is the projection of the high-fidelity initial condition: $\mathbf{u}_r^0 = \mathbf{\Phi}^T \mathbf{u}^0$.

For either steady or unsteady problems, to calculate outputs from a reduced state, we can either use the reconstructed high-fidelity solution, or we can calculate the outputs directly from $\mathbf{u}_r$. The latter is possible in particular if the outputs are linear functions of the state.

Suppose that we obtain an output vector $\mathbf{y} \in \mathbb{R}^q$ from the high-fidelity state $\mathbf{u} \in \mathbb{R}^N$ through a matrix-vector multiplication

$$\mathbf{y} = \mathbf{C}\mathbf{u}, \quad (5.17)$$

where $\mathbf{C} \in \mathbb{R}^{q \times N}$. Using the approximation for $\mathbf{u}$ from Eqn. 5.5, we have

$$\mathbf{y} = \underbrace{\mathbf{C}\Phi}_{\mathbf{C}_r} \mathbf{u}_r, \quad (5.18)$$

where $\mathbf{C}_r \in \mathbb{R}^{q \times n}$ is the reduced output matrix. $\mathbf{C}_r$ can be precomputed during the offline stage, so that the online output calculation is independent of the number of high-fidelity degrees of freedom, N .

Finally, all of the discussion so far has been for linear systems and outputs. When either or both of these are nonlinear, the projection technique can still be applied, but additional steps are necessary to keep the complexity of the online stage to $\mathcal{O}(n)$. These steps, which amount to further approximations and rely on certain smoothness of the nonlinear functions in the problem, constitute what is known as *hyperreduction*. This topic is beyond the scope of this course, but those interested may look into the discrete empirical interpolation method (DEIM), one of the first methods employing hyperreduction.

5.4 Interpolation Surrogates

Projection-based model reduction gives us a reduced state $\mathbf{u}_r$ that can be reconstructed to the high-fidelity state $\mathbf{u}$ through the basis matrix Φ . This is useful in cases when we would like to visualize or otherwise post-process the high-fidelity state. However, in some cases, all we care about is one or a few output quantities computed from the state. In this case, a simpler approach is to map inputs, μ , directly to outputs, $\mathbf{y}$. Such a mapping can be done outside of the high-fidelity simulation code(s), and thus has the advantage of not being as *intrusive* as projection-based methods. Specifically, projection-based methods require access to the discretized equations, which may not be available if, for example, a commercial closed-source code is used for the high-fidelity analyses. In contrast, interpolation methods can still be applied in these cases, by treating the high-fidelity code as a “black-box”.

5.4.1 Polynomial Fitting

A simple example of interpolation is polynomial fitting. Suppose that we have one scalar input μ , and one scalar output y . We run the high-fidelity code 4 times and obtain the data shown in Figure 5.8.

Without regard to the intermediate state vector that generated the output, we can try to fit the y versus μ data. One way to do this is with regression, generated by a least-squares approach. This could be linear, quadratic, cubic, etc., as shown in Figure 5.8. With only four points, the maximum order polynomial that we can fit is 3. Although high-order

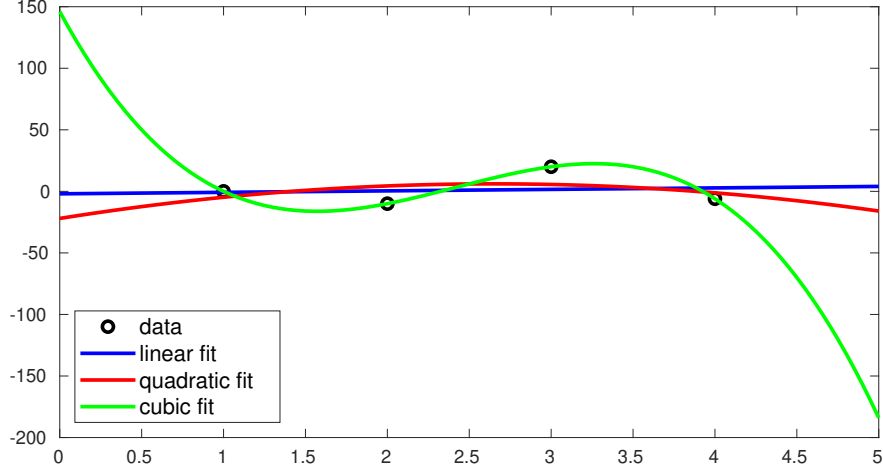


Figure 5.8: Curve fits of four data points using polynomial functions of different orders.

polynomial fits will yield more accuracy at the sampled points, a potential disadvantage is *over-fitting*: accuracy at sampled points does not necessarily translate to accuracy in between the points and may cause large errors away from the samples, e.g. when extrapolating. In higher-dimensions (more parameters), polynomial fitting can be extended by using multi-dimensional tensor-product polynomials, defined either globally or in local patches. As the number of parameters grows, the complexity, cost, and data requirements of polynomial fitting grow exponentially, so that the number of parameters is usually capped at 10 – 15. This is known as the *curse of dimensionality*.

5.4.2 Kriging

Alternatives to polynomial fitting exist for interpolating outputs, particularly in high-dimensions. One approach is *kriging*, also known as Gaussian-process (GP) regression. In kriging, the output is expressed as a weighted combination of nearby (in parameter space) sampled data, where the weights are determined from assumed, parametrized correlation functions, whose parameters (known as hyper-parameters) are obtained through an optimization procedure using the known samples. Kriging has the advantage of requiring less structure in the training data, although it places the burden of quality fits on the assumed correlation functions.

5.4.3 Artificial Neural Networks

Another approach to fitting is artificial neural networks (ANNs). A typical ANN for creating an input-to-output mapping consists of an input layer of *neurons*, which represents the input parameter vector $\boldsymbol{\mu}$, one or more hidden layers of neurons in which each layer is a function of the previous layer through a linear map combined with a nonlinear activation function, and a final output layer that returns $\mathbf{y}$. The maps between neurons involve parameters that are calculated through an optimization procedure that minimizes the error between the

ANN predictions and the training data. ANNs have become popular in machine-learning applications, and many implementations are now available. They do require choices in the number of layers and their size, in the definition of the error/loss function, and in tuning the training parameters.

A neural network maps an input vector to an output vector through one or more hidden layers. Figure 5.9 shows an example structure of such a network, called a multi-layer perceptron. The network shown consists of two hidden layers, $\mathbf{x}_1, \mathbf{x}_2$, between the input layer, $\mathbf{x}_0$, and the output layer, $\mathbf{x}_{\text{out}}$. Multi-layer perceptron networks are typically fully-connected,

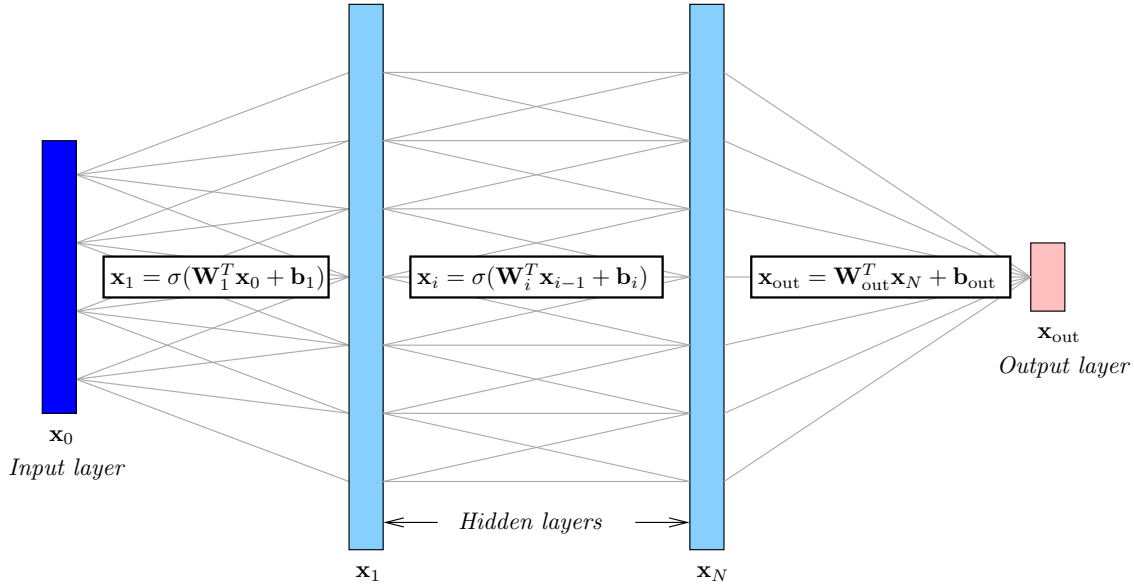


Figure 5.9: Structure of an artificial neural network with $N = 2$ hidden layers

and the formulas for computing the states within each layer are shown in Figure 5.9. An activation function, $\sigma(x)$, is applied entry-wise in the maps to the hidden layers. Examples of activation functions include the sigmoid function, $\sigma(x) = 1/(1 + e^{-x})$, and the rectified linear unit, $\sigma(x) = \max(x, 0)$. No activation function is typically used in the output layer calculation. The parameters associated with the network consist of the weights and biases,

$$\mathbf{W}_i \in \mathbb{R}^{n_i \times n_{i-1}}, \quad \mathbf{b}_i \in \mathbb{R}^{n_i},$$

where n_i is the number of neurons in layer i .

A neural network is trained to reproduce outputs for given inputs. As such, it requires *training data*, consisting of both inputs and outputs. The more training data points, the better the network training, and typically thousands or more data points are required to obtain a usable network. The amount of data required also depends on the complexity of the function being approximated, and on the structure of the network, i.e. the number of hidden layers and their sizes. A large network with many weights and biases will require more training data than a small network, which has fewer parameters to optimize.

While large networks provide accurate representations of the input data, they come with the danger of *over-fitting*: reproducing the training data well, but not being able to generalize to unseen data. An analogy from curve fitting is using a very high-order polynomial to fit noisy data: for a sufficiently high order, a polynomial can fit the input data exactly, but the fit will be oscillatory, and predictions in between the training data points will generally not be accurate. Therefore, in choosing the network architecture, an effort should be made to keep the network as small as possible while providing acceptable results.

When training a network, the figure of merit is the *loss function*, which measures how accurately the network is able to reproduce the training data. Typically, the *mean-squared error* (MSE) loss function is used, defined as

$$\text{MSE} \equiv \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} \|\mathbf{y}_i - \mathbf{y}_i^{\text{train}}\|^2, \quad (5.19)$$

where the index i ranges over the training data points, the vector $\mathbf{y}_i^{\text{train}}$ is the output vector for training data point i , and the vector $\mathbf{y}_i$ is the network prediction of the output vector for the input data at point i . Monitoring the MSE during training gives an indication of how well the optimization is going: i.e. how accurately the network can reproduce the training data.

To avoid over-fitting, a percentage (e.g. 30%) of the training data is often reserved for *validation*. The validation data are not used in training, but the neural network is applied to the validation inputs, and the validation MSE is calculated and monitored. If the validation MSE begins to increase, the training is stopped, as any further optimization of the weights/biases will improve performance only for the training data, without generalizing to the validation data that is not part of training.

Example 5.1 (Neural network for an analytical function). In this example we look at setting up, training, and testing a small neural network that replicates the following scalar function of two variables, x_1 and x_2 :

$$y = 2x_1^2 + 3x_2^2 + 0.1\epsilon, \quad (5.20)$$

where ϵ is a random variable uniformly distributed between -0.5 and 0.5, included to simulate noise in the data. We generate 1000 pieces of input data randomly, by creating a random 1000×2 matrix, which is `Xtrain` in the code given in Listing 5.1. Each row of this matrix contains the two variables x_1 and x_2 . The training data vector `Ytrain` is constructed from the input data by evaluating Eqn. 5.20.

Listing 5.1: Code for training and testing a neural network with a known analytical function.

```
1 function M = nn
2 % Example of training a simple neural network to reproduce a known
3 % function of two variables
4
5 rng('default'); % set default random number generator
6 ntrain = 1000; % number of training data rows
7 Xtrain = rand(ntrain,2); % training inputs
8 Ytrain = 2*Xtrain(:,1).^2 + 3*Xtrain(:,2).^2 + 0.1*(rand(ntrain,1)-0.5);
9
10 % train a neural network
11 M = fitrnet(Xtrain,Ytrain,'Standardize',true,'Activations','sigmoid','LayerSizes',[10 10]);
12
13 % plot the training convergence history
14 H = M.ConvergenceInfo.History;
15 figure(1); clf;
16 semilogy(H(:,1), H(:,2), 'b-', 'linewidth', 2);
17 xlabel('Network_training_iteration'); ylabel('Training_loss');
18 set(gcf, 'paperposition', [0,0,4,3], 'papersize', [4,3]);
19 print('-dpdf', '../figs/nn_hist.pdf');
20
21 % plot predictions on new test data
22 ntest = 200;
23 Xtest = rand(ntest,2); % testing inputs
24 Ytest = 2*Xtest(:,1).^2 + 3*Xtest(:,2).^2 + 0.1*(rand(ntest,1)-0.5);
25 P = predict(M,Xtest);
26 figure(2); clf;
27 plot(Ytest,P,'b.', 'linewidth', 2); hold on
28 plot(Ytest,Ytest, 'r-', 'linewidth', 1)
29 h = legend('predictions', 'ideal', 'location', 'northwest');
30 xlabel('True_output'); ylabel('Predicted_output');
31 set(gcf, 'paperposition', [0,0,4,3], 'papersize', [4,3]);
32 print('-dpdf', '../figs/nn_pred.pdf');
```

After constructing the input and output matrices, we call the function `fitrnet` to create the neural network model that maps a size-2 input layer to a scalar output through two hidden layers, each of 10 neurons. The call to `fitrnet` also trains the network (optimizes the weights and biases), and we can plot the convergence history of the loss – shown in Figure 5.10(a). Once the network is trained, we generate a new data set for testing, `Xtest`. We then evaluate the true output on the test data and store it in `Ytest`, and we compare it to the neural-network predictions on the data, stored in `P`. The scatter plot in Figure 5.10(b) shows the predictions versus the true outputs: the agreement is very good for the simple function considered here.

5.4.4 Example of Surrogates

Models obtained from interpolation techniques are often called *surrogates*. Due to their simplicity and minimal invasiveness, they are usually the first reduced model attempted in applications. However, because they only use inputs and outputs, and not the underlying governing equations, they generally require more training data for a given level of accuracy compared to projection-based methods. The latter, although more intrusive, incorporates

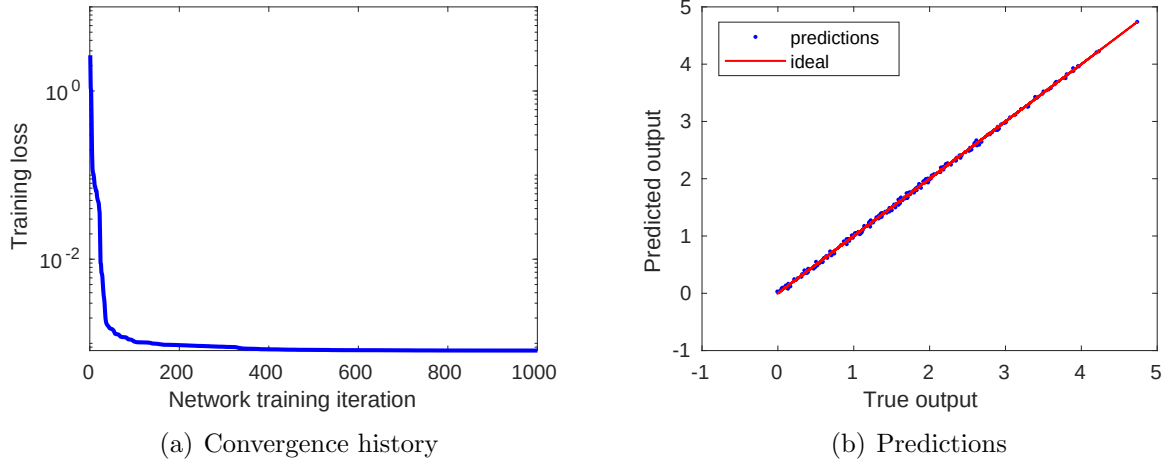


Figure 5.10: Example neural network training history and prediction results.

the discrete equations (through a projection) into the reduced model. This then brings more information into each online evaluation. Figure 5.11 demonstrates this effect for a two-dimensional turbulent flow example on a parametrized trapezoidal geometry. With just four high-fidelity samples, the projection-based reduced model produces output data over the two-dimensional parameter space that much more closely track the true output compared to simple (bi-linear) interpolation.

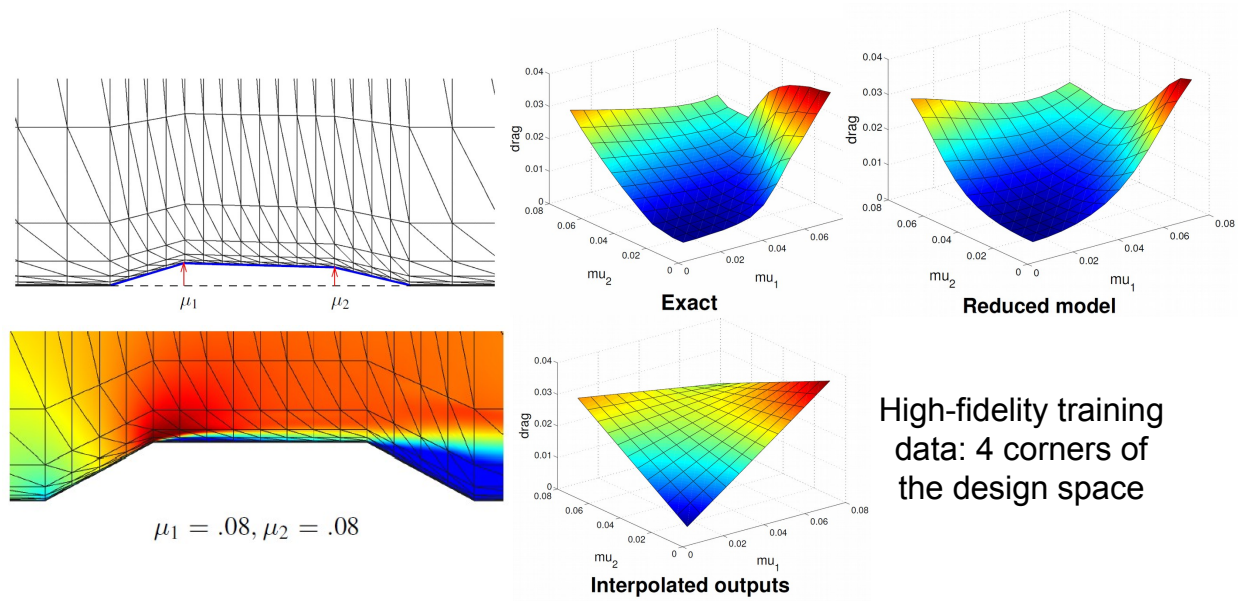


Figure 5.11: Comparison of a projection-based reduced model versus output interpolation using four high-fidelity samples of flow over a parametrized (μ_1, μ_2) trapezoidal geometry.

Chapter 6

Finite Element Methods

6.1 Function Approximation

So far we have seen finite difference methods, in which we solve for an approximate state at nodes of a grid, and finite volume methods, in which we solve for the averages of the solution on cells of a mesh. In both of these cases, if we were to inquire about the value of the solution at an arbitrary point in the domain, such as in between grid nodes in a finite-difference method, we would have to do a little interpretation. For example, we might interpolate nearby grid nodes or cell averages.

In contrast, finite element methods rely on a *functional representation* of the solution. That is, the result of a finite element method is a function that we can evaluate at any arbitrary point. Now, a function is a collection of an infinite number of points, and as discrete method a finite element algorithm cannot “solve” for all of these values at once. Instead, in finite elements we make approximations. We do this by restricting the set of possible functions that we are going to look at when calculating the solution. The resulting set is a *finite-dimensional function space*.

6.1.1 Finite-Dimensional Function Spaces

What is a finite-dimensional function space? Let’s look at an example: polynomials of a certain order, p , constitute a finite dimensional function space. In particular, in one spatial dimension, x , we can write a polynomial, $u(x)$, of order p using $p + 1$ coefficients,

$$u(x) = a_p x^p + a_{p-1} x^{p-1} + \dots + a_1 x + a_0.$$

Since we can choose the $p + 1$ coefficients a_i arbitrarily, there are an infinite number of polynomials of a given order p . In addition, each polynomial is defined at an infinite number of points, x . However, we say that $u(x)$ as defined above lies in a $(p+1)$ -dimensional function space. Why? Because $u(x)$ is a superposition of only $p + 1$ linearly-independent functions, which are: $1, x, x^2, \dots, x^p$. That is, we only have $p + 1$ “knobs” that we can adjust when defining $u(x)$. This is now tractable from a computational point of view, since we can easily store $(p + 1)$ numbers on a computer.

The space of polynomials of order p is just one example of a finite-dimensional function space, although it is by far the most widely-used for finite elements. In general, we define a function space by *basis functions*, $\phi_i(x)$, $i \in [1 \dots N]$, where N is the number of basis functions. For example, for polynomials of order p , a possible set of $N = p + 1$ basis functions is

$$\phi_0 = 1, \quad \phi_1 = x, \quad \phi_2 = x^2, \quad \dots, \quad \phi_p = x^p.$$

This is called the *monomial* basis, and it is usually fine to use for low orders (up to about $p = 5$) – for higher-orders, the basis functions start looking more and more linearly-dependent when using finite precision arithmetic on computers, so methods based on this basis will become increasingly ill-conditioned.

We then approximate a function $u(x)$ by a linear combination of the basis functions,

$$u(x) \approx \sum_{j=1}^N U_j \phi_j(x). \quad (6.1)$$

where U_j is the coefficient on basis function j . These coefficients are the “knobs” in the functional approximation. They will become the unknowns in a finite element method. Figure 6.1 provides an illustration of such a linear combination using two basis functions.

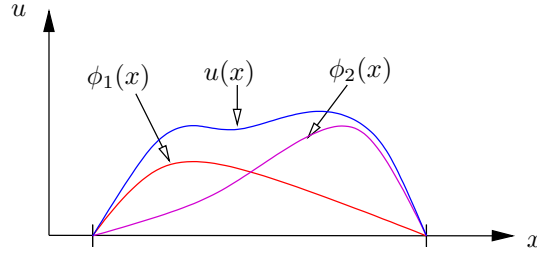


Figure 6.1: Schematic of a representation of a function as the linear combination of basis functions, see Eqn. 6.1. In this case, $u(x)$ is obtained by combining $N = 2$ basis functions, $\phi_1(x)$ and $\phi_2(x)$.

Note, in multiple dimensions, basis functions must be defined over $\vec{x} \in \mathbb{R}^d$ ($d = \text{dimension}$) instead of just $x \in \mathbb{R}$, and there are various ways to do this. For example, in two dimensions one can define polynomials of x and y , and a possible basis is the set of functions $\{x^r y^s\}$, where $r + s \leq p$.

6.1.2 Local Support

Constructing basis functions over complex domains can be a daunting task if we consider the entire domain all at once. For example when analyzing an intricate structural part, it is nearly impossible ahead of time to determine basis functions over the entire domain

that are relevant/necessary/admissible for representing the desired solution. To address this problem, finite element methods break the computational domain into a large number of simpler domains, called *elements* of a mesh. An element is the same object as a cell in finite volume methods.

Given a mesh of elements, we can now simplify the task of constructing basis functions by considering only basis functions with *local support*. A function has local support if it is nonzero only on a small subset of the elements. This simplifies the definition of basis functions because we do not have to define these functions everywhere. Approximating a solution over the entire domain is now a systematic process.

Standard finite element methods typically use linear (or bi/trilinear) basis functions associated with nodes of a mesh. Figure 6.2 shows examples of such basis functions in one dimension and in two dimensions, on triangles. Note that these basis functions take on a value of 1 at one and only one node, and that they are nonzero (piecewise linear, in fact) only on the elements adjacent to that node.

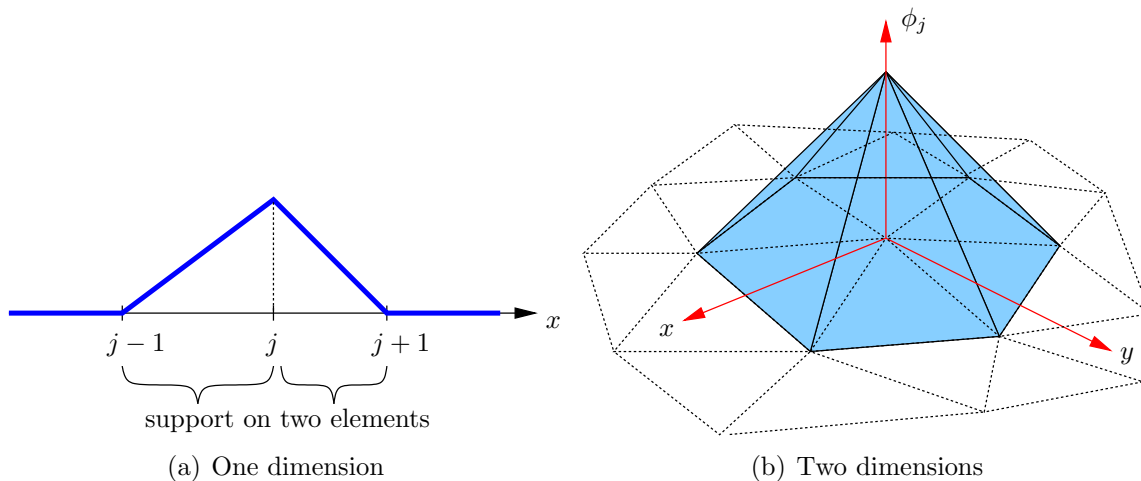


Figure 6.2: Hat functions in one and two spatial dimensions.

With this basis, we still use Eqn. 6.1 to approximate the solution. Note that the unknowns U_j , which are the coefficients of the basis functions in the approximation, become just the solution values at the nodes. This is because each basis function is one at the node with which it is associated and zero at all other nodes. So in the solution approximation in Eqn. 6.1, we are effectively “pitching a tent” from nodal values of the solution, U_j . Although with this basis we are solving for nodal values, which we also do in finite difference discretizations, the equations for these unknowns will generally be different between the two methods. For example, in finite element methods we can systematically obtain equations on an arbitrary unstructured mesh, a task that would be difficult/ambiguous for finite-difference methods.

Finally, boundary conditions can also be handled systematically in finite element methods, unlike in finite difference methods where we potentially have to change the stencil at

boundaries. When approximating a function for which boundary values are known, we just use the known nodal values on these boundaries as the coefficients of the corresponding basis functions. These nodes will no longer carry unknowns. Boundary conditions that specify fluxes or gradients can also be handled systematically, as we will soon see.

6.2 The Equations

Using Eqn. 6.1 we can approximate the solution over the entire domain, as long as the set of basis functions covers the domain. However, we still need equations for U_j , the unknown coefficients on each basis function. Let's look at two approaches for obtaining these equations.

6.2.1 Collocation

In collocation we obtain equations by requiring the PDE to hold at a fixed number of points.

Example 6.1 (Collocation for a 1D diffusion problem). Consider a diffusion PDE,

$$\begin{aligned} u_{xx} + 50e^x &= 0, \\ u(-1) = u(1) &= 0, \\ \Omega &= [-1, 1]. \end{aligned} \tag{6.2}$$

The exact solution is

$$u^{\text{exact}} = -50e^x + 50x \sinh 1 + 50 \cosh 1. \tag{6.3}$$

We approximate the solution to this PDE using two basis functions,

$$u(x) = U_1 \underbrace{(1 - x^2)}_{\phi_1(x)} + U_2 \underbrace{(x - x^3)}_{\phi_2(x)}. \tag{6.4}$$

Let's try the method of collocation using the following two points: $x_1 = -1/3$ and $x_2 = 1/3$. To substitute Eqn. 6.4 into Eqn. 6.2, we need to evaluate the second derivative, u_{xx} . This gives

$$u_{xx} = -2U_1 - 6xU_2.$$

Requiring the PDE to hold at the two collocation points gives two equations for U_1 and U_2 ,

$$\begin{aligned} -2U_1 - 6x_1U_2 + 50e^{x_1} &= 0 \\ -2U_1 - 6x_2U_2 + 50e^{x_2} &= 0 \end{aligned}$$

Solving this system gives $U_1 = 26.4018$, $U_2 = 8.4885$. Using these values in our approximation, we can now construct a plot of $u(x)$ and compare it to the exact solution, $u^{\text{exact}}(x)$, as shown in Figure 6.3.

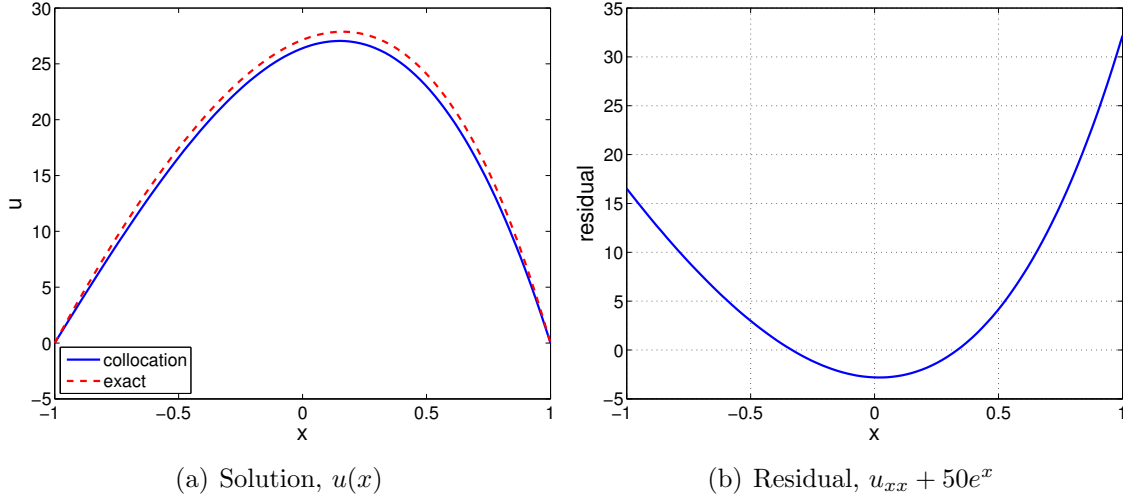


Figure 6.3: Comparison of exact solution to that computed by two-point collocation. Note that the zero intercepts of the residual plot occur at the collocation points.

Determining at which points to enforce the PDE in the collocation method is a somewhat arbitrary process. There are reasonable choices of points that yield poor results. In addition, the degree of polynomials used for the basis functions needs to be sufficiently high in order to be able to evaluate the PDE residual. For example, for the above diffusion equation we could not have used linear basis functions, since two derivatives of a linear function yield zero. For these reasons, the collocation method is generally not a very robust method for obtaining our equations.

6.2.2 Weighted Residuals

A more systematic approach for obtaining equations is the method of weighted residuals. Instead of enforcing the PDE at individual points, we instead integrate the PDE over the entire domain. However, doing simply this just gives us one equation. To obtain multiple unique equations, we multiply the PDE by linearly-independent weight (also called “test”) functions prior to integrating over the domain.

We need the same number of weight functions as we have unknowns. Rather than defining new functions for this purpose (which is viable, but requires a little more work), a simple approach is to use the basis functions from the approximation of u as weight functions. That is, the ϕ_j functions serve two purposes: approximation of the state and weight functions on the residual.

Example 6.2 (Weighted Residuals for a 1D diffusion problem). Consider the same diffusion PDE as in the previous example, Eqn. 6.2. We again approximate the solution to this PDE

using two basis functions,

$$u(x) = \underbrace{U_1(1-x^2)}_{\phi_1(x)} + \underbrace{U_2(x-x^3)}_{\phi_2(x)}.$$

However, we now obtain equations by weighting the PDE with the basis functions and integrating over the domain. So our two equations are

$$\int_{-1}^1 \phi_i(x) [u_{xx} + 50e^x] dx = 0,$$

where $i = 1, 2$. We can now integrate by parts to shift one derivative off u and onto the basis functions. The result is

$$-\int_{-1}^1 \frac{d\phi_i}{dx} u_x dx + \underbrace{[\phi_i u_x]_{-1}^1}_0 + \int_{-1}^1 \phi_i 50e^x dx = 0.$$

The boundary term from the integration by parts drops out because the functions ϕ_i are zero on the boundary. Substituting our approximation for $u_x = U_1 \frac{d\phi_1}{dx} + U_2 \frac{d\phi_2}{dx}$, we obtain

$$-\int_{-1}^1 \frac{d\phi_i}{dx} \left[U_1 \frac{d\phi_1}{dx} + U_2 \frac{d\phi_2}{dx} \right] dx + \int_{-1}^1 \phi_i 50e^x dx = 0.$$

Rearranging,

$$\begin{aligned} i = 1 : \quad & -\int_{-1}^1 \frac{d\phi_1}{dx} \frac{d\phi_1}{dx} dx U_1 - \int_{-1}^1 \frac{d\phi_1}{dx} \frac{d\phi_2}{dx} dx U_2 + \int_{-1}^1 \phi_1 50e^x dx = 0 \\ i = 2 : \quad & -\int_{-1}^1 \frac{d\phi_2}{dx} \frac{d\phi_1}{dx} dx U_1 - \int_{-1}^1 \frac{d\phi_2}{dx} \frac{d\phi_2}{dx} dx U_2 + \int_{-1}^1 \phi_2 50e^x dx = 0 \end{aligned}$$

Evaluating the integrals gives a system of two equations,

$$\begin{aligned} -\frac{8}{3}U_1 - 0U_2 + \frac{200}{e} &= 0 \\ -0U_1 - \frac{8}{5}U_2 + 100e - \frac{700}{e} &= 0 \end{aligned}$$

The solution to this system is $U_1 = 27.5910, U_2 = 8.9454$. Figure 6.4 compares this solution to the exact one.

Note the integration by parts in the above example, which shifts derivatives from the approximation functions to the weight/test functions. This allows us to use basis functions of lower degree than that suggested by the differential operator. For example, we will soon use linear basis functions for elliptic differential equations.

In the method of weighted residuals, the equations fall out automatically once we define a basis for approximating the solution. There is also a sounder theoretical framework supporting integrating over the entire domain compared to evaluating the PDE at just a few collocation points. For this reason, the most common finite element methods are based on the method of weighted residuals.

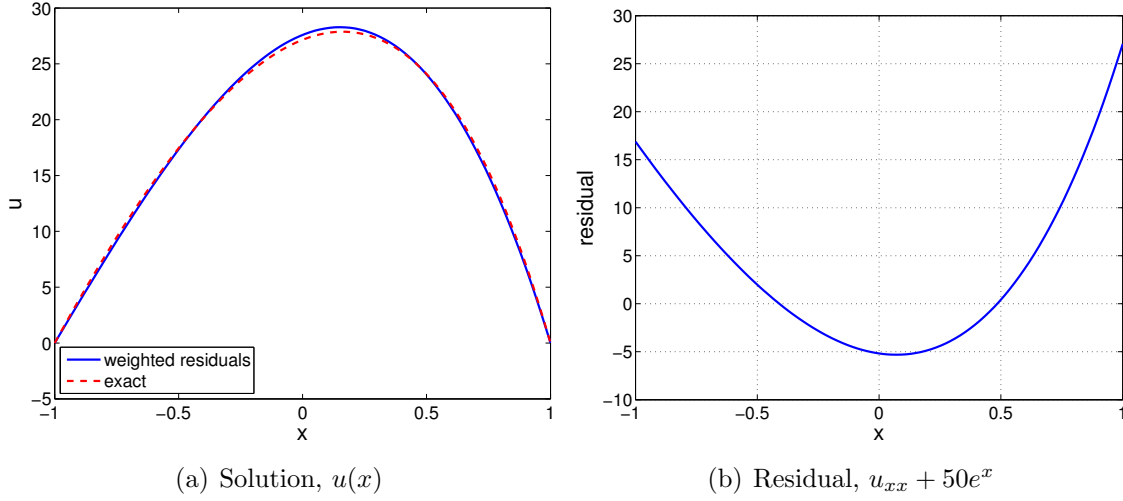


Figure 6.4: Comparison of exact solution to that computed by the method of weighted residuals.

6.3 Numerical Integration

In the method of weighted residuals, we have to integrate basis functions multiplying the PDE over portions of the domain. For simple problems, we can do the integrals analytically. However, for more complex problems, such as nonlinear PDEs in multiple dimensions, analytical integration is not practical. Instead, we can use *numerical integration*.

We will consider using weighted sums of point values of a function to approximate the integral of that function. We will restrict our attention to one spatial dimension at first, on a fixed domain, $[-1, 1]$. Other intervals can be mapped to this domain, so this choice is not restrictive. Specifically, we seek one set of N_q points, x_q , and weights, w_q , such that for *arbitrary* polynomials $f(x)$ of order p ,

$$\int_{-1}^1 f(x) dx = \sum_{q=1}^{N_q} w_q f(x_q).$$

Note that while for general functions $f(x)$ the numerical integration will be approximate, we require that for polynomial $f(x)$ (up to a certain order) the numerical integration is *exact*.

One approach to numerical integration is as follows. First, sample $f(x)$ at $N_q = p + 1$ arbitrary distinct points x_q . Second, fit an order p polynomial and integrate this polynomial using tabulated values for the integrals of, say, each of the monomials. Third, express the result as a weighted linear combination of $f(x_q)$; the weights are the w_q that we seek.

The problem with this simple approach is that with a careless choice of x_q , some of the weights may turn out to be very large and/or negative, creating conditioning and numerical precision problems. We can do much better in terms of conditioning and accuracy by choosing x_q judiciously.

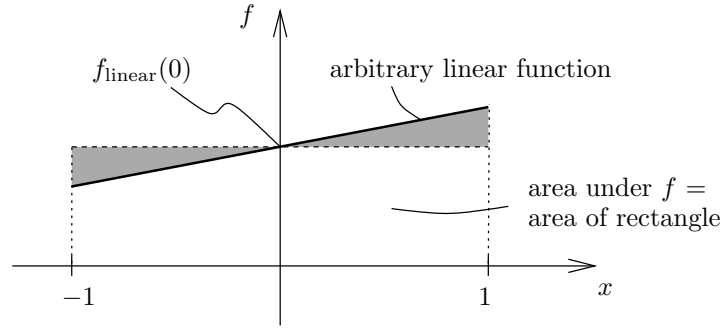
A key observation is that the process of integration turns a function of x into a scalar (the integral). This loss of information suggests that less information about $f(x)$ is needed to compute the integral than to fully describe the function.

Example 6.3 (Numerical integration of a linear function). Suppose we want to integrate an arbitrary linear function on $x \in [-1, 1]$. We only need one point, which is at the midpoint of the interval – for us at $x = 0$.

$$\int_{-1}^1 f_{\text{linear}}(x) dx = 2f_{\text{linear}}(0)$$

$$\Rightarrow N_q = 1, x_1 = 0, w_1 = 2$$

The graphical proof of this integration rule is illustrated below. Note that we would need two points to fully determine the function f_{linear} . The fact that we can get away with one point for integration is that we do not need to know the function exactly in order to compute its integral. In addition, in numerical integration we can tune both the weights *and* the point locations.



Consider an arbitrary order p polynomial,

$$f(x) = c_0 + c_1x + \dots + c_{p-1}x^{p-1} + c_px^p. \quad (6.5)$$

Integrating this polynomial from -1 to 1 we obtain,

$$I \equiv \int_{-1}^1 f(x) dx = 2 \left(c_0 + \frac{c_2}{3} + \frac{c_4}{5} + \dots \right). \quad (6.6)$$

Note that the odd powers of x integrate to zero, leaving $\lfloor p/2 \rfloor + 1$ terms, where $\lfloor a \rfloor$ is the greatest integer less than or equal to a . So to compute I we just need $N_q = \lfloor p/2 \rfloor + 1$ pieces of information (e.g. values of f). Put another way, N_q points will let us integrate polynomial orders up to $p = 2N_q - 1$.

Example 6.4 (Two-point integration). Suppose we choose to use $N_q = 2$ integration points. We should be able to integrate polynomials up to $p = 3$ (cubics) exactly. For the case of $p = 3$, the integral in Eqn. 6.6 is

$$\begin{aligned} I = 2 \left(c_0 + \frac{c_2}{3} \right) &= w_1 f(x_1) + w_2 f(x_2) \\ &= w_1 (c_0 + c_1 x_1 + c_2 x_1^2 + c_3 x_1^3) + w_2 (c_0 + c_1 x_2 + c_2 x_2^2 + c_3 x_2^3). \end{aligned}$$

Matching coefficients on c_i , we have

$$\begin{aligned} c_0 : \quad 2 &= w_1 + w_2 \\ c_1 : \quad 0 &= w_1 x_1 + w_2 x_2 \\ c_2 : \quad \frac{2}{3} &= w_1 x_1^2 + w_2 x_2^2 \\ c_3 : \quad 0 &= w_1 x_1^3 + w_2 x_2^3 \end{aligned}$$

The following values of w_q, x_q satisfy these equations

$$\begin{aligned} w_1 &= 1, & w_2 &= 1 \\ x_1 &= \frac{-1}{\sqrt{3}}, & x_2 &= \frac{1}{\sqrt{3}} \end{aligned}$$

Note that computing the weights and points for numerical integration requires the solution of a nonlinear system of equations, and this gets harder for large N_q . For example, a Newton-Raphson approach is not guaranteed to converge, especially for poor initial guesses of the point locations. Moreover, the system becomes increasingly ill-conditioned for high N_q as columns start to appear linearly dependent – this is due to the fact that monomials are not a good basis. It turns out that in one dimension there is a clever choice of polynomial basis functions that circumvents these difficulties. The result is a set of points called the *Gauss-Legendre* points, where the quadrature point locations are the roots of Legendre polynomials, and the weights come from solving a well-conditioned linear system.

For domains in multiple dimensions, integration rules are generally harder to derive. This is not the case for squares and cubes, where tensor products of the one-dimensional rules can be used. However, for other shapes, such as triangles or tetrahedra, the process is more painstaking and once found integration rules are usually tabulated for some prototypical reference shape (e.g. a unit right triangle).

6.4 The Finite Element Method in One Dimension

In this section we present the finite element method for the one-dimensional diffusion equation,

$$(\nu u_x)_x + f(x) = 0,$$

where ν is the diffusivity and $f(x)$ is a source term. This equation can be used to model the conduction of heat or the diffusion of a chemical species, and it is a prototypical form for more complicated diffusion phenomena. Note, in some cases the diffusivity ν may not be constant. We will allow it to depend on the position, x , but not on the state, u , as the latter case would result in a nonlinear system.

The finite-element method is, at its core, the method of weighted residuals presented in Section 6.2.2. It is in fact a specialized version of this method, in that the basis functions are judiciously chosen to enable a systematic and local construction of the system of equations. Let's see how this works for the problem at hand.

6.4.1 Mesh and Basis Functions

Suppose our domain of interest is $x \in [0, L]$. We divide this domain into N_e intervals that are not necessarily equally-spaced. The resulting mesh is shown in Figure 6.5. On this mesh

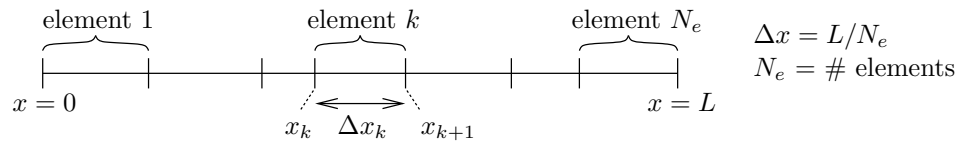


Figure 6.5: Mesh of N_e elements for a one-dimensional finite-element discretization.

we construct a set of $N = N_e + 1$ basis functions to support a piecewise linear approximation of our solution. Each basis function is a “hat” shape that is one at a single node and drops linearly to zero at the adjacent neighbor nodes. Figure 6.6 illustrates these basis functions on our mesh. Note that on the two boundaries we have basis functions that look like half of the interior basis functions.

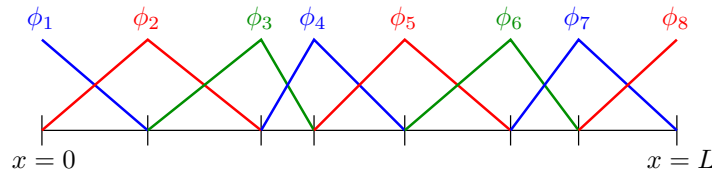


Figure 6.6: $N_e + 1$ linear basis functions on a mesh of N_e elements for a one-dimensional finite-element discretization.

6.4.2 The Reference Element

In the finite element we need to define basis functions over elements that could be of different sizes. We also need to integrate over these elements when computing the weighted residuals. Both of these tasks are simplified by the use of a *reference element*.

In one dimension, as a reference element we use the interval $[-1, 1]$. We denote by ξ the reference-space coordinate in this interval. Now, for every element in the mesh, we consider a mapping to this reference element, as illustrated in Figure 6.7. Note that the indexing on

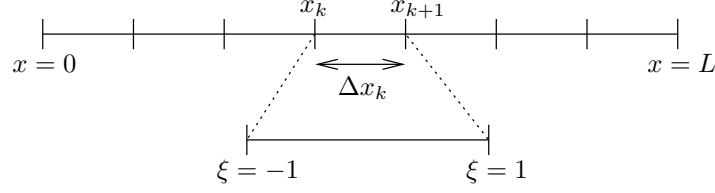


Figure 6.7: Mapping of element k to a reference element, $\xi \in [-1, 1]$.

the nodes and elements is such that $x_{k+1} = x_k + \Delta x_k$. The mapping from ξ to x is

$$x = x_k + \frac{\xi + 1}{2} \Delta x_k. \quad (6.7)$$

The reverse mapping, from x to ξ is

$$\xi = 2 \frac{x - x_k}{\Delta x_k} - 1. \quad (6.8)$$

6.4.3 The Weak Form

To obtain equations, we use the method of weighted residuals. That is, we multiply the PDE by test functions and integrate by parts over the elements. We will use a *Galerkin* finite element method, in which the test functions are the same as the approximation basis functions.

Consider test function ϕ_i , where $i \in [1 \dots N_e + 1]$ is an index over the nodes. When we multiply the PDE by these test functions, the integrand is only nonzero over the two elements adjacent to node i . This is due to the local support of each finite element test function. So we only need to integrate over the support of ϕ_i – i.e. element $i - 1$ (between x_{i-1} and x_i) and element i (between x_i and x_{i+1}).

$$\begin{aligned} \int_{\Omega} \phi_i [(\nu u_x)_x + f(x)] dx &= 0 \\ \Rightarrow \int_{x_{i-1}}^{x_{i+1}} \phi_i [(\nu u_x)_x + f(x)] dx &= 0 \\ \Rightarrow \int_{x_{i-1}}^{x_{i+1}} \phi_i (\nu u_x)_x dx + \int_{x_{i-1}}^{x_{i+1}} \phi_i f(x) dx &= 0 \\ \Rightarrow - \int_{x_{i-1}}^{x_{i+1}} \phi_{i,x} \nu u_x dx + [\phi_i \nu u_x]_{x_{i-1}}^{x_{i+1}} + \int_{x_{i-1}}^{x_{i+1}} \phi_i f(x) dx &= 0 \end{aligned} \quad (6.9)$$

We have assumed that we are dealing with an interior basis function, i.e. one that is not on the boundary. The treatment of boundary basis functions is discussed in the next subsection.

Now, for every interior basis function, the middle term in the above expression vanishes because $\phi_i = 0$ at x_{i-1} and at x_{i+1} . So the equation associated with every interior node is

$$-\int_{x_{i-1}}^{x_{i+1}} \phi_{i,x} \nu u_x dx + \int_{x_{i-1}}^{x_{i+1}} \phi_i f(x) dx = 0. \quad (6.10)$$

We call this a *weak* enforcement of the PDE since we only require the PDE to hold in an integral sense.

6.4.4 Boundary Conditions

The weak form requires special treatment at domain boundaries. At the boundaries, extra information is generally available in the form of boundary conditions, and this information must be incorporated into the finite element method. We will see how this happens for three of the most common types of boundary conditions.

1. *Dirichlet*: the value of u is prescribed on the boundary.
2. *Neumann*: the flux (gradient) of u , specifically νu_x , is prescribed on the boundary. For example, in a heat-transfer problem, an adiabatic wall would require $\nu u_x = 0$.
3. *Robin*: a linear combination of u and the flux νu_x is prescribed on the boundary. In a heat-transfer problem, this could be a model for a convective boundary.

We present each of these conditions applied to the left boundary of our domain. The relevant equation is the weighted residual associated with ϕ_1 , which, starting from Eqn. 6.9, is

$$-\int_{x_1}^{x_2} \phi_{1,x} \nu u_x dx + [\phi_1 \nu u_x]_{x_1}^{x_2} + \int_{x_1}^{x_2} \phi_1 f(x) dx = 0. \quad (6.11)$$

Let's see how this equation looks for each boundary condition.

6.4.4.1 Dirichlet Boundary Condition

In the Dirichlet case, we know the value of u on the boundary. Let's call it $u_{\text{Dirichlet}}$. For our case, this means we know u at the left boundary, i.e. u_1 . This value is the coefficient on the first basis function, i.e. U_1 which multiplies ϕ_1 , in our nodal Lagrange basis. So we do not need the boundary weak form in Eqn. 6.11, and instead our equation is

$$U_1 = u_{\text{Dirichlet}}. \quad (6.12)$$

Another strategy is to simply eliminate the unknown U_1 from the system by replacing all occurrences of U_1 with $u_{\text{Dirichlet}}$ in the other equations.

6.4.4.2 Neumann Boundary Condition

In the Neumann case, we know the value of νu_x on the boundary. Let's call this q_{Neumann} . Eqn. 6.11 then becomes

$$-\int_{x_1}^{x_2} \phi_{1,x} \nu u_x dx - \phi_1(x_1) q_{\text{Neumann}} + \int_{x_1}^{x_2} \phi_1 f(x) dx = 0 \quad (6.13)$$

Note that while $\phi_1(x_2) = 0$, $\phi_1(x_1) \neq 0$, so the term containing the Neumann flux boundary condition does not vanish. It is only through this boundary term, which arose from our integration by parts, that the Neumann boundary condition is incorporated. This is another advantage of applying integration by parts to the weighted residuals.

6.4.4.3 Robin Boundary Condition

Finally, in Robin boundary conditions, the value of $q_{\text{Robin}} \equiv au + b\nu u_x$ is known on the boundary, where a and b are some constants. We can solve for νu_x in terms of the known quantity and u ,

$$\nu u_x \Big|_{x_1} = \frac{1}{b} (q_{\text{Robin}} - au(x_1)).$$

As in the Neumann case, we substitute this value of νu_x into Eqn. 6.11, obtaining

$$-\int_{x_1}^{x_2} \phi_{1,x} \nu u_x dx - \phi_1(x_1) \frac{1}{b} (q_{\text{Robin}} - au(x_1)) + \int_{x_1}^{x_2} \phi_1 f(x) dx = 0 \quad (6.14)$$

This is the equation for the weighted residual associated with the first basis function. Note that in the Robin boundary condition, the boundary flux term that came from the integration by parts now brings in a dependence on the state. This dependence must be taken into account when assembling the stiffness matrix.

6.4.5 Matrix Assembly and Solution

6.4.5.1 The Stiffness Matrix, A

In the finite element method, each basis function brings both an unknown (the coefficient in the approximation of $u(x)$) and an equation (the weighted residual). We will consider only linear PDEs, for which the resulting system of equations will also be linear. Let's take a closer look at how the weak form equations become a matrix system.

We start with the equations for every interior basis function, Eqn 6.10 (repeated here),

$$-\int_{x_{i-1}}^{x_{i+1}} \phi_{i,x} \nu u_x dx + \int_{x_{i-1}}^{x_{i+1}} \phi_i f(x) dx = 0.$$

Into this equation we substitute the expansion for $u(x)$ from Eqn. 6.1 (repeated here),

$$u(x) \approx \sum_{j=1}^N U_j \phi_j(x).$$

Recall that N is the number of basis functions, which is $N_e + 1$ for our 1D mesh of N_e elements. The resulting equation is

$$\begin{aligned}
& - \int_{x_{i-1}}^{x_{i+1}} \phi_{i,x} \nu \left(\sum_{j=1}^N U_j \phi_j(x) \right) dx + \int_{x_{i-1}}^{x_{i+1}} \phi_i f(x) dx = 0 \\
& \sum_{j=1}^N \underbrace{\left[- \int_{x_{i-1}}^{x_{i+1}} \phi_{i,x} \nu \phi_{j,x} dx \right]}_{A_{ij}} U_j + \underbrace{\int_{x_{i-1}}^{x_{i+1}} \phi_i f(x) dx}_{F_i} = 0
\end{aligned} \tag{6.15}$$

This is now just an $N \times N$ matrix system,

$$\mathbf{A}\mathbf{U} + \mathbf{F} = 0,$$

where the entries of the *stiffness matrix* $\mathbf{A}$ are A_{ij} , the entries of the solution vector $\mathbf{U}$ are U_j , and the entries of the (LHS) forcing vector $\mathbf{F}$ are F_i . The matrix $\mathbf{A}$ is *sparse* (mostly zeros), because the basis functions have local support. That is, A_{ij} will only be nonzero if the nonzero values of ϕ_i and ϕ_j overlap – which does not happen often since each basis function is nonzero over only two elements.

6.4.5.2 Stiffness Matrix Assembly

We can build the matrix $\mathbf{A}$ using a systematic approach by looping over the elements. From Eqn. 6.15, we have that (for i not a boundary node), the matrix components are

$$A_{ij} = - \int_{x_{i-1}}^{x_{i+1}} \phi_{i,x} \nu \phi_{j,x} dx. \tag{6.16}$$

Building the $\mathbf{A}$ matrix one row at a time is a bit cumbersome because each basis function is piecewise linear, so that the slope ($\phi_{i,x}$) changes over the elements. This creates even more cumbersome bookkeeping in two and three dimensions. An alternative approach is to initialize $\mathbf{A} = 0$ and then to build the matrix up with contributions to $\mathbf{A}$ from each element.

Let's pick one element, $k \in [1 \dots N_e]$. There are two nonzero basis functions on this element, ϕ_k and ϕ_{k+1} , as illustrated in Figure 6.8. So according to Eqn. 6.16, element k

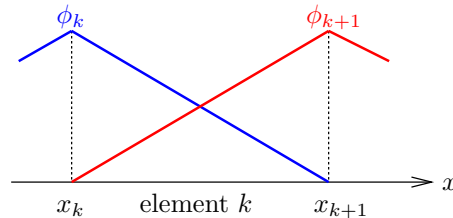


Figure 6.8: Nonzero basis functions on element k .

contributes to four entries in the calculation of $\mathbf{A}$. These four contributions are

$$\begin{aligned} A_{k,k} &= A_{k,k} - \int_{x_k}^{x_{k+1}} \phi_{k,x} \nu \phi_{k,x} dx & A_{k,k+1} &= A_{k,k+1} - \int_{x_k}^{x_{k+1}} \phi_{k,x} \nu \phi_{k+1,x} dx \\ A_{k+1,k} &= A_{k+1,k} - \int_{x_k}^{x_{k+1}} \phi_{k+1,x} \nu \phi_{k,x} dx & A_{k+1,k+1} &= A_{k+1,k+1} - \int_{x_k}^{x_{k+1}} \phi_{k+1,x} \nu \phi_{k+1,x} dx \end{aligned}$$

Note that we say these are *contributions* to, not necessarily the actual entries in, $\mathbf{A}$. This is because on element k we are only looking at a portion of the basis functions ϕ_k and ϕ_{k+1} – both of these will generally be nonzero on other elements (e.g. $k-1$ or $k+1$). Additional contributions will come from integrating over these other elements.

Looping over the elements and adding to $\mathbf{A}$ in the manner described above is called (stiffness) *matrix assembly*. In fact, for each element k we can create a 2×2 matrix $\mathbf{A}^k$ that is the contribution of element k to the total stiffness matrix $\mathbf{A}$. Then by looping over the elements we “tile” the total stiffness matrix as illustrated in Figure 6.9.

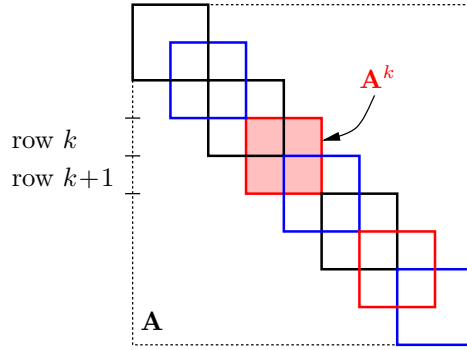


Figure 6.9: Assembly of matrix $\mathbf{A}$ with elemental contributions $\mathbf{A}^k$. Note the overlap of the elemental contributions, which is due to the fact that each interior basis function spans two elements.

6.4.5.3 Incorporating Boundary Conditions into $\mathbf{A}$ and $\mathbf{F}$

When using Dirichlet boundary conditions, we need to enforce Eqn. 6.12. So we replace the first row of the stiffness matrix by a single 1 on the main diagonal, and we replace the first entry in the forcing vector $\mathbf{F}$ by $-u_{\text{Dirichlet}}$.

When using Neumann boundary conditions, the stiffness matrix does not change. Looking at Eqn. 6.13, the only difference in this equation compared to that of an interior node is the presence of the constant $-\phi_1(x_1)q_{\text{Neumann}}$ term, which we transfer over to the forcing vector.

Finally, when using Robin boundary conditions, both the stiffness matrix and the forcing vector change. Looking at Eqn. 6.14, we see that the first extra term, $-\phi_1(x_1)\frac{1}{b}q_{\text{Robin}}$, is a constant that can be incorporated into the forcing vector. The second additional term, $\phi_1(x_1)\frac{a}{b}u(x_1) = \phi_1(x_1)\frac{a}{b}U_1$ affects the (1,1) entry of the stiffness matrix – specifically, we need to add the quantity $\phi_1(x_1)\frac{a}{b}$ to this entry in $\mathbf{A}$.

6.4.5.4 Reference Element Calculations

The entries in the stiffness matrix and forcing vector, e.g. in Eqn. 6.15, require integrals over the elements. In the case of the stiffness matrix, the derivatives of the basis functions are constants, and so we can evaluate these analytically. However, the forcing vector requires an integral of the product of a basis function and some arbitrary $f(x)$. This is most easily done using numerical integration, after mapping to a reference element. Specifically, the map from element k with $x \in [x_k, x_{k+1}]$ to a reference element with $\xi \in [-1, 1]$ is given in Eqn. 6.7 and Eqn. 6.8. So we can write, for example,

$$\begin{aligned} \int_{x_k}^{x_{k+1}} \phi_k(x) f(x) dx &= \int_{-1}^1 \phi_L(\xi) f(x(\xi)) \frac{\Delta x_k}{2} d\xi \\ \int_{x_k}^{x_{k+1}} \phi_{k+1}(x) f(x) dx &= \int_{-1}^1 \phi_R(\xi) f(x(\xi)) \frac{\Delta x_k}{2} d\xi \end{aligned}$$

where $\phi_L(x) = (1 - \xi)/2$ and $\phi_R(x) = (1 + \xi)/2$ are the two basis functions on the reference element. Note, $\frac{\Delta x_k}{2}$ is the Jacobian of the mapping – i.e. $dx = \frac{\Delta x_k}{2} d\xi$. We can now evaluate this integral using a numerical quadrature rule with a number of points N_q that is appropriate for the desired accuracy.

The reference element mapping is also useful for the analytical stiffness matrix calculation. For example, let's look at the 2×2 contribution associated with element k , $\mathbf{A}^k$. The first entry in this matrix is

$$\begin{aligned} - \int_{x_k}^{x_{k+1}} \phi_{k,x} \nu \phi_{k,x} dx &= - \int_{-1}^1 \underbrace{\frac{2}{\Delta x_k} \phi_{L,\xi}}_{\phi_{k,x}} \nu \underbrace{\frac{2}{\Delta x_k} \phi_{L,\xi}}_{\phi_{k,x}} \frac{\Delta x_k}{2} d\xi \\ &= - \frac{2\nu}{\Delta x_k} \int_{-1}^1 \phi_{L,\xi} \phi_{L,\xi} d\xi \end{aligned} \quad (6.17)$$

We used the chain rule to transform basis function derivatives with respect to x into derivatives with respect to ξ . We can now evaluate the integral, since $\phi_{L,\xi} = -1/2$. Doing the same for the other three entries of $\mathbf{A}^k$ gives

$$\mathbf{A}^k = \frac{\nu}{\Delta x_k} \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix}$$

6.4.5.5 Putting it all Together

Let's now look at an implementation of the finite element method for a one-dimensional heat transfer problem.

Example 6.5 (Steady Heat Conduction in a Bar with a Source). The governing equation for the temperature, denoted by $T(x)$, in a one-dimensional rod is

$$\frac{d}{dx} \left(\kappa \frac{dT}{dx} \right) + f(x) = 0,$$

where κ is the thermal conductivity, and $f(x)$ is the (negative of the) heat source. We consider a domain of $x \in [0, 1]$, a source term of the form $f(x) = \sin(\pi x)$, $\kappa = 1$, and the following boundary conditions:

$$\begin{aligned} \text{A Dirichlet BC at } x = 0: \quad & T(0) = 1, \\ \text{A Neumann BC at } x = 1: \quad & \kappa \frac{dT}{dx} = 0. \end{aligned}$$

Listing 6.1 presents a code that implements the finite element discretization for this problem, using a uniform grid of elements of size $\Delta x = 1/N_e$.

Listing 6.1: Computes the temperature in a heated 1D bar using the FEM

```

1 function [x,T] = heat1d(TL, kappa, Ne);
2 % function [x,T] = heat1d(TL, kappa, Ne);
3 % PURPOSE:
4 %   Solves a 1D heat transfer problem using FEM.
5 % INPUTS:
6 %   TL    : temperature at the left end of the bar
7 %           the right of the bar is adiabatic
8 %   kappa : heat conductivity inside the bar
9 %   Ne    : number of elements
10 % OUTPUTS
11 %   x     : Ne+1 by 1 vector of node locations
12 %   T     : Ne+1 by 1 vector of temperatures at each node
13
14 dx = 1/Ne;           % element length (all the same)
15 x = linspace(0,1,Ne+1); % node positions
16
17 N = Ne+1; % # unknowns = # nodes
18 A = sparse(N,N); % stiffness matrix
19 F = zeros(N,1); % forcing vector (on LHS)
20
21 % elemental stiffness matrix
22 Ak = kappa/dx * [-1 1; 1 -1];
23
24 % numerical integration (quadrature) rule
25 xiq = [-1;1]/sqrt(3); wq = [1, 1];
26
27 % Ref element basis functions at quadrature points
28 Phi1q = [0.5*(1-xiq), 0.5*(1+xiq)];
29
30 % set up system by looping over elements
31 for k = 1:Ne,
32
33     Ik = [k, k+1]; % unknowns associated with this element
34
35     A(Ik, Ik) = A(Ik, Ik) + Ak; % add to stiffness matrix
36
37     xq = (k-1)*dx + 0.5*(xiq + 1)*dx; % quad points in global space
38
39     F(Ik) = F(Ik) + Phi1q'*diag(wq)*sin(pi*xq)*dx/2; % numerical integration
40
41 end
42
43 % boundary conditions
44 A(1,:) = zeros(1,N); A(1,1) = 1; F(1) = -TL; % Dirichlet on left
45 % adiabatic on right -> nothing to do for Neumann
46
47 % solve system
48 T = -A\F;
```

Running the code with $N_e = 8$ elements produces the plot shown in Figure 6.10 on the left. Note how both boundary conditions appear satisfied. A more quantitative comparison is shown in the error convergence plot in Figure 6.10, where the L_2 error is computed against the exact analytical solution. Note the second-order rate of convergence of the error. We can reason this from the fact that we are using linear polynomials to approximate our solution: the leading error term will be quadratic in Δx .

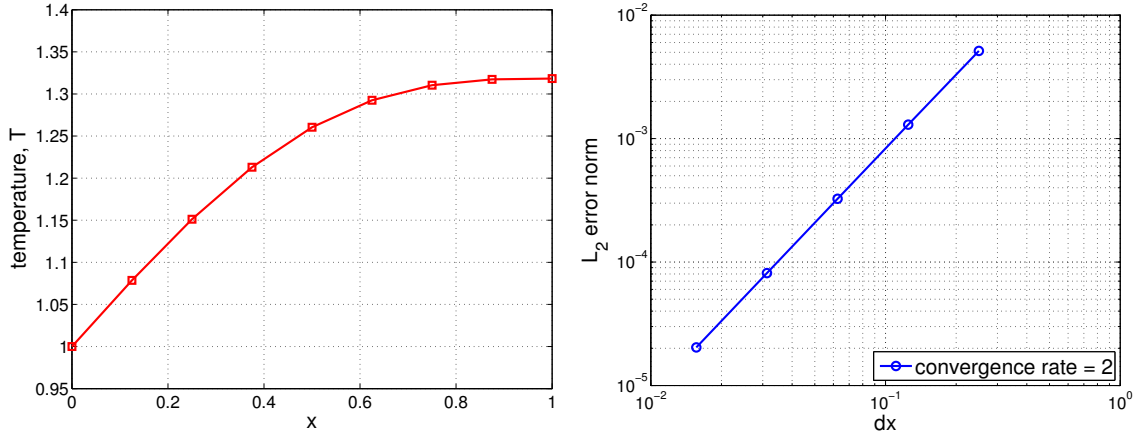


Figure 6.10: Sample finite element solution and convergence of the discrete L_2 error norm of the computed temperature distribution relative to the actual temperature distribution for Example 6.5.

6.5 The Finite Element Method in Two Dimensions

We now present the finite element method for the two-dimensional diffusion equation,

$$\nabla \cdot (\nu \nabla u) + f(x, y) = 0. \quad (6.18)$$

Again, ν is a diffusivity and f is a source term. Note that the diffusive flux, $\nu \nabla u$, is now a vector.

6.5.1 Mesh and Basis Functions

We discretize Eqn. 6.18 on a domain broken down into non-overlapping triangles. The set of these triangles constitutes a mesh. Each of the triangles is an element, and the nodes are the mesh vertices.

On the mesh of triangles, we approximate the state using piecewise linear “hat” basis functions, as illustrated in Figure 6.2(b). Note that each basis function is one at a single node and is nonzero only on the elements adjacent to that node. As in the one-dimensional case, we will associate basis functions with all nodes of the mesh, be they interior or boundary.

6.5.2 The Reference Element

To simplify calculations we will work with a reference element, which in two dimensions will be a unit right triangle, as illustrated in Figure 6.11. We denote the reference triangle by T , and in the figure we show a mapping between T and an arbitrary element Ω_k . The mapping

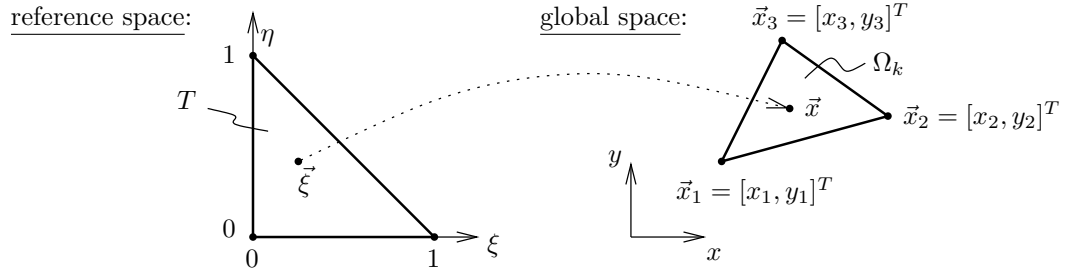


Figure 6.11: Mapping from the reference element, with coordinates $\vec{\xi} = (\xi, \eta)$, to an arbitrarily-shaped and oriented reference element with coordinates $\vec{x} = (x, y)$.

from reference space to global space, $\vec{x}(\vec{\xi})$, is a linear function that interpolates the three global nodes that define the triangle, $\vec{x}_1, \vec{x}_2, \vec{x}_3$. Using the interpolation requirement, we can construct this function as

$$\vec{x}(\vec{\xi}) = \vec{x}_1 + (\vec{x}_2 - \vec{x}_1)\xi + (\vec{x}_3 - \vec{x}_1)\eta,$$

where (ξ, η) are the two components of $\vec{\xi}$. Inspecting this mapping we see that it is linear and that the three nodes of the reference triangle map to the three nodes $\vec{x}_1, \vec{x}_2, \vec{x}_3$. In matrix form, we can write

$$\vec{x} = \vec{x}_1 + \underline{J}\vec{\xi},$$

where $\underline{J}$ is the *mapping Jacobian* matrix given by

$$\underline{J} = \frac{\partial \vec{x}}{\partial \vec{\xi}} = \begin{bmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{bmatrix}.$$

The inverse mapping is then $\vec{\xi} = \underline{J}^{-1}(\vec{x} - \vec{x}_1)$, where the inverse mapping Jacobian is

$$\underline{J}^{-1} = \frac{\partial \vec{\xi}}{\partial \vec{x}} = \frac{1}{J} \begin{bmatrix} y_3 - y_1 & x_1 - x_3 \\ y_1 - y_2 & x_2 - x_1 \end{bmatrix}, \quad J \equiv \det(\underline{J}).$$

When transforming integrals from global space to reference space, we must include the determinant of the mapping, J , inside the reference-space integrand,

$$\int_{\Omega_k} f(\vec{x}) d\Omega = \int_T f(\vec{x}(\vec{\xi})) J dT = J \int_T f(\vec{x}(\vec{\xi})) dT.$$

Finally, we can use the chain rule when changing variables during differentiation.

$$\left. \begin{aligned} \frac{\partial f}{\partial x} &= \frac{\partial f}{\partial \xi} \frac{\partial \xi}{\partial x} + \frac{\partial f}{\partial \eta} \frac{\partial \eta}{\partial x} \\ \frac{\partial f}{\partial y} &= \frac{\partial f}{\partial \xi} \frac{\partial \xi}{\partial y} + \frac{\partial f}{\partial \eta} \frac{\partial \eta}{\partial y} \end{aligned} \right\} \frac{\partial f}{\partial \vec{x}} = \frac{\partial f}{\partial \vec{\xi}} \frac{\partial \vec{\xi}}{\partial \vec{x}} = \frac{\partial f}{\partial \vec{\xi}} \underline{J}^{-1} = [\cdot \cdot] \begin{bmatrix} \cdot & \cdot \\ \cdot & \cdot \end{bmatrix}$$

Note that gradients such as $\frac{\partial f}{\partial \vec{\xi}}$ are treated as row vectors.

6.5.3 The Weak Form

As in the one-dimensional case, we use the method of weighted residuals to obtain our equations. This means we multiply the PDE in Eqn. 6.18 by test functions that are the same as the basis functions, and then we integrate by parts over the entire domain, Ω . The result is

$$\begin{aligned} \int_{\Omega} \phi_i [\nabla \cdot (\nu \nabla u) + f(x, y)] d\Omega &= 0 \\ \int_{\Omega} \phi_i \nabla \cdot (\nu \nabla u) d\Omega + \int_{\Omega} \phi_i f(x, y) d\Omega &= 0 \\ \underbrace{- \int_{\Omega} \nabla \phi_i \cdot (\nu \nabla u) d\Omega + \int_{\partial\Omega} \phi_i (\nu \nabla u) \cdot \vec{n} dl}_{\text{result of integrating by parts}} + \int_{\Omega} \phi_i f(x, y) d\Omega &= 0 \end{aligned} \quad (6.19)$$

Since each basis/test function has local support, once we substitute an expansion for u , we expect to obtain a sparse system, as in the one-dimensional case.

6.5.4 Boundary Conditions

The boundary condition treatment in two dimensions is very similar to that in one dimension. First, for nodes on Dirichlet boundaries, we replace the weak form corresponding to the test function on that node with the equation,

$$U_1 = u_{\text{Dirichlet}}.$$

On Neumann boundaries, we prescribe the normal flux $q_{\text{Neumann}} = (\nu \nabla u) \cdot \vec{n}$. Looking at the weak form in Eqn. 6.19, the contribution to the equation for test function ϕ_i is

$$\int_{\partial\Omega} \phi_i q_{\text{Neumann}} dl.$$

This contribution is only nonzero for those test functions that are nonzero on the Neumann boundaries. Note that we still have unknowns associated with the nodes on these boundaries.

Finally, for Robin boundary conditions, we can again solve for the normal flux $(\nu \nabla u) \cdot \vec{n}$ in terms of the prescribed data and the state u on the boundary. Using the resulting expression in the weak form brings in a dependence on the state that must be taken into account in the formation of the stiffness matrix.

6.5.5 Matrix Assembly

We obtain the stiffness matrix $\mathbf{A}$ by substituting the expansion for $u(x, y)$,

$$u(x, y) = \sum_{j=1}^N U_j \phi_j(x, y).$$

into the weak form in Eqn. 6.19. Note that the sum is over N , the total number of nodes. In the following we assume that there are no Robin boundary conditions, so that the only dependence on the state comes from the domain-interior integrals in Eqn. 6.19. Incorporating Robin boundary conditions is a straightforward extension. Since

$$\nabla u = \sum_{j=1}^N U_j \nabla \phi_j,$$

the stiffness matrix is calculated as follows:

$$-\int_{\Omega} \nabla \phi_i \cdot (\nu \nabla u) d\Omega = \sum_{j=1}^N \underbrace{\left[-\int_{\Omega} \nu \nabla \phi_i \cdot \nabla \phi_j d\Omega \right]}_{A_{ij}} U_j.$$

The resulting system is

$$\mathbf{A}\mathbf{U} + \mathbf{F} = 0,$$

where the forcing vector $\mathbf{F}$ contains contributions from the source term in Eqn. 6.19 and from boundary condition data.

As in 1D, we can form $\mathbf{A}$ and $\mathbf{F}$ efficiently by first zeroing these vectors out and then looping over the elements and adding each element's contribution to these quantities. To see how this works, let's pick one element, k . Denote by $i, j \in [1, 2, 3]$ the *local* indices of the nodes that make up the triangular element Ω_k . Denote by $g(j) \in [1 \dots N]$ the corresponding *global* indices of the local nodes. Then, the contribution of element k to the global stiffness matrix is,

$$A_{g(i),g(j)} = A_{g(i),g(j)} + A_{ij}^k,$$

where A_{ij}^k are components of the local stiffness matrix, $\mathbf{A}^k$, for element k . Similarly, the contribution to the global forcing vector is

$$F_{g(i)} = F_{g(i)} + F_i^k,$$

where F_i^k are the components of the local forcing vector.

We can evaluate the local stiffness matrix by mapping the integral to the reference element. Using the equations in Section 6.5.2, we obtain

$$\begin{aligned}
A_{ij}^k &= \int_{\Omega_k} -\nu \nabla \phi_i \cdot \nabla \phi_j d\Omega \\
&= \int_{\Omega_k} -\nu (\nabla_{\xi} \phi_i \underline{J}^{-1}) \cdot (\nabla_{\xi} \phi_j \underline{J}^{-1}) d\Omega \\
&= \int_T -\nu (\nabla_{\xi} \phi_i \underline{J}^{-1}) \cdot (\nabla_{\xi} \phi_j \underline{J}^{-1}) J dT
\end{aligned}$$

In the above equations, ∇_{ξ} denotes the gradient with respect to the reference coordinates, i.e. $\nabla_{\xi} = \frac{\partial}{\partial \xi}$ as opposed to $\nabla = \frac{\partial}{\partial \mathbf{x}}$. Finally, on the reference triangle, the basis functions and gradients are

$$\begin{array}{lll}
\phi_1 &= 1 - \xi - \eta & \frac{\partial \phi_1}{\partial \xi} = -1 & \frac{\partial \phi_1}{\partial \eta} = -1 \\
\phi_2 &= \xi & \frac{\partial \phi_2}{\partial \xi} = 1 & \frac{\partial \phi_2}{\partial \eta} = 0 \\
\phi_3 &= \eta & \frac{\partial \phi_3}{\partial \xi} = 0 & \frac{\partial \phi_3}{\partial \eta} = 1
\end{array}$$

6.6 High-Order Finite Elements

So far we have presented the finite element for piecewise linear (“hat”) basis functions. This is indeed the most common form of the finite element method. However, higher-order versions also exist. The idea behind high-order elements is to use polynomials of a degree higher than 1 to approximate the solution inside each element – see for example the discussion in section on finite-dimensional spaces, Section 6.1.1. Fortunately, aside from this change in the form of basis functions, not much else changes in the finite element method. We show this for quadratic ($p = 2$) elements in one dimension.

We need a basis to approximate the solution with quadratic polynomials inside each element. The basis of monomials, $1, x, x^2$, would work, but the bookkeeping with monomials is harder when we go to enforce continuity of basis functions at the nodes. Instead, a better basis results from the nodal Lagrange functions, illustrated in Figure 6.12. The three functions that comprise the Lagrange basis on one element are all quadratics. Each function is equal to 1 on one “node” of the element, and zero on all other nodes. In the case of quadratic, $p = 2$, approximation, we define an additional node as the midpoint of the element.

We can calculate the local stiffness matrix using the same procedure as in the $p = 1$ case. By Eqn. 6.17, we have that that i, j component of the local stiffness matrix $\mathbf{A}^k$ is

$$A_{ij} = -\frac{2\nu}{\Delta x_k} \int_{-1}^1 \phi_{i,\xi} \phi_{j,\xi} d\xi,$$

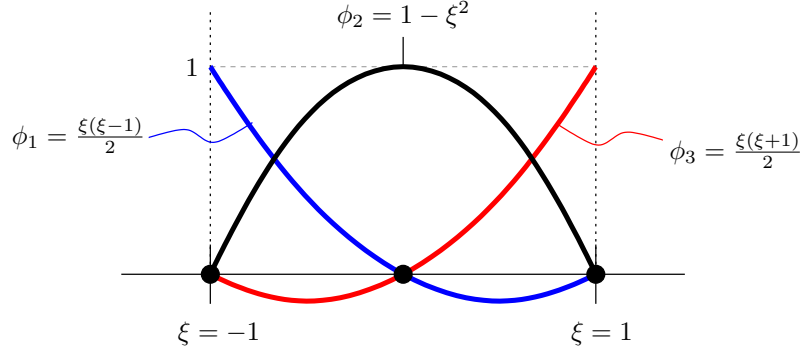


Figure 6.12: Basis functions for one-dimensional approximation using quadratic functions, defined on the reference element.

where $i, j \in [1, 2, 3]$. The derivatives of the three Lagrange basis functions in reference space are

$$\begin{aligned}\phi_{1,\xi} &= \xi - \frac{1}{2} \\ \phi_{2,\xi} &= -2\xi \\ \phi_{3,\xi} &= \xi + \frac{1}{2}\end{aligned}$$

Using these formulas, the local stiffness matrix becomes

$$\mathbf{A}^k = -\frac{2\nu}{\Delta x_k} \begin{bmatrix} \frac{7}{6} & -\frac{4}{3} & \frac{1}{6} \\ -\frac{4}{3} & \frac{8}{3} & -\frac{4}{3} \\ \frac{1}{6} & -\frac{4}{3} & \frac{7}{6} \end{bmatrix}$$

Example 6.6 (Steady Heat Conduction in a Bar with a Source using a $p = 2$ FEM). In this example we solve the same problem as in Example 6.5, but this time using quadratic elements. The governing equations and boundary conditions (repeated below) remain the same.

$$\frac{d}{dx} \left(\kappa \frac{dT}{dx} \right) + f(x) = 0, \quad x \in [0, 1], \quad f(x) = \sin(\pi x), \quad \kappa = 1$$

$$\begin{aligned}\text{A Dirichlet BC at } x = 0: & \quad T(0) = 1, \\ \text{A Neumann BC at } x = 1: & \quad \kappa \frac{dT}{dx} = 1.\end{aligned}$$

Listing 6.2 presents a code that implements the finite element discretization for this problem, using a uniform grid of elements of size $\Delta x = 1/N_e$.

Listing 6.2: Computes the temperature in a heated 1D bar using a quadratic FEM

```

1 function [x,T] = heat1dHO(TL, kappa, Ne);
2 % function [x,T] = heat1dHO(TL, kappa, Ne);
3 % PURPOSE:
4 %   Solves a 1D heat transfer problem using quadratic FEM.
5 % INPUTS:
6 %   TL    : temperature at the left end of the bar
7 %           the right of the bar is adiabatic
8 %   kappa : heat conductivity inside the bar
9 %   Ne    : number of elements
10 % OUTPUTS
11 %   x      : 2*Ne+1 by 1 vector of node locations
12 %   T      : 2*Ne+1 by 1 vector of temperatures at each node
13
14 dx = 1/Ne;           % element length (all the same)
15 x = linspace(0,1,2*Ne+1); % node positions
16
17 N = 2*Ne+1;          % # unknowns = # nodes + # elements
18 A = sparse(N,N); % stiffness matrix
19 F = zeros(N,1); % forcing vector (on LHS)
20
21 % elemental stiffness matrix
22 Ak = -2*kappa/dx * [7, -8, 1; -8, 16, -8; 1, -8, 7]/6;
23
24 % numerical integration (quadrature) rule
25 xiq = [-1;0;1]*sqrt(3/5); wq = [5,8,5]/9;
26
27 % Ref element basis functions at quadrature points
28 Phi = [0.5*xiq.*(xiq-1), 1-xiq.^2, 0.5*xiq.*(1+xiq)];
29
30 % set up system by looping over elements
31 for k = 1:Ne,
32
33     Ik = [2*k-1, 2*k, 2*k+1]; % unknowns associated with this element
34
35     A(Ik, Ik) = A(Ik, Ik) + Ak; % add to stiffness matrix
36
37     xq = (k-1)*dx + 0.5*(xiq + 1)*dx; % quad points in global space
38
39     F(Ik) = F(Ik) + Phi'*diag(wq)*sin(pi*xq)*dx/2; % numerical integration
40
41 end
42
43 % boundary conditions
44 A(1,:) = zeros(1,N); A(1,1) = 1; F(1) = -1; % Dirichlet on left
45 % adiabatic on right -> nothing to do for Neumann
46
47 % solve system
48 T = -A\F;

```

Running the code with $N_e = 8$ quadratic elements produces the plot shown in Figure 6.13 on the left. In fact, we are only plotting the nodal values of the quadratic solution, connected with straight lines. We could obtain a better picture if we subdivided each element into a larger number of sub-elements for plotting. Nevertheless, we can see that both boundary conditions again appear satisfied. A more quantitative comparison is shown in the error convergence plot in Figure 6.13, where the error is computed against the exact analytical solution. Note the *third*-order rate of convergence of the error. We can reason this from the fact that we are using quadratic polynomials to approximate our solution: the leading error

term will be cubic in Δx .

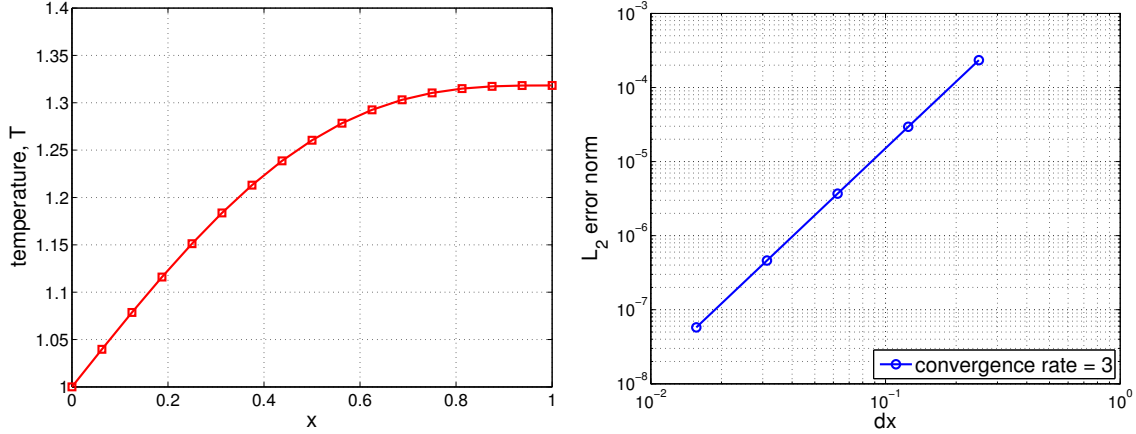


Figure 6.13: Sample quadratic finite element solution and convergence of the discrete L_2 error norm of the computed temperature distribution relative to the actual temperature distribution for Example 6.6.

6.7 Discontinuous Finite Elements

The linear and high-order finite element methods discussed up to this point are of the *continuous* variety, meaning that the approximated solution, $u(x)$ or $u(x, y)$, is a (piecewise) continuous function over the domain. Continuous finite elements are generally a great choice for differential equations that are diffusion-dominated: e.g. heat transfer or structural dynamics. However, they yield unstable discrete systems for convection-dominated equations, such as those that govern compressible fluid flows.

For convection-dominated equations, the finite volume method is the historically-preferred approach because of the stability brought about by the upwind-biased flux functions. Such upwinding cannot be easily applied to continuous finite elements because there is no ambiguity about the solution at element interfaces ... the solution is continuous there. How can we “fix” this? We can allow for discontinuities at element interfaces in the solution approximation, as illustrated in Figure 6.14¹. The resulting method is called Discontinuous Galerkin (DG), where “Galerkin” refers to the fact that the approximation and test space are going to be the same. Adding discontinuities in the solution approximation seems counter-intuitive from a stability point of view: we can imagine even wilder oscillations when jumps are possible between elements. However, the jumps bring about flexibility in choosing fluxes with

¹This is not the only fix – another popular approach is to add a specific form of diffusion to the continuous finite-element weak form to eliminate the unstable modes, and the resulting method is called the Streamwise-Upwind Petrov-Galerkin (SUPG).

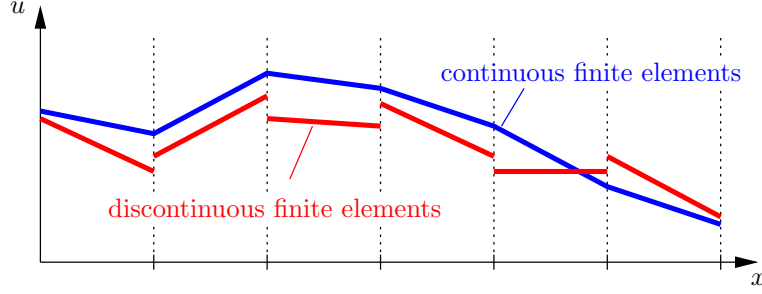


Figure 6.14: Comparison of linear continuous and discontinuous finite element solution approximations in one dimension.

an upwind bias and this improves stability. The discontinuities do increase the expense of the discretization because, for a given number of elements, it takes more numbers (degrees of freedom) to represent a discontinuous solution compared to a continuous solution.

6.7.1 Advection in One Dimension

The prototypical equation for convection-dominated flows is scalar advection. We'll consider a steady-state advection with a source term,

$$a \frac{du}{dx} + s(x) = 0, \quad x \in [0, L] \quad (6.20)$$

$$u(x=0) = u_0 \quad (6.21)$$

where $a > 0$ is a constant advection velocity, L is the length of the domain, $s(x)$ is a known source function, and u_0 is a specified value on the left boundary. Note that we do not need a boundary condition at $x = L$, the outflow.

We divide the domain into N equal elements of size $\Delta x = L/N$. On each element, k , we represent the solution as a polynomial. Just like in the continuous case, we'll write the polynomial using basis functions, $\phi_{k,j}(x)$. For an order p polynomial, we will need $p + 1$ basis functions, so $1 \leq j \leq p + 1$. We now have two indices on the basis functions: k for the element number and j for the index of the basis function within the element. So we write

$$\text{on element } k: u(x) = \sum_{j=1}^{p+1} U_{k,j} \phi_{k,j}(x). \quad (6.22)$$

To get the weak form, we use the discontinuous basis functions as test functions. Because of the discontinuities between elements, we cannot apply integration by parts to the entire domain all at once. Instead, we use integration by parts on a single element k . That is, we multiply the differential equation in Eqn. 6.20 by test functions $\phi_{k,i}$ and integrate the

advection term by parts over the element,

$$\begin{aligned}
\int_{x_k}^{x_{k+1}} \phi_{k,i} \left[a \frac{du}{dx} + s(x) \right] dx &= 0 \\
\int_{x_k}^{x_{k+1}} \phi_{k,i} a \frac{du}{dx} dx + \int_{x_k}^{x_{k+1}} \phi_{k,i} s(x) dx &= 0 \\
- \int_{x_k}^{x_{k+1}} \frac{d\phi_{k,i}}{dx} au dx + [\phi_{k,i} au]_{x_k}^{x_{k+1}} + \int_{x_k}^{x_{k+1}} \phi_{k,i} s(x) dx &= 0
\end{aligned} \tag{6.23}$$

The above weak form requires evaluations of au (the “flux”) on the element boundaries. However, the solution can be discontinuous between elements! To resolve this problem, we borrow an idea from the finite-volume method and use an *upwind* flux. That is, we set au on an element interface to au from the left element (since $a > 0$). So we write

$$\begin{aligned}
au|_{x_k} &= au_{k-1}(x_k), \\
au|_{x_{k+1}} &= au_k(x_{k+1}),
\end{aligned} \tag{6.24}$$

where $u_k(x)$ is the solution representation on element k . The resulting weak form on element k is

$$\begin{aligned}
- \int_{x_k}^{x_{k+1}} \frac{d\phi_{k,i}}{dx} au dx + \phi_{k,i}(x_{k+1}) au_k(x_{k+1}) - \phi_{k,i}(x_k) au_{k-1}(x_k) \\
+ \int_{x_k}^{x_{k+1}} \phi_{k,i} s(x) dx &= 0.
\end{aligned} \tag{6.25}$$

Note that the upwind flux *couples* solutions on adjacent elements because the right-most value of $u(x)$ on element $k - 1$ is required in the equations for element k . Without this coupling we would have N independent systems and we would never be able to solve the global problem. Also, for the first element, $k = 1$, the upwind flux is just au_0 ; i.e. it uses the boundary condition.

To obtain a system of equations, we substitute the solution approximation from Eqn. 6.22 into the weak form and make a linear system by evaluating integrals of the basis functions and their derivatives. These integrals are most easily done on a reference element. Depending on the form of the basis functions, the integrals may be evaluated analytically; an alternative is numerical integration. The integral of the source term is typically done with numerical integration. Both the source term and terms involving the known boundary condition will move to the right-hand-side of the linear system. After solving the system, we will have the solution coefficients $U_{k,j}$ from which we can plot the solution, or evaluate outputs and errors.

Chapter 7

Optimization

7.1 Introduction

In this chapter we discuss numerical methods associated with optimization, which is important to many engineering design applications. When presenting the methods, we will consider two types of systems. The first is a “toy problem,” in which a scalar output, J , is a simple function of n inputs, $\mathbf{x} \in \mathbb{R}^n$, as illustrated in Figure 7.1(a). In practical applications, however, the system to be optimized will not be as simple. For example, in numerical simulations of fluids or structures, we have the concept of a state, $\mathbf{U}$, which acts as an intermediary between the inputs and the output. This situation is illustrated in Figure 7.1(b). Note that we will often write $J(\mathbf{x})$ as shorthand for $J(\mathbf{U}, \mathbf{x})$, since the state $\mathbf{U}$ is itself a function of $\mathbf{x}$.

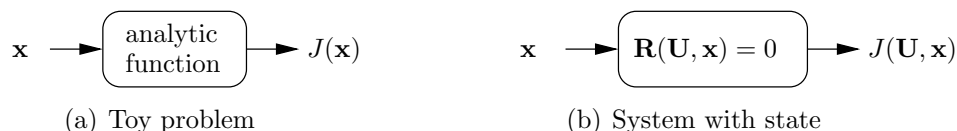


Figure 7.1: Input-to-output mapping of a simple “toy problem” in which the output J is an analytic function of the input vector, $\mathbf{x}$, as well as one of a system that involves a possibly-large state vector, $\mathbf{U}$.

We will deal with an optimization problem in which we try to minimize the output, i.e. the objective function, J , by finding the best possible input vector, $\mathbf{x}$. Note that a maximization problem could be easily handled in this context just by using $-J$ instead of J .

7.2 Gradient-Based Methods

Gradient-based optimization methods make use of a code’s ability to compute not only the output $J(\mathbf{x})$ but also the output gradient,

$$\text{output gradient} = \nabla J = \frac{\partial J}{\partial \mathbf{x}} = \text{a } 1 \times n \text{ vector.}$$

The general form of an iterative gradient-based optimization algorithm is given by the following steps (k is the iteration number):

1. Given $\mathbf{x}_k$ = solution from the previous iteration or guess at $k = 0$.
2. Compute a search direction, $\mathbf{p}_k \in \mathbb{R}^n$, using gradient information.
3. Compute a step length α_k such that

$$J(\mathbf{x}_k + \alpha_k \mathbf{p}_k) < J(\mathbf{x}_k)$$

4. Update the state according to

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k,$$

and return to step 1 with k incremented.

7.2.1 Steepest Descent

By definition, the gradient ∇J gives the direction in the input space in which J increases most rapidly. So the opposite direction, $-\nabla J$, should give us the direction in which J *decreases* most rapidly. To obtain a unit-length search direction, we normalize the gradient by defining,

$$\mathbf{p}_k = \frac{-(\nabla J)^T}{|\nabla J|}. \quad (7.1)$$

Note that the transpose on the gradient just converts a $1 \times n$ vector into an $n \times 1$ vector so that we can use it to increment $\mathbf{x}_k$. The normalization of the search direction in Eqn. 7.1 is solely out of convenience for use in the step-length determination. We do not lose any information here because the magnitude of the negative gradient $-\nabla J$ does not contain information about how far we have to go in that direction in input space to hit the minimum J . Instead, the step length α_k is computed via a line search, as discussed in Section 7.2.3.

Figure 7.2 illustrates the convergence of the steepest-descent for a two-dimensional input vector, $\mathbf{x} = (x_1, x_2)$, and an analytic $J(\mathbf{x})$. Note the “zig-zag” convergence towards the minimum, the severity of which depends on the starting location. This effect is due to the fact that, by construction, each search direction is orthogonal to the previous one, i.e. $\mathbf{p}_{k+1} \perp \mathbf{p}_k$. The anisotropy of the objective function with respect to parameters also plays a role; e.g. if the contours of $J(\mathbf{x})$ were circles around the minimum, steepest descent would converge in one step. However, general $J(\mathbf{x})$ will not be perfectly isotropic, and in these cases steepest descent performs rather poorly due to the zig-zag convergence towards the minimum.

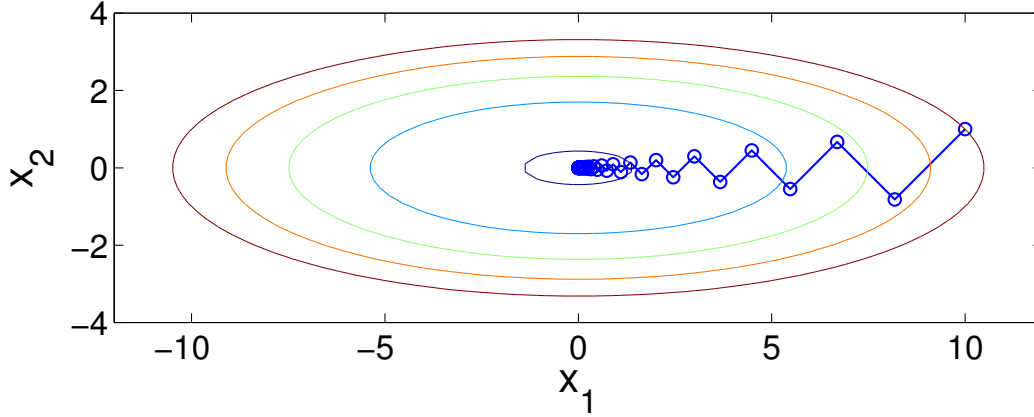


Figure 7.2: Steepest descent for $J(\mathbf{x}) = 0.5(x_1^2 + 10x_2^2)$, starting with $\mathbf{x}_0 = (10, 1)$.

7.2.2 Conjugate Gradients

When applicable, the method of conjugate gradients improves on steepest descent by keeping track of previous search directions to prevent the zig-zag convergence pattern. At the most basic level, the conjugate gradient method applies to minimizations of *quadratic* $J(\mathbf{x})$, which can be expressed as

$$J(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{b}^T \mathbf{x}.$$

where $\mathbf{A} \in \mathbb{R}^{n \times n}$ is a positive-definite matrix, and $\mathbf{b} \in \mathbb{R}^n$. The method extends to general convex $J(\mathbf{x})$, which locally near the minimum can be approximated as quadratic. Considering the linear system, $\mathbf{A} \mathbf{x} = \mathbf{b}$, the *residual* of this system turns out to be the negative gradient of our J ,

$$\mathbf{r} \equiv \mathbf{b} - \mathbf{A} \mathbf{x} = -\nabla J.$$

The conjugate gradient method begins like steepest descent: for $k = 0$ we set the first search direction (forgetting about normalization for the moment) equal to the negative gradient, $\mathbf{p}_0 = -\nabla J = \mathbf{r}_0$. Subsequent search directions, $\mathbf{p}_k$, also start out as the gradient, but “projections” (in the conjugate, not orthogonal, sense) of this gradient along previous directions are then taken out, according to the following formula:

$$\mathbf{p}_k = \underbrace{\mathbf{r}_k}_{\text{gradient of } J \text{ at } \mathbf{x}_k} - \underbrace{\sum_{i=0}^{k-1} \frac{\mathbf{p}_i^T \mathbf{A} \mathbf{r}_k}{\mathbf{p}_i^T \mathbf{A} \mathbf{p}_i} \mathbf{p}_i}_{\text{ensures } \mathbf{p}_k \text{ is conjugate to } \mathbf{p}_i}.$$

We say that two vectors $\mathbf{p}_i, \mathbf{p}_k$ are *conjugate* with respect to $\mathbf{A}$ if

$$\mathbf{p}_i^T \mathbf{A} \mathbf{p}_k = 0.$$

By making the newest search direction $\mathbf{p}_k$ conjugate to the previous search directions, $i < k$, we ensure that at the updated state $\mathbf{x}_{k+1}$ the gradient $\mathbf{r}_{k+1}$ will also be automatically conjugate to the previous directions, $i < k$. This means that we actually do not have to store the whole history of search directions; instead, just the current and previous one are enough.

The matrix $\mathbf{A}$ is the second derivative of J with respect to $\mathbf{x}$, also called the Hessian. If we have access to $\mathbf{A}$, or at least to matrix-vector products involving $\mathbf{A}$, we do not need to perform a line search to determine α_k at each step. Instead, taking advantage of the fact that J is quadratic in $\mathbf{x}$ the step length is

$$\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k}.$$

For general convex functions J , without access to $\mathbf{A}$, we would resort to a line search to determine α_k .

Figure 7.3 illustrates the rapid convergence of the conjugate gradient method. Comparing to steepest descent, we note that the zig-zag behavior is gone and that the method converges in just two iterations. In fact, for n dimensions, the conjugate gradient method will converge in at most n iterations.

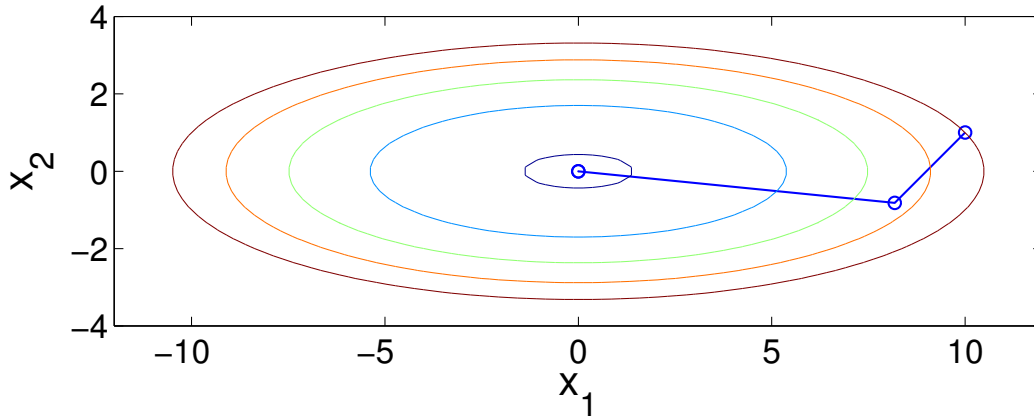


Figure 7.3: Conjugate gradient method for $J(\mathbf{x}) = 0.5(x_1^2 + 10x_2^2)$, starting with $\mathbf{x}_0 = (10, 1)$. The method converges in two steps.

7.2.3 Line Search Methods

In a gradient-based optimization method, the step length α_k used in the update

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$$

should be chosen wisely to yield the largest possible decrease in J . That is, we'd like to find α_k for which

$$J(\mathbf{x}_k + \alpha_k \mathbf{p}_k) \text{ is minimum.}$$

Some gradient-based methods, such as Newton’s method, generally already return a well-scaled search direction for which we would expect $\alpha_k \approx 1$. However, steepest descent or the general form of conjugate gradients returns search vectors whose magnitudes do not give us information about α_k . It is in these cases that a line search is most useful.

A typical plot of J versus α_k is given in Figure 7.4. In a line search, we usually start with a guess for α_k , either from some prior information or from a (not too wild) estimate.

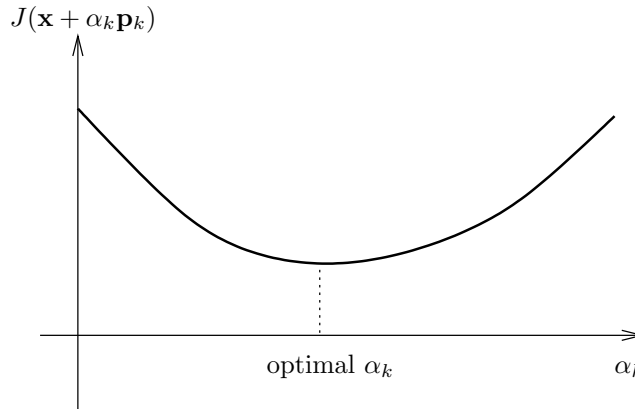


Figure 7.4: Typical output behavior with step length.

If we start with a conservative estimate for α_k we can try ramping it up, e.g. by factors of 2, until $J(\mathbf{x}_k + \alpha_k \mathbf{p}_k)$ begins to increase. The last α_k before the increase would then be our answer. We could refine this answer by “zooming in” in the neighborhood of α_k , depending on how much effort we were willing to expend on finding the precise minimizer of J . Conversely, if we start with a large estimate of α_k , or if the ramping up never causes a decrease of J , then we could *backtrack* by dividing α_k by a factor, say 2, until J begins to increase. The last value of α_k before this increase would then be our answer.

More sophisticated line search techniques exist; for example, in some methods quadratic functions are fit to J evaluations at three recent α_k points, in an effort to efficiently and accurately determine the minimizing α_k . These fits can be powerful, but they do rely on a certain degree of smoothness in $J(\mathbf{x})$.

7.2.4 Computing Gradients

For gradient-based optimization methods we need to know how J changes with $\mathbf{x}$,

$$\frac{dJ}{d\mathbf{x}} \in \mathbb{R}^{1 \times n} = n \text{ sensitivities.}$$

These are constant if the equations and output are linear. Otherwise $\frac{dJ}{d\mathbf{x}}$ changes with $\mathbf{x}$. For practical problems with large state vectors $\mathbf{U}$, the output J cannot be written in closed form as a function of the inputs and the gradient may be considerably expensive to compute.

7.2.4.1 Finite Differences

A simple way to obtain (approximate) derivatives is to use finite differencing. For each input, x_i , $1 \leq i \leq n$, we calculate

$$\frac{dJ}{dx_i} \approx \frac{J(x_1, x_2, \dots, x_i + \epsilon, \dots, x_n) - J(x_1, x_2, \dots, x_i, \dots, x_n)}{\epsilon}$$

for some “small” choice of the scalar ϵ . This calculation is easy to implement but expensive for large n when each J calculation requires solving a large system of equations for the state $\mathbf{U}$. Conversely, it is efficient when we have small n but many outputs J .

Choosing ϵ for finite differencing is not easy, as it cannot be too small (machine precision errors) or too large (finite difference discretization errors). A good choice for ϵ that works in many cases is the square root of machine precision. The *complex step method* resolves the “too small” problem by using ϵi , where $i = \sqrt{-1}$, assuming that J can be evaluated for complex numbers.

7.2.4.2 Adjoint Methods

For systems of equations in which there is a state governed by N residuals, $\mathbf{R}(\mathbf{U}, \mathbf{x}) = 0$, we can use the *adjoint method* to precompute the expensive step in finite differencing, which is the sensitivity of J to the N residuals. We call this sensitivity $\Psi \in \mathbb{R}^N$, so that by the chain rule, the required sensitivities are

$$\frac{dJ}{d\mathbf{x}} = \Psi^T \frac{\partial \mathbf{R}}{\partial \mathbf{x}}.$$

In this expression we assume that J does not explicitly depend on $\mathbf{x}$; if it does we just add the linearization of this dependence to the right-hand side of the above equation. The adjoint method is efficient for computing a large number of sensitivities for one output, at an incremental cost of one residual calculation and one vector product per sensitivity. This is useful when the *forward problem* ($\mathbf{x} \Rightarrow J$) is very expensive.

Let’s take a closer look at how the adjoint method works. The key idea underpinning this method is that we do not need to solve the forward problem each time we want a sensitivity.

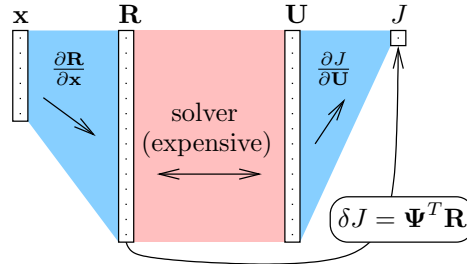


Figure 7.5: An adjoint method precomputes the expensive portion of the input-to-output calculation.

Suppose that given $\mathbf{x}$ we find $\mathbf{U}$ such that $\mathbf{R}(\mathbf{U}, \mathbf{x}) = 0$. Now perturb $\mathbf{x} \rightarrow \mathbf{x} + \delta\mathbf{x}$. What is the effect on J ? Well, we can re-solve for the state, but this is expensive if we have to do it many times. Instead, we can separate the effects of $\mathbf{x}$ on $\mathbf{R}$, and $\mathbf{R}$ on J . The adjoint method precomputes the effect of $\mathbf{R}$ on J , which is the expensive step. The resulting N sensitivities are stored in the vector Ψ .

The chain of operations in the adjoint method is as follows:

Assuming small perturbations

1. Input perturbation $\mathbf{x} \rightarrow \mathbf{x} + \delta\mathbf{x}$
2. Residual perturbation $\mathbf{R}(\mathbf{U}, \mathbf{x} + \delta\mathbf{x}) = \delta\mathbf{R} \neq 0 \rightarrow \mathbf{R}(\mathbf{U}, \mathbf{x}) + \left. \frac{\partial \mathbf{R}}{\partial \mathbf{x}} \right|_{\mathbf{U}, \mathbf{x}} \delta\mathbf{x} = \delta\mathbf{R}$
3. State perturbation $\mathbf{R}(\mathbf{U} + \delta\mathbf{U}, \mathbf{x} + \delta\mathbf{x}) = 0 \rightarrow \mathbf{R}(\mathbf{U}, \mathbf{x}) + \left. \frac{\partial \mathbf{R}}{\partial \mathbf{x}} \right|_{\mathbf{U}, \mathbf{x}} \delta\mathbf{x} + \left. \frac{\partial \mathbf{R}}{\partial \mathbf{U}} \right|_{\mathbf{U}, \mathbf{x}} \delta\mathbf{U} = 0$
4. Output perturbation $J(\mathbf{U} + \delta\mathbf{U}) = J(\mathbf{U}) + \delta J \rightarrow \delta J = \left. \frac{\partial J}{\partial \mathbf{U}} \right|_{\mathbf{U}, \mathbf{x}} \delta\mathbf{U}$

Subtracting step 2 from step 3 and assuming small disturbances, we obtain

$$\left. \frac{\partial \mathbf{R}}{\partial \mathbf{U}} \right|_{\mathbf{U}, \mathbf{x}} \delta\mathbf{U} = -\delta\mathbf{R} \Rightarrow \delta\mathbf{U} = - \left[\left. \frac{\partial \mathbf{R}}{\partial \mathbf{U}} \right|_{\mathbf{U}, \mathbf{x}} \right]^{-1} \delta\mathbf{R}.$$

Combining this result with the output linearization in step 4 to express the output perturbation in terms of the residual perturbation, we have

$$\delta J = \left. \frac{\partial J}{\partial \mathbf{U}} \right|_{\mathbf{U}, \mathbf{x}} \delta\mathbf{U} = - \underbrace{\left. \frac{\partial J}{\partial \mathbf{U}} \right|_{\mathbf{U}, \mathbf{x}} \left[\left. \frac{\partial \mathbf{R}}{\partial \mathbf{U}} \right|_{\mathbf{U}, \mathbf{x}} \right]^{-1}}_{\Psi^T \in \mathbb{R}^n} \delta\mathbf{R}. \quad (7.2)$$

Taking the transpose of the equation $\Psi^T = - \left. \frac{\partial J}{\partial \mathbf{U}} \right|_{\mathbf{U}, \mathbf{x}} \left[\left. \frac{\partial \mathbf{R}}{\partial \mathbf{U}} \right|_{\mathbf{U}, \mathbf{x}} \right]^{-1}$ and moving everything to the left-hand side gives the *adjoint equation*,

$$\boxed{\left(\left. \frac{\partial \mathbf{R}}{\partial \mathbf{U}} \right|_{\mathbf{U}, \mathbf{x}} \right)^T \Psi + \left(\left. \frac{\partial J}{\partial \mathbf{U}} \right|_{\mathbf{U}, \mathbf{x}} \right)^T = 0}$$

The i^{th} component of Ψ is the sensitivity of J to changes in the i^{th} residual.

Since $\mathbf{R}(\mathbf{U}, \mathbf{x}) = 0$, from step 2 above we have $\delta\mathbf{R} = \left. \frac{\partial \mathbf{R}}{\partial \mathbf{x}} \right|_{\mathbf{U}, \mathbf{x}} \delta\mathbf{x}$ and (7.2) becomes

$$\delta J = \Psi^T \left. \frac{\partial \mathbf{R}}{\partial \mathbf{x}} \right|_{\mathbf{U}, \mathbf{x}} \delta\mathbf{x} \Rightarrow \boxed{\frac{dJ}{d\mathbf{x}} = \Psi^T \left. \frac{\partial \mathbf{R}}{\partial \mathbf{x}} \right|_{\mathbf{U}, \mathbf{x}}}$$

Therefore, once we have Ψ , no more solves are required for new sensitivities for the same output. Note that the calculation of $\left. \frac{\partial \mathbf{R}}{\partial \mathbf{x}} \right|_{\mathbf{U}, \mathbf{x}}$ is cheap compared to a forward solve.

7.3 Gradient-Free Methods

In certain optimization problems, gradients of the objective function are not readily available, e.g. due to the complexity of the solver or to a non-differentiable nature of $J(\mathbf{x})$. For such cases, gradient-based methods are no longer applicable. Yet even by only knowing how to evaluate $J(\mathbf{x})$, we can still solve optimization problems. In this section we briefly review two popular options for gradient-free optimization.

7.3.1 Nelder-Mead/Simplex Method

The (nonlinear) simplex method can be used for a problem with a small or moderate number of inputs. This method defines a *simplex* structure, which in n -dimensional space consists of $n + 1$ points – think of a line segment in 1-d space, a triangle in 2-d space, a tetrahedron in 3-d space, etc. At each vertex of the simplex, we evaluate J , and iterations consist of changing the simplex through simple operations by comparing the relative values of J at the vertices – no gradient information is required. The operations that modify the simplex consist of reflection, contraction, expansion, and shrinking. Upon convergence, the simplex will ideally enclose a small volume containing the optimal solution, so that the final $\mathbf{x}$ can be estimated as the centroid of the simplex.

7.3.2 Genetic Algorithms

A genetic algorithm iterates a population of $\mathbf{x}$ values to locate the minimizer of J . The use of a population, as opposed to a single iterate as in gradient-based methods, makes genetic algorithms less likely to get stuck in local extrema. However, iterating a large population requires many evaluations of the output J , which can be expensive for large systems of equations.

The three basic properties of a genetic algorithm are selection, crossover, and mutation. Selection refers to the idea that members of the population who are close to the optimum (have a low J) are more likely to survive to a subsequent generation. Crossover refers to the fact that new members of the population take on “traits” of existing population members, in an effort to preserve optimal parameter combinations. Finally, mutation refers to an inherent randomness in which traits, or parameter combinations, are modified during the generation of new population members.

Genetic algorithms are useful for *multi-objective* optimization in which we do not have a single output but instead want to study the trade-off between multiple outputs (e.g. range versus cost of an aircraft design). This is because of the availability of a large population of points in parameter space, which can be used to plot *Pareto fronts* on axes of different outputs.